

边缘智能管理平台

v2.0

技术白皮书

Edge Platform Team

2026年4月

目 录

边缘智能管理平台技术白皮书

核心观点

章节导读

阅读建议

第一章：执行摘要

- 1.1 边缘计算的价值悖论
- 1.2 产品定位
- 1.3 核心价值主张
- 1.4 关键技术指标
- 1.5 技术架构概览
- 1.6 应用生命周期全景
- 1.7 与传统方案的本质区别
- 1.8 阅读路径建议

第二章：从硬件投入到应用价值 — 边缘计算的范式转移

- 2.1 边缘计算的成本与价值错配
- 2.2 应用价值的持续释放特征
- 2.3 为什么 Kubernetes 是应用价值释放的最佳载体
- 2.4 传统 IoT 能力的云原生重生
- 2.5 边缘计算的四代演进
- 2.6 本章小结：白皮书的阅读地图

第三章：基础设施就绪 — 应用运行的基石

- 引言：应用的第一个前提是"地基"
- 3.1 硬件快速接入 — OTA 批量运维系统
- 3.2 集群联邦管理
- 3.3 可插拔边缘运行时
- 3.4 分层镜像分发
- 3.5 本章小结

第四章：算力就绪 — 让应用无需感知硬件差异

- 引言：算力是手段，应用是目的
- 4.1 九大 GPU 厂商统一建模

- 4.2 四层 CRD 抽象架构
- 4.3 GPU 虚拟化 — 双模切分
- 4.4 全局调度策略
- 4.5 设备状态与健康管理
- 4.6 国产化适配策略
- 4.7 本章小结

第五章：组织与安全 — 多团队安全共享应用平台

- 引言：应用越多，权限越关键
- 5.1 多租户隔离架构
- 5.2 六层权限模型
- 5.3 认证与审计
- 5.4 数据安全
- 5.5 与竞品的差异
- 5.6 本章小结

第六章：应用供应链 — 从开发到上架

- 引言：应用的"最后一公里"不是部署，是上架
- 6.1 三层应用资源模型
- 6.2 可插拔 Provisioner 架构
- 6.3 应用商店与审核流程
- 6.4 版本管理与资源所有权
- 6.5 本章小结

第七章：应用部署 — 到达每一个边缘

- 引言：部署不是一次性操作，是持续管理
- 7.1 Topology-Based 云边混合部署
- 7.2 部署状态聚合与追踪
- 7.3 版本管理与变更历史
- 7.4 Label 体系与资源关联
- 7.5 删除与清理
- 7.6 端到端部署场景示例
- 7.7 本章小结

第八章：应用运行 — 断网不断服务

- 引言：边缘的常态是"不完美"
- 8.1 边缘离线自治
- 8.2 IoT 设备交互
- 8.3 边缘节点管理
- 8.4 本章小结

第九章：应用可观测 — 看得见才管得住

引言：边缘的监控比云端更难

9.1 三层监控架构

9.2 多租户指标隔离

9.3 算力与设备监控

9.4 告警管理

9.5 全局拓扑可视化

9.6 本章小结

第十章：平台架构设计 — 统一的技术底座

引言：一个平台，不是七个系统

10.1 整体架构全景

10.2 七层 Filter Chain 架构

10.3 RequestInfo：请求的语义化解析

10.4 多集群请求路由

10.5 CRD 资源模型

10.6 Controller 设计模式

10.7 API 注册与 Installer 模式

10.8 OpenAPI 驱动的全链路类型安全

10.9 本章小结

第十一章：竞品对比与优势分析

引言：对比的标尺

11.1 竞品全景

11.2 八维度竞品矩阵

11.3 深度对比：逐家分析

11.4 总结：产品哲学的差异

11.5 适用场景选择建议

11.6 本章小结

第十二章：典型行业场景 — 应用价值的行业映射

引言：从技术能力到行业价值

12.1 智能制造：质检 AI 从一条产线到全国工厂

12.2 智慧城市：多部门应用的安全共存

12.3 电信 5G MEC：万级站点的应用统一管理

12.4 金融网点：离线场景下的交易连续性

12.5 能源/电力：异构设备上的监控应用快速部署

12.6 AI 推理服务：算法镜像的应用商店化分发

12.7 本章小结：六个行业，一个模式

边缘智能管理平台技术白皮书

本白皮书面向技术决策者、架构师和高级工程师，系统阐述边缘智能管理平台的核心设计理念、技术架构和价值主张。

核心观点

边缘计算平台的设计哲学必须从"面向硬件"转向"面向应用"。平台无法改变硬件的采购成本，但可以显著改变应用在硬件上释放价值的效率和速度。

章节导读

章节	标题	核心主题
第一章	执行摘要	核心价值主张与关键技术差异，5分钟快速了解平台全貌
第二章	从硬件投入到应用价值	边缘计算的范式转移：成本集中在硬件，价值产生在应用
第三章	基础设施就绪	多运行时集群管理、节点组调度、断网自治
第四章	算力就绪	GPU/NPU 异构算力池化、vGPU切分与调度
第五章	组织与安全	多租户隔离、RBAC 权限模型、安全策略
第六章	应用供应链	应用商店、版本审核、Helm Chart 管理
第七章	应用部署	拓扑调度、滚动升级、边缘分发
第八章	应用运行	断网不断服务、边缘自治、运行时保障
第九章	应用可观测	集中监控、告警管理、趋势分析
第十章	平台架构设计	API 架构、CRD 设计、控制器模式、扩展机制
第十一章	竞品对比与优势分析	与同类平台的技术对比和差异化优势
第十二章	典型行业场景	智能制造、智慧交通、能源电力等行业案例

阅读建议

技术决策者：建议阅读第一章（执行摘要）和第二章（价值主张），快速了解平台定位。

架构师：建议完整阅读第三至十章，深入理解平台的技术设计决策。

产品经理：建议阅读第十一章（竞品对比）和第十二章（行业场景），了解市场定位。

第一章：执行摘要

本章为快速阅读通道。技术决策者可以在 5 分钟内获取平台的核心价值主张和关键技术差异。

1.1 边缘计算的价值悖论

边缘计算正在从概念验证走向规模部署。然而，多数企业在推进边缘项目时遭遇了同一个结构性矛盾：

硬件占据了 **70-80%** 的项目预算，但 **100%** 的业务价值来自运行在硬件上的应用。

一台配置了 NVIDIA A100 的边缘服务器，闲置在机柜里和一块砖头没有区别。只有当质检模型在上面运行、每天拦截数百件瑕疵品时，这台服务器才开始回报投资。

传统边缘/IoT 项目“重前期硬件、轻后期应用”的运营模式，导致应用的价值释放速度远低于预期——硬件部署 3 个月完成，应用推广却花了 18 个月。

核心结论：边缘计算平台的设计哲学必须从“面向硬件”转向“面向应用”。平台无法改变硬件的采购成本，但可以显著改变应用在硬件上释放价值的效率和速度。

1.2 产品定位

基于 **Kubernetes** 应用抽象构建的云原生边缘智能管理平台，让 **AI** 应用和业务服务在异构边缘基础设施上持续释放价值。

平台以应用的全生命周期为核心组织所有技术能力。每一项技术能力都不是孤立的功能点，而是应用在某个生命周期阶段面对的具体问题的解法。

1.3 核心价值主张

应用即价值

应用商店 + 审核上架 + 一键分发到任意边缘，让应用的价值随业务演进持续释放。

- 三层应用资源模型：Application → ApplicationVersion → ApplicationDeployment，应用定义与部署实例完全解耦
- 可插拔 **Provisioner**：Workload（标准容器）、Helm（复杂多组件）、Model（AI 推理）三种部署引擎，统一管理接口
- 应用商店审核机制：Workspace 开发 → 平台审核 → 全域可见，借鉴 App Store 模式实现跨团队应用共享
- **Topology-Based** 部署：一个声明式资源管理应用在所有边缘站点的部署，50 个站点的推广与 5 个站点的操作步骤完全相同

算力为应用服务

9 大 GPU 厂商统一调度，应用无需感知底层硬件差异。

- 四层 **CRD** 抽象：DeviceModel（硬件规格）→ ComputeTemplate（切分配比）→ ResourcePool（资源池）→ NodeConfig（节点配置），从物理设备到可分配算力的完整管线
- 双模 **GPU** 虚拟化：MIG 硬切分（硬件级隔离）+ HAMi-Core 软切分（全厂商通用），GPU 利用率从 <50% 提升到 ~92%
- **9 大厂商覆盖**：NVIDIA、海光、昇腾、沐曦、昆仑芯、平头哥、天数智芯、燧原、寒武纪，应用定义不包含任何硬件细节

管得住

五层权限模型，不同团队安全共享应用平台。

- 五层 **Scope** 授权：Platform → Cluster → Workspace → NodeGroup → Namespace，映射企业组织架构
- 授权缓存优化：TTL 300s + CRD 变化触发失效，兼顾性能与一致性
- 内置 **OAuth2/OIDC**：JWT Token 生命周期管理、LoginRecord 审计记录、bcrypt 密码加密

断不了

边缘离线自治，应用持续运行不受网络影响。

- 三种离线方案：KubeEdge（嵌入式边缘）、OpenYurt（标准服务器边缘）、VCluster（独立边缘集群），适配不同场景
- 可插拔运行时：ComponentInstaller 接口 + init() 注册，新增运行时只需一个文件
- 分层镜像缓存：Registry → ClusterRegistry → Repository → Node，四级分发确保弱网环境下镜像可用

1.4 关键技术指标

维度	指标
集群管理	支持 1000+ 集群联邦管理
节点规模	单集群 10000+ 节点
GPU 厂商	9 大厂商原生支持
GPU 虚拟化	单卡最多 20 份切分
应用管理	三种 Provisioner 覆盖全谱系应用类型
监控指标	291+ 预定义指标
权限层级	5 层 Scope 授权模型
CRD 体系	5 个 API 组、20+ CRD 类型

1.5 技术架构概览



1.6 应用生命周期全景

平台以应用的全生命周期为主线组织所有技术能力：

阶段	应用面对的问题	平台的解法	详见
基础设施就绪	硬件怎么快速接入？集群怎么建？	OTA 批量接入 + 集群联邦 + 分层镜像分发	第三章
算力就绪	异构 GPU 怎么统一使用？	四层 CRD 抽象 + 双模虚拟化	第四章
组织与安全	多团队怎么安全共享平台？	5 层权限模型 + 多租户隔离	第五章
应用供应链	应用怎么打包？怎么审核上架？	Provisioner 插件 + 应用商店审核	第六章
应用部署	怎么部署到多个边缘？	Topology-Based 部署 + 批量版本切换	第七章
应用运行	断网了怎么办？	边缘离线自治 + IoT CRD 云原生化	第八章
应用可观测	怎么知道应用健康不健康？	291+ 指标 + 多租户隔离 + 告警	第九章

1.7 与传统方案的本质区别

传统边缘平台讲的故事：我们支持什么硬件、管理多少节点、提供什么运行时。

本平台讲的故事：你的应用在边缘的每个阶段都有解法——打包、审核、部署、运行、监控——无论底层硬件怎么变。

这不是措辞差异，而是产品哲学的根本不同：

维度	传统边缘平台	本平台
核心抽象	设备/节点/集群	应用/版本/部署
技术组织方式	按基础设施分类	按应用生命周期阶段组织
GPU 管理视角	"支持多少种 GPU"	"应用的算力需求如何被透明满足"
部署模型	每个站点独立操作	一个声明覆盖所有站点
权限模型	租户 + 项目二级	Platform Cluster Workspace NodeGroup Namespace 五级
应用共享	无内置机制	应用商店 + 审核 + 全域可见

1.8 阅读路径建议

角色	推荐阅读路径
CTO / CIO / 技术决策者	第一章（本章） 第二章（立论） 第十一章（竞品对比）
架构师	第二章 第三至九章（应用全生命周期） 第十章（架构设计）
运维工程师	第三章（基础设施） 第七章（部署） 第九章（可观测）
AI / 算力工程师	第四章（算力就绪） 第六章（模型部署） 第九章（GPU 监控）
安全 / 合规负责人	第五章（组织与安全） 第十章（架构设计）

下一章：第二章 — 从硬件投入到应用价值，边缘计算的范式转移

第二章：从硬件投入到应用价值 —— 边缘计算的范式转移

本章是白皮书的立论基础。后续所有章节的技术能力，都是对这一章核心论点的展开论证。

2.1 边缘计算的成本与价值错配

边缘计算正在从概念验证走向规模部署。然而，多数企业在推进边缘项目时，遭遇了同一个结构性矛盾：成本集中在硬件，价值产生在应用。

传统边缘投入的成本结构

一个典型的边缘计算项目，其一次性投入通常包括：边缘服务器采购（含 GPU/NPU 加速卡）、网络设备与专线部署、机房改造与环境适配、现场安装与初始化调试。根据行业实践，这些硬件相关支出占项目总预算的 70-80%。

但硬件本身不产生业务价值。一台配置了 NVIDIA A100 的边缘服务器，闲置在机柜里和一块砖头没有区别。只有当质检模型在上面运行、每天拦截数百件瑕疵品时，这台服务器才开始回报投资。

边缘计算项目的成本 - 价值结构	
硬件投入（一次性沉没成本）	应用运行（持续价值释放）
■	▨
占总预算 70-80%	产生业务价值 100%
· 边缘服务器 & GPU 卡	· AI 推理：质检/安防/巡检
· 网络设备 & 专线	· 数据处理：清洗/聚合/预判
· 机房改造 & 环境适配	· 业务逻辑：调度/控制/告警
· 安装调试 & 初始化	· 模型迭代：v1→v2→v3 持续
矛盾：成本在左边一次性锁定，价值在右边持续递增	

"重前期、轻后期"的陷阱

传统边缘/IoT 项目的运营模式普遍呈现"重前期、轻后期"特征：

- 前期：投入大量预算采购硬件、部署网络、搭建环境。项目团队的注意力集中在“设备不能上线”。
- 后期：应用的开发、部署、更新和运维缺少系统化工具支撑。应用上线后，版本更新靠手工、故障排查靠现场、跨站点推广靠复制。

这种模式的直接后果是：硬件部署完成后，应用的价值释放速度远低于预期。一家制造企业花费 3 个月完成了 50 个工厂的边缘服务器部署，却用了 18 个月才把质检模型推广到其中 30 个工厂——因为每个工厂都需要独立配置、独立部署、独立调试。

成本结构决定了平台的设计哲学

认识到这一结构性矛盾之后，结论是清晰的：边缘计算平台的设计哲学必须从“面向硬件”转向“面向应用”。

硬件是沉没成本（Sunk Cost），一旦采购部署，其投入已经不可逆。平台无法改变硬件的采购成本，但可以显著改变应用在硬件上释放价值的效率和速度。因此，平台的核心使命不是“管理更多设备”，而是“让应用在每个边缘节点上更快地产生业务价值”。

2.2 应用价值的持续释放特征

与硬件的一次性投入不同，应用的价值具有持续释放、逐步递增的特征。理解这一特征，是理解平台设计决策的关键。

特征一：模型迭代带来价值倍增

AI 推理是边缘计算最重要的应用场景之一。以工业质检为例：

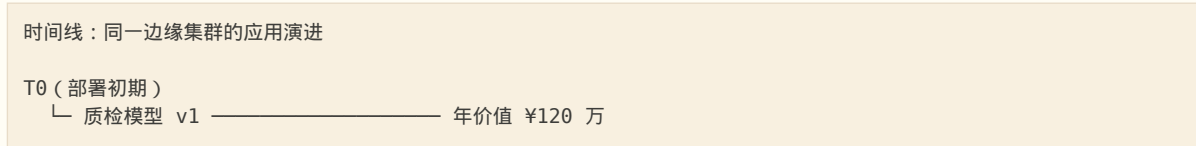
版本	准确率	每日拦截废品	年度节省成本
模型 v1（初始上线）	85%	~200 件	¥120 万
模型 v2（3 个月后）	92%	~350 件	¥210 万
模型 v3（6 个月后）	97%	~480 件	¥290 万

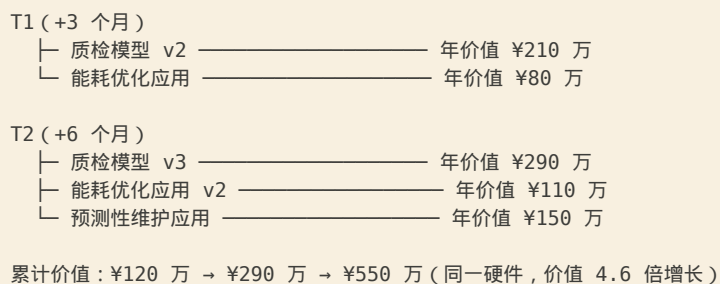
同样的硬件，同样的边缘服务器，随着模型的迭代优化，业务价值提升了 2.4 倍。这就是应用价值的持续释放——硬件不变，价值倍增。

但这种价值释放有一个前提：平台必须能够高效地完成模型版本的打包、审核、分发和批量升级。如果每次模型更新都需要运维人员逐台登录边缘服务器手动替换，那么一年三次迭代就是极限；如果平台支持一键批量版本切换，一年可以迭代数十次。

特征二：场景扩展带来价值叠加

一个边缘集群通常不只运行一个应用。随着业务演进，同一组硬件上的应用组合会不断扩展：





这种应用组合效应（Application Portfolio Effect）意味着：单个应用的价值 < 多个应用协同运行的总价值。平台需要支持多应用并行部署、资源隔离、独立版本管理，才能最大化每个边缘节点的业务回报。

特征三：规模推广带来价值复制

当一个应用在试点站点验证成功后，下一步是推广到所有类似站点。这是应用价值释放的乘数效应：

- 1 个工厂验证成功 → 50 个工厂批量部署
- 1 个城市试点 → 全国 200 个站点同步推广
- 1 个算法团队开发 → 10 个业务团队共享使用

从试点到规模推广的速度，直接决定了企业的投资回报周期。传统方式下，推广一个应用到新站点需要“安装环境 → 手动部署 → 配置调试 → 功能验证”的完整流程，平均耗时数天。平台化方式下，通过应用商店审核上架 + 声明式拓扑部署，推广周期可以压缩到分钟级别。

小结：平台的核心价值指标

综合以上三个特征，评估一个边缘计算平台的核心指标不应该是“管理了多少设备”或“支持多少种硬件”，而是：

1. 应用迭代效率：一次模型更新从开发到全量部署需要多久？
2. 应用推广速度：一个验证成功的应用推广到 N 个站点需要多久？
3. 应用并行能力：同一硬件上能否安全地运行多个团队的多个应用？
4. 应用运维成本：跨站点的应用监控、故障排查需要多少人力？

本白皮书后续章节将围绕这四个指标，逐一展开平台的技术解法。

2.3 为什么 Kubernetes 是应用价值释放的最佳载体

既然平台的核心使命是"让应用在边缘高效释放价值"，那么选择什么技术基座来承载应用，就成为最关键的架构决策。

Kubernetes 的应用抽象优势

Kubernetes 已经成为云原生应用管理的事实标准。它提供的抽象层天然契合应用价值驱动的设计哲学：

Kubernetes 应用抽象	解决的问题	对应用价值释放的意义
声明式 API (Declarative API)	应用描述"期望状态是什么"，而非"操作步骤是什么"	应用定义与底层硬件完全解耦，换硬件不改应用
工作负载模型 (Workload)	Deployment/StatefulSet/DaemonSet 覆盖主流部署模式	标准化的应用生命周期管理，无需为每种应用定制部署逻辑
Helm Chart	应用打包、版本管理、参数化配置	应用可以像商品一样被打包、上架、分发
Namespace 隔离	逻辑层面的资源隔离与访问控制	多团队可以安全地共享同一个集群，独立管理各自的应用
自愈机制 (Self-healing)	容器崩溃自动重启、节点故障自动迁移	边缘断网场景下，应用自动恢复，减少人工干预

关键推论：面向未来的应用投资

基于 Kubernetes 构建的平台有一个根本性优势：应用的定义不随硬件的更迭而改变。

- GPU 从 NVIDIA 换成昇腾 (Ascend) → 应用的 Deployment YAML 不变，只有底层的 Device Plugin 和调度策略需要适配
- 边缘运行时从 KubeEdge 换成 OpenYurt → 应用部署方式不变，只有集群组件层面的切换

- 监控从 Prometheus 升级为 VictoriaMetrics → 应用的监控指标定义不变，只有采集层替换这意味着企业在应用层面的投资（模型开发、业务逻辑、运维经验）是可积累、可复用、可延续的，不会因为硬件换代或底层技术栈演进而报废。

本平台对 **Kubernetes** 应用抽象的扩展

标准 Kubernetes 的应用管理能力在边缘场景下存在明显的空白。本平台在 K8s 原生抽象的基础上，进行了针对性的扩展：

边缘场景需求	K8s 原生能力	本平台扩展
应用需要打包审核后上架	无内置机制	Application CRD + 应用商店 + ReviewPhase 审核流程
一个应用部署到多个边缘	需手动创建多个 Deployment	Topology-Based 部署，一个 ApplicationDeployment 自动管理多个 Topology
不同类型应用需要不同部署引擎	手动选择 Deployment/StatefulSet	可插拔 Provisioner（Workload/Helm/Model），统一抽象
异构 GPU 的统一调度	无内置设备管理	四层 CRD 抽象：DeviceModel ComputeTemplate ResourcePool NodeConfig
多层次组织权限	单集群 RBAC	五层 Scope 权限模型：Platform Cluster Workspace NodeGroup Namespace
大规模镜像分发	依赖外部 Registry	四级分发链路：总部 云端 边缘缓存 节点

这些扩展遵循一个统一的设计原则：通过 **CRD（Custom Resource Definition）** 扩展 **Kubernetes** 的资源模型，所有管理操作通过标准的 **K8s API** 完成。应用开发者和运维人员使用他们已经熟悉的声明式 API 范式，无需学习私有管理接口。

2.4 传统 IoT 能力的云原生重生

边缘计算的前身是 IoT（Internet of Things，物联网）。传统 IoT 领域积累了大量成熟的技术能力：物模型（Thing Model）、设备管理（Device Management）、消息路由（Message Routing）、协议适配（Protocol Adaptation）。

一个常见的疑问是：选择 **Kubernetes** 技术路线，是否意味着放弃这些 **IoT** 领域的成熟积累？答案是否定的。这些能力并未被放弃，而是以云原生的方式被重新实现。

从私有 SDK 到标准 API

传统 IoT 平台的技术栈通常是厂商私有的：

传统 IoT 技术栈（厂商锁定）

物模型定义 → 厂商私有格式 → 绑定特定平台 SDK
设备注册 → 私有协议 → 换平台需重写对接代码
消息路由 → 私有消息总线 → 无法与 K8s 生态集成
应用部署 → 手工安装脚本 → 无法规模化推广

云原生边缘技术栈则基于开放标准：

云原生边缘技术栈（开放标准）

物模型定义 → EdgeX Foundry Device Profile → 标准 YAML/JSON 格式
设备注册 → Device CRD（K8s 自定义资源）→ kubectl / K8s API 统一操作
消息路由 → EdgeX Message Bus / MQTT Broker → 标准协议，可替换实现
应用部署 → Helm Chart / K8s Deployment → 声明式，一键分发

关键区别：逻辑抽象 vs 厂商实现

两种技术路线的本质区别不在于功能覆盖面，而在于抽象层级。

传统 IoT 平台的能力是建立在厂商私有实现之上的——使用某家的物模型格式，就被锁定在这家的 SDK 和平台上。一旦需要更换平台，所有对接工作需从头开始。

云原生边缘的能力是建立在逻辑抽象之上的——Device Profile 是一个标准的数据结构定义，Device CRD 是一个标准的 Kubernetes 资源。底层的实现可以是 EdgeX Foundry，可以是 KubeEdge Device Plugin，也可以是其他任何遵循同一接口的实现。

本平台在 IoT 能力上的设计选择正是基于这一判断：

IoT 能力	传统实现	本平台实现	可替换性
设备模型	厂商私有 SDK	DeviceModel CRD + DeviceProfile CRD	任何符合 CRD Schema 的实现
设备管理	私有管理协议	Device CRD + Device Service	可替换 Device Service 实现
设备数据	私有消息总线	EdgeX Core Data + K8s Event	标准消息总线，可替换
批量操作	手工脚本	OTA Task CRD + Controller 自动编排	声明式任务，平台自动执行

未来迭代不受制于人

这种基于逻辑抽象的设计，其深层价值在于技术主权：

- 如果 EdgeX Foundry 的发展方向不符合需求，可以替换为其他兼容实现，应用层无感知
- 如果 KubeEdge 的社区活跃度下降，可以切换到 OpenYurt 或 SuperEdge，集群管理接口不变
- 如果某个 GPU 厂商的 Device Plugin 存在缺陷，可以自行开发替代版本，调度策略不受影响

企业的技术投资保护不在于选择“永远不变的技术”，而在于选择“变化时影响范围最小的架构”。基于 Kubernetes CRD 的模块化架构，把每一层的变化封装在接口边界之内，上层应用和下层硬件互不影响。

2.5 边缘计算的四代演进

回顾边缘计算的发展脉络，可以清晰地看到一条从"硬件驱动"到"应用驱动"的演进路径：

第一代：内容分发阶段（CDN 时代）

驱动力：网络带宽 代表技术：CDN 边缘节点 核心能力：静态内容缓存、就近访问加速 局限性：只能缓存内容，不能运行计算逻辑。"边缘"只是网络拓扑的概念，没有计算能力。

第二代：容器化边缘阶段

驱动力：运行时标准化 代表技术：K3s、KubeEdge、OpenYurt 核心能力：在边缘运行容器化工作负载、云边状态同步 局限性：解决了"边缘能跑容器"的问题，但缺少应用全生命周期管理。开发者能把容器部署到边缘，但管理 50 个站点上的 20 个应用版本仍然依赖手工操作。

第三代：AI 推理边缘阶段

驱动力：算力下沉 代表技术：GPU/NPU 边缘部署、模型推理框架 核心能力：边缘 AI 推理、异构算力管理 局限性：注意力集中在算力调度层面——"怎么让 GPU 利用率更高""怎么支持更多厂商的卡"。算力是手段，但不是目的。一张 GPU 利用率达到 92% 本身不产生业务价值，运行在上面的模型才产生价值。

第四代：应用驱动的全栈智能边缘（当前阶段）

驱动力：应用价值 核心理念：所有技术能力（硬件管理、算力调度、网络优化、安全隔离）都服务于一个目标——让应用在边缘高效地产生业务价值。

本平台所处的正是这一阶段。其标志性特征包括：

特征	具体表现
应用是一等公民	Application/ApplicationVersion/ApplicationDeployment 三层 CRD 体系，应用具有独立的生命周期管理
应用与硬件解耦	可插拔 Provisioner 模式，应用定义不包含任何硬件细节
应用可被共享	应用商店 + 审核流程，一个团队开发，全平台可用
应用跨边缘分发	Topology-Based 部署，同一应用声明式分发到任意数量的边缘站点
应用持续演进	版本管理 + 批量升级，模型迭代的频率不再受限于运维能力
应用运行有保障	边缘离线自治 + 自愈机制，断网不影响应用运行
应用健康可视	291+ 预定义监控指标，多租户隔离的可观测体系

四代演进的叙事逻辑

第一代（CDN）→ 边缘有"位置"，但没有"计算"

↓

第二代（容器化）→ 边缘有"计算"，但没有"管理"

↓

第三代（AI 推理）→ 边缘有"算力管理"，但目光停留在硬件层

↓

第四代（应用驱动）→ 技术回归本质：一切为了应用价值

每一代的进步都是真实的。CDN 解决了延迟问题，容器化解决了部署标准化问题，AI 推理解决了算力利用率问题。但只有当视角从硬件转向应用，边缘计算的投资回报才能被系统性地最大化。

2.6 本章小结：白皮书的阅读地图

本章确立了整篇白皮书的核心论点：价值重心从硬件前置投入转向应用持续释放。后续所有技术章节都将围绕这一论点展开：

应用旅程的阶段	对应章节	核心问题
在应用部署之前，硬件和集群怎么准备好？	第三章：基础设施就绪	OTA 接入 + 集群管理 + 镜像分发
应用需要 GPU 算力，但边缘有各种异构硬件	第四章：算力就绪	四层 CRD 抽象 + 双模虚拟化
多个团队共享平台，怎么安全隔离？	第五章：组织与安全	五层权限模型 + 多租户架构
应用怎么打包、审核、上架？	第六章：应用供应链	Provisioner 插件 + 应用商店
应用怎么部署到每一个边缘站点？	第七章：应用部署	Topology-Based 部署 + 批量版本切换
边缘断网了应用还能跑吗？需要操作设备怎么办？	第八章：应用运行	离线自治 + IoT CRD
怎么知道应用在每个站点是否健康？	第九章：应用可观测	291+ 指标 + 多租户隔离

读者可以根据自身角色选择阅读路径：

- CTO/CIO：第一章（摘要）→ 第二章（本章）→ 第十一章（竞品对比）→ 第十二章（行业场景）
 - 架构师：第二章 → 第三至九章（应用全生命周期）→ 第十章（架构设计）
 - 运维工程师：第三章（基础设施）→ 第七章（部署）→ 第九章（可观测）→ 第十三章（运维指南）
 - AI/算力工程师：第四章（算力就绪）→ 第六章（模型部署）→ 第九章（GPU 监控）
- 下一章：第三章 — 基础设施就绪，应用运行的基石

第三章：基础设施就绪 —— 应用运行的基石

应用面对的问题：在应用部署之前，边缘硬件怎么快速纳管？集群怎么建？镜像怎么到达边缘？

引言：应用的第一个前提是"地基"

第六章和第七章分别讲述了应用的打包上架和拓扑部署。但在应用开始部署之前，有一个更基础的问题需要回答：应用运行的基础设施从哪里来？

一个典型的边缘计算项目，在应用上线之前需要完成三项基础工作：

1. 硬件接入：数十甚至数千台边缘服务器如何快速上架、安装驱动、配置环境？如果每台服务器都需要运维人员到现场操作，那么光是硬件准备就要消耗数月时间。
2. 集群建立：边缘服务器接入后，如何将它们组织成可管理的 Kubernetes 集群？云端与边缘之间的网络连接如何解决？
3. 镜像到位：应用容器镜像从中心仓库到边缘节点，跨越数千公里的网络链路，如何保证高效可靠？

本章介绍平台在这三个维度上的技术方案：OTA 批量运维系统解决硬件接入问题，集群联邦管理解决集群创建和连接问题，分层镜像分发解决应用镜像的"最后一公里"问题。

3.1 硬件快速接入 — OTA 批量运维系统

问题：一千台边缘服务器如何上架

大规模边缘计算项目的第一个运维挑战发生在硬件上架阶段。以一家制造企业在全国 50 个工厂部署边缘服务器为例，每个工厂 10-20 台服务器，总计 500-1000 台。每台服务器上架后需要：

- 安装操作系统和基础软件（Docker、containerd）
- 安装 GPU/NPU 驱动（NVIDIA Driver、CUDA Toolkit 等）
- 配置网络和存储环境
- 注册到平台管理系统

如果采用传统的"运维人员到现场操作"方式，即使每台服务器只需要 2 小时，1000 台服务器也需要 2000 人时——这还不算差旅时间和多次返工。

解决方案：声明式任务驱动的批量运维

OTA（Over-The-Air）模块将硬件上架过程从"人工操作问题"转化为"声明式任务问题"。运维人员不需要登录每台服务器执行命令，而是创建一个任务声明"在这批服务器上执行这些操作"，平台自动分发到每台设备并行执行。

四种任务类型

OTA 支持四种原子任务类型，覆盖硬件接入的所有操作场景：

任务类型	用途	典型场景
exec （执行命令）	在多个节点上同时执行 Shell 命令	批量安装 Docker、NVIDIA 驱动、配置系统参数
read （读取文件）	从多个节点读取指定文件内容	批量检查配置文件、收集系统日志、审计环境一致性
write （写入文件）	向多个节点写入文件	批量分发配置文件、更新证书、统一网络配置
playbook （运行 Playbook）	在多个节点上运行自动化脚本	编排多步骤纳管流程、复杂环境初始化

四种任务类型可以组合使用。例如，一个完整的"边缘服务器纳管"流程可以编排为：

```
Step 1 (write) → 写入 apt/yum 镜像源配置
Step 2 (exec) → 安装 Docker 和 containerd
Step 3 (exec) → 安装 NVIDIA Driver 和 CUDA Toolkit
Step 4 (read) → 读取 nvidia-smi 输出，验证 GPU 驱动安装成功
Step 5 (exec) → 注册到 Kubernetes 集群
Step 6 (playbook) → 运行完整的健康检查 Playbook
```

分布式执行与结果聚合

OTA 的执行模型是分布式的：一个任务被创建后，Controller 将其分发到目标设备，每个设备的 OTA Agent 独立执行并上报结果。

```
Task (声明式任务定义)
├── Controller 分发
├── Device-1: OTA Agent 执行 → DeviceResult (成功)
├── Device-2: OTA Agent 执行 → DeviceResult (成功)
├── Device-3: OTA Agent 执行 → DeviceResult (失败：驱动版本不兼容)
├── Device-4: OTA Agent 执行 → DeviceResult (执行中)
├── ...
└── Task Status (自动聚合)
    ├── succeeded: 2
    ├── failed: 1
    ├── running: 1
    └── queued: N-4
```

关键设计：每个设备的执行结果是独立跟踪的。Device-3 失败不影响 Device-1 和 Device-2 的成功结果，也不阻塞 Device-4 的执行。Task 级别自动聚合 succeeded/failed/running/queued 计数，运维人员可以一眼看到全局进展。

万级节点支持

OTA 的设计目标是支持万级节点的并行执行：

- 队列化执行：不是同时向所有设备下发任务，而是通过队列控制并发度，避免 Controller 过载
- 独立结果追踪：每个设备的 DeviceResult 是独立的资源对象，不存在全局锁竞争
- 自动超时处理：设备离线或执行超时会被自动标记为失败，不阻塞整体任务

OTA 的定位澄清

需要特别说明的是：**OTA** 解决的是硬件接入问题，不是应用更新问题。应用的版本更新通过 `ApplicationDeployment` 的版本切换实现（第七章），不需要通过 OTA。

OTA 的核心价值在于：将"一千台边缘服务器上架"这个运维问题，从人工操作变成声明式任务。每个节点的成败互不影响，全局进展一目了然。

3.2 集群联邦管理

问题：边缘集群怎么建、怎么连

硬件接入后，下一步是将边缘服务器组织成可管理的 Kubernetes 集群，并建立与云端管控面的连接。边缘场景下的集群管理面临两个特有挑战：

1. 网络多样性：有的边缘站点与云端在同一机房（或通过 VPN 连接），可以直接访问；有的站点在公网另一侧，需要穿透 NAT。
2. 运行时多样性：不同场景对边缘运行时有不同需求——资源极度受限的嵌入式场景需要 KubeEdge，标准服务器级边缘需要 OpenYurt，独立运行的边缘站点需要轻量级集群。

双模式连接

平台支持两种集群连接模式，适配不同的网络环境：

源码实证：连接模式通过 `ConnectionType` 类型定义：

```
`go
const (
    ConnectionTypeDirect ConnectionType = "direct"
    ConnectionTypeProxy  ConnectionType = "proxy"
)
```

— [api/scope/v1alpha1/cluster_types.go:54-56](#)

连接模式	适用场景	要求	特点
Direct （直接连接）	同机房、VPN、专线	提供 KubeConfig 和 API Endpoint	低延迟，直接访问边缘 API Server
Proxy （反向隧道）	跨公网、NAT 穿透	边缘 Agent 主动连接云端	无需暴露边缘端口，自动建立隧道

Direct 模式下，云端直接使用 KubeConfig 访问边缘集群的 API Server：

```
type Connection struct {
    Type ConnectionType `json:"type,omitempty"`

    // Direct 模式需要提供 API Endpoint 和 KubeConfig
    KubernetesAPIEndpoint string `json:"kubernetesAPIEndpoint,omitempty"`
    KubeConfig []byte `json:"kubeconfig,omitempty"`

    // Proxy 模式由 Agent 反向连接，自动填充
    Token string `json:"token,omitempty"`
    KubernetesAPIServerPort uint16 `json:"kubernetesAPIServerPort,omitempty"`
    TheriseunionAPIServerPort uint16 `json:"theriseunionAPIServerPort,omitempty"`
}
```

— [api/scope/v1alpha1/cluster_types.go:58-111](#)

Proxy 模式下，边缘 Agent 使用 Token 主动连接云端的反向代理。云端为每个 Proxy 集群分配独立的端口（`KubernetesAPIServerPort` 和 `TheriseunionAPIServerPort`），所有请求通过隧道转发到边缘。

关键优势：Proxy 模式下，边缘站点无需暴露任何公网端口，只需出方向连接到云端即可。这解决了大多数边缘站点的网络安全限制问题。

VCluster 支持

对于需要完整 Kubernetes 控制面但不希望部署独立物理集群的场景，平台支持通过 VCluster（虚拟集群）快速创建轻量级边缘集群。

注意：VCluster 是在 Host K8s 内部创建的虚拟控制面，调度依赖宿主机节点，适用于多租户隔离或快速创建测试环境。它不适用于物理独立的边缘站点场景——独立边缘站点需要直接在边缘硬件上部署 K3s/K0s，再以 Member Cluster 形式接入平台。

VCluster 的创建完全由 Controller 自动化管理，通过 Helm Chart 安装，支持 K3s/K0s/K8s 三种发行版（Distro）。

源码实证：Cluster Controller 在 Reconcile 时检查是否启用了 VCluster：

```
`go
// 1. 检查是否需要安装 vCluster
if isVClusterEnabled(instance) && !isVClusterReady(instance) {
    err := r.ensureVCluster(ctx, instance)
    if err != nil {
```

```
return ctrl.Result{}, err
}

return updateCondition(instance, scopev1alpha1.ClusterVirtualClusterReady, ...)
}
、
```

— [internal/controller/cluster/cluster_controller.go:217-223](#)

VCluster 安装完成后，KubeConfig 自动同步到 Cluster CRD 的 Connection 字段，后续所有请求通过标准 Kubernetes API 路由到虚拟集群。

Condition 状态链

每个 Cluster 的生命周期通过 Condition 状态链精确追踪。每个 Condition 独立跟踪，外部系统可以精确读取集群所处的阶段：

```
Initialized → 集群初始化完成，等待 Agent 连接
↓
AgentAvailable → Agent 已连接，通信正常
↓
CoreReady → 平台核心组件（APIServer/Controller）已就绪
↓
Ready → 集群整体就绪，可以接受应用部署
↓
Schedulable → 集群可调度，节点资源充足
```

以及可选的运行时 Condition：

Condition	含义
VirtualClusterReady	VCluster 安装完成并就绪
OpenyurtReady	OpenYurt 运行时就绪
KubeedgeReady	KubeEdge 运行时就绪

源码实证：完整的 Condition 类型定义：

```
`go
const (
    ClusterInitialized ClusterConditionType = "Initialized"
    ClusterAgentAvailable ClusterConditionType = "AgentAvailable"
    ClusterReady ClusterConditionType = "Ready"
    ClusterSchedulable ClusterConditionType = "Schedulable"
    ClusterCoreReady ClusterConditionType = "CoreReady"
    ClusterVirtualClusterReady ClusterConditionType = "VirtualClusterReady"
    ClusterOpenyurtReady ClusterConditionType = "OpenyurtReady"
```

```
ClusterKubeedgeReady ClusterConditionType = "KubeedgeReady"
```

```
)
```

```
、
```

— [api/scope/v1alpha1/cluster_types.go:115-149](#)

并发控制

Cluster Controller 支持多集群并发 Reconcile，同时通过 InstanceLock 确保同一集群不会被并发处理：

```
// MaxConcurrentReconciles: 3, 允许同时处理 3 个不同的集群
// 同一集群的 Reconcile 通过 InstanceLock 串行化

getLock := r.TryLock(instance.Name)
if !getLock {
    // 当前集群正在被处理, 5 秒后重试
    return ctrl.Result{RequeueAfter: 5 * time.Second}, nil
}
defer r.Unlock(instance.Name)
```

— [internal/controller/cluster/cluster_controller.go:87-92](#)

删除流程

集群删除通过 Finalizer 保护，确保资源的正确清理：

```
DeletionTimestamp 被设置
├── undoReconcile 执行清理逻辑
│   ├── 如果启用了 VCluster → cleanupVCluster (卸载 Helm Release)
│   └── 清理关联组件
└── 移除 Finalizer → K8s 完成删除
```

源码实证：

```
`go
```

```
const ClusterFinalizer = "finalizer.cluster.theriseunion.io"
```

```
、
```

— [api/scope/v1alpha1/cluster_types.go:20](#)

3.3 可插拔边缘运行时

问题：不同场景需要不同的运行时

边缘计算没有"一种运行时解决所有问题"的银弹。不同的场景对运行时有截然不同的需求：

场景	特征	适合的运行时
嵌入式边缘（ARM 网关、树莓派）	资源极度受限（<1GB 内存）	KubeEdge
标准服务器级边缘（工厂、门店）	资源充足，需要与云端无缝协作	OpenYurt
独立运行的边缘站点	需要完整 K8s 控制面，独立自主	K3s/K0s（独立部署，作为 Member Cluster 接入平台）

平台通过可插拔的 ComponentInstaller 接口支持多种边缘运行时，新增运行时只需要实现这个接口并通过 `init()` 注册。

ComponentInstaller 接口

```
// ComponentInstaller 组件管理器接口，定义组件安装的完整流程
type ComponentInstaller interface {
    // PreInstall 安装前的准备工作，例如检查前置条件、创建命名空间等
    PreInstall(ctx context.Context, cluster *scopev1alpha1.Cluster,
        config *ComponentConfig) (*ComponentCondition, error)

    // Install 执行实际的安装逻辑，返回组件配置
    Install(ctx context.Context, cluster *scopev1alpha1.Cluster,
        config *ComponentConfig) (*ComponentCondition, error)

    // PostInstall 安装后的处理，例如验证安装结果、设置状态等
    PostInstall(ctx context.Context, cluster *scopev1alpha1.Cluster,
        config *ComponentConfig) (*ComponentCondition, error)
}
```


— [internal/component/types.go:22-40](#)

每个安装步骤返回一个 `ComponentCondition`，Controller 据此更新 Cluster 的 Condition 状态。这意味着外部系统可以精确观测到"OpenYurt 正在安装""KubeEdge 安装失败"等细粒度状态。

插件注册机制

新的运行时通过 `init()` 函数注册到全局注册表，无需修改 Controller 核心逻辑：

```
// 注册组件安装器实现
func RegisterComponentInstallerFunc(name string, fn ComponentInstallerFunc) {
    if _, ok := componentInstallerFuncs[name]; ok {
        klog.Errorf("component installer func name %s already exists", name)
        return
    }
    componentInstallerFuncs[name] = fn
}
```

— [internal/component/types.go:64-70](#)

已注册的组件在 Local Cluster Reconcile 时自动遍历安装：

```
var components = []string{
    "apiserver",
    "controller",
    openyurt.COMPONENT_NAME,
    kubeedge.COMPONENT_NAME,
    "etcd",
}

// 遍历所有需要安装的组件
for _, componentName := range components {
    if err := installComponent(ctx, r, logger, clusterCopy, componentName); err != nil {
        logger.Error(err, "Failed to install component", "component", componentName)
    }
}
```

— [internal/controller/cluster/cluster_controller_local_cluster.go:16-41](#)

每个组件的安装过程是完整的三阶段流程（PreInstall → Install → PostInstall），每个阶段独立反馈状态：

```
componentCondition, err := componentInstaller.PreInstall(ctx, instance, componentConfig)
if componentCondition != nil {
    r.setClusterCondition(instance, componentCondition.ConditionType, ...)
}
// ... Install ... PostInstall 同理
```

— [internal/controller/cluster/cluster_controller_local_cluster.go:75-101](#)

已注册组件

组件	命名空间	对应 Condition	作用
APIServer	edge-system	CoreReady	平台 API 服务
Controller	edge-system	CoreReady	平台控制器
OpenYurt	kube-system	OpenyurtReady	边缘自治运行时

组件	命名空间	对应 Condition	作用
KubeEdge	ke-system	KubeedgeReady	轻量级边缘运行时
VCluster	动态创建	VirtualClusterReady	虚拟集群运行时

开放架构的关键体现

ComponentInstaller 接口是平台开放架构的关键体现。未来如果需要支持新的边缘运行时（例如 SuperEdge 或自研方案），只需要：

1. 创建一个新文件，实现 `ComponentInstaller` 接口的三个方法
2. 在 `init()` 函数中注册到全局注册表
3. 将组件名称添加到 `components` 列表

不需要修改 `Controller` 核心逻辑，不需要修改已有组件的代码，不需要改动 `CRD Schema`。

3.4 分层镜像分发

问题：应用镜像如何到达边缘

应用容器镜像是应用运行的"原材料"。在边缘场景下，镜像分发面临独特的挑战：

- 带宽有限：边缘站点到中心仓库的网络带宽远低于数据中心内部
- 延迟较高：跨地域的网络延迟导致镜像拉取时间显著增加
- 可靠性要求：镜像拉取失败意味着应用无法启动，业务中断

四级分发架构

平台采用分层缓存的镜像分发架构，将镜像的传输路径缩短到最近的可用节点：



四种镜像资源 CRD

平台通过四种 CRD 管理镜像分发链路的每一层：

CRD	作用域	职责
Registry	Namespace 级	命名空间内的镜像仓库定义，包含认证信息
ClusterRegistry	Cluster 级	集群级别的镜像仓库定义，跨命名空间可见
Repository	Namespace 级	具体的镜像存储库（含拉取统计）
ClusterRepository	Cluster 级	集群级别的镜像存储库

源码实证：Registry 和 ClusterRegistry 共享相同的 `RegistrySpec` 结构（认证信息、域名、Provider），区别在于作用域——Registry 限制在特定 Namespace，ClusterRegistry 全集群可见。

— `api/registry/v1alpha1/registry_types.go` 和 `api/registry/v1alpha1/clusterregistry_types.go`

Registry 通过 Finalizer 保护，确保删除时不会遗留关联的 Repository 资源：

```
const RegistryFinalizer = "finalizer.registry.theriseunion.io"
const ClusterRegistryFinalizer = "finalizer.clusterregistry.theriseunion.io"
```

— `api/registry/v1alpha1/registry_types.go:26-29`

Repository 的运行时统计

每个 Repository 跟踪镜像的运行时统计信息：

```
type RepositoryStatus struct {
    Status string `json:"status,omitempty"`
    ArtifactCount int64 `json:"artifactCount,omitempty"` // 镜像制品数量
    PullCount int64 `json:"pullCount,omitempty"` // 拉取次数
}
```

— `api/registry/v1alpha1/repository_types.go:37-43`

这些统计数据帮助运维人员了解每个边缘站点的镜像使用情况——哪些镜像拉取频率最高、哪些站点的镜像缓存需要扩容。

多架构支持

边缘设备的 CPU 架构多样（x86_64、ARM64、RISC-V），同一个应用可能需要支持多种架构的镜像。平台的镜像管理原生支持 multi-arch manifest，确保：

- 同一个 Repository 可以存储 x86 和 ARM 版本的镜像
- 边缘节点拉取时自动匹配本地架构
- 推送时只同步变化的镜像层（增量同步），节省带宽

分层缓存的价值

分层缓存架构的核心价值在于：即使云端到边缘的网络不稳定，只要边缘站点本地有缓存，应用就能快速启动。

这与第八章将介绍的边缘离线自治能力形成互补——离线自治保证应用在断网时持续运行，分层镜像缓存保证新应用在弱网环境下也能快速部署。

3.5 本章小结

基础设施就绪回答了"应用部署之前需要准备什么"的问题：

环节	机制	效果
硬件接入	OTA 声明式任务（ exec/read/write/playbook ）	千台服务器的纳管从人工操作变成声明式任务
集群连接	双模式连接（ Direct/Proxy ）	同机房直连 + 跨公网隧道，覆盖所有网络场景
虚拟集群	VCluster 自动安装（ K3s/K0s/K8s ）	无需独立物理集群即可获得完整 K8s 控制面
运行时多样性	ComponentInstaller 插件接口	新增运行时只需一个文件
Condition 追踪	每个组件独立 Condition	精确观测集群的每个就绪阶段
镜像分发	四级缓存（ Registry ClusterRegistry Repository Node ）	弱网环境下镜像仍可快速拉取
并发安全	InstanceLock + MaxConcurrentReconciles: 3	多集群并发处理，单集群串行化

这些基础设施能力本身不直接产生业务价值，但它们是第二章提出的"应用价值持续释放"的前提条件。应用无法在不存在的集群上运行，无法使用不存在的镜像启动，无法部署到没有纳管的服务器上。基础设施就绪，是应用旅程的第零步。

下一章：第四章 — 算力就绪，让应用无需感知硬件差异

第四章：算力就绪 — 让应用无需感知硬件差异

应用面对的问题：我的 AI 模型需要 GPU，但边缘有 NVIDIA、昇腾、海光各种卡，怎么办？一张卡太浪费，怎么切分共享？

引言：算力是手段，应用是目的

上一章解决了基础设施的就绪问题——硬件接入、集群建立、镜像到位。但对于 AI 推理类应用，还有一个关键问题：应用需要 GPU 算力，而边缘的 GPU 硬件五花八门。

这不是一个理论问题。在实际的边缘部署中，一家企业可能同时使用：

- **NVIDIA A100**：早期采购的高端推理卡
- **昇腾 310P**：国产化替代的首选
- **海光 DCU**：特定合规场景的要求
- **寒武纪 MLU270**：与特定框架深度集成

如果应用开发者需要为每种 GPU 编写不同的部署配置——不同的 Device Plugin、不同的资源请求格式、不同的调度标签——那么“应用与硬件解耦”就是一句空话。

本章介绍平台的异构算力管理体系。其核心设计理念是：应用只需声明“我需要多少算力”，平台负责将这个需求映射到具体的 GPU 硬件上。这通过四层 CRD 抽象、双模虚拟化切分和全局调度策略三个机制实现。

4.1 九大 GPU 厂商统一建模

覆盖范围

平台原生支持 9 大 GPU/NPU/DCU 厂商，覆盖国内外主流异构计算平台：

厂商	英文标识	中文名称	代表产品
NVIDIA	NVIDIA	英伟达	A100、H100、T4
Hygon	Hygon	海光	DCU Z100
Ascend	Ascend	昇腾	310P、910B
Metax	Metax	沐曦	C200、N100
Kunlunxin	Kunlunxin	昆仑芯	R200、R480
Alibaba	Alibaba	平头哥	含光 800
Iluvatar	Iluvatar	天数智芯	BI150
Enflame	Enflame	燧原	S60、T20
Cambricon	Cambricon	寒武纪	MLU270、MLU370

源码实证：所有厂商定义在统一的常量文件中：

```
`go
type Manufacturer string

>

const (
    ManufacturerNVIDIA = "NVIDIA"
    ManufacturerHygon = "Hygon"
```

```
ManufacturerAscend = "Ascend"

ManufacturerMetax = "Metax"

ManufacturerKunlunxin = "Kunlunxin"

ManufacturerAlibaba = "Alibaba"

ManufacturerIluvatar = "Iluvatar"

ManufacturerEnflame = "Enflame"

ManufacturerCambricon = "Cambricon"

)
、
```

— pkg/constants/device.go:28-39

每个厂商都有对应的 HAMi Device Plugin 适配：

```
const (
    DevicePluginNVIDIA = "nvidia"
    DevicePluginNVIDIAHAMI = "nvidia-hami"
    DevicePluginHygonHAMI = "hygon-hami"
    DevicePluginKunlunHAMI = "kunlunxin-hami"
    DevicePluginIluvatarHAMI = "iluvatar-hami"
    DevicePluginCambriconHAMI = "cambricon-hami"
    DevicePluginMetaxGPUHAMI = "metax-gpu-hami"
    DevicePluginAscendVNPUGHAMI = "ascend-vnpu-hami"
)
```

— pkg/constants/device.go:139-148

统一标签体系

无论什么厂商的 GPU，都通过统一的 Label 体系标识：

Label Key	用途	示例值
theriseunion.io/manufacturer	GPU 厂商	NVIDIA
theriseunion.io/model	GPU 型号	nvidia-a100-pcie-40gb
theriseunion.io/virtual-model	是否为虚拟设备	true
theriseunion.io/config-type	配置类型（内置/自定义）	fixed / custom
theriseunion.io/resource-pool	所属资源池	default

— pkg/constants/device.go:80-93

4.2 四层 CRD 抽象架构

设计哲学：从物理硬件到应用可用算力

平台通过四层 CRD 将物理 GPU 硬件逐步抽象为应用可用的算力资源。每一层解决一个特定的问题：

```
第 1 层 DeviceModel "这张卡是什么?" → 硬件规格定义
      ↓
第 2 层 ComputeTemplate "怎么切分和超卖?" → 算力配比模板
      ↓
第 3 层 ResourcePool "分给谁用?" → 资源池与调度策略
      ↓
第 4 层 NodeConfig "这个节点怎么配?" → 节点级配置
```

第 1 层：DeviceModel — 硬件规格定义

DeviceModel 定义了 GPU 硬件的物理规格，是整个算力管理体系的基础数据。

```
type DeviceModelSpec struct {
    // 设备类型：物理卡或虚拟卡
    DeviceType DeviceType `json:"deviceType"`

    // GPU 厂商
    Manufacturer constants.Manufacturer `json:"manufacturer,omitempty"`

    // GPU 型号
    Model string `json:"model,omitempty"`

    // 显存大小 (MB)
    MemorySize int `json:"memorySize,omitempty"`

    // 算力百分比 (0-100)
    // 物理卡：必须为 100 (全部算力)
    // 虚拟卡：必须 < 100 (部分算力)
    Core int `json:"core,omitempty"}`
```

```
// 多精度算力定义
ComputeCapabilities []ComputeCapability `json:"computeCapabilities,omitempty"`

// 是否支持虚拟化切分
SupportVirtualDevice bool `json:"supportVirtualDevice,omitempty"`

// 支持的切分模式（硬切分/软切分）
SupportComputeSplitMode []ComputeSplitMode `json:"supportComputeSplitMode,omitempty"`

// MIG 切分模板（如果支持硬切分）
// ... (truncated)
```

— [api/device/v1alpha1/devicemodel_types.go:117-182](#)

关键设计：物理卡与虚拟卡的统一建模

DeviceModel 通过 `DeviceType` 字段区分物理设备和虚拟设备：

DeviceType	Core 约束	含义
physical	必须为 100	代表一张完整的物理 GPU 卡
virtual	必须 < 100	代表物理卡的一个虚拟切分

源码实证：验证逻辑确保 Core 值与 DeviceType 一致：

```
`go
func (s *DeviceModelSpec) Validate() error {
    isPhysical := s.IsPhysical()
    if isPhysical && s.Core != constants.PhysicalDeviceCore {
        return fmt.Errorf("physical device must have core=%d, got %d",
            constants.PhysicalDeviceCore, s.Core)
    }
    if !isPhysical && s.Core >= constants.PhysicalDeviceCore {
        return fmt.Errorf("virtual device must have core<=%d, got %d",
            constants.PhysicalDeviceCore, s.Core)
    }
    return nil
}
```

— [api/device/v1alpha1/devicemodel_types.go:252-261](#)

多精度算力定义

不同的 AI 应用对浮点精度的需求不同——训练通常使用 FP32，推理可以使用 FP16 或 INT8。

DeviceModel 通过 `ComputeCapability` 结构支持多精度的算力定义：

```
type ComputeCapability struct {
    // 精度类型：FP32、FP16、BF16、FP8
    Precision Precision `json:"precision"`

    // 算力值（字符串，避免浮点精度问题）
    Value string `json:"value"`

    // 单位：TFLOPS、GFLOPS、MFLOPS
    Unit ComputeUnit `json:"unit,omitempty"`
}
```

— [api/device/v1alpha1/devicemodel_types.go:64-76](#)

以 NVIDIA A100 为例，同一张卡在不同精度下的算力差异巨大：

精度	算力	适用场景
FP32	19.5 TFLOPS	科学计算、高精度训练
FP16	312 TFLOPS	AI 推理
BF16	312 TFLOPS	大模型训练
FP8	624 TFLOPS	量化推理

设计细节：为什么算力值使用字符串？

Value 字段使用 `string` 而非 `float64`：

```
// Value represents the compute capability value as a string (e.g., "19.5", "312").
// Using string type to avoid precision issues across different languages in CRD.
Value string `json:"value"``
```

这是对 CRD 跨语言互操作性的务实考量——浮点数在 JSON 序列化/反序列化时可能出现精度丢失（如 $0.1 + 0.2 \neq 0.3$ ），而算力规格是精确的硬件参数，不允许精度偏差。

MIG 切分模板

对于支持 NVIDIA MIG（Multi-Instance GPU）的硬件，DeviceModel 通过二维数组定义多种切分方案：

```
type MigTemplate struct {
    Name string `json:"name"` // 切分名称，如 "1g.10gb"
    Memory int32 `json:"memory"` // 显存（MB）
    Count int32 `json:"count"` // 实例数量
}

// 二维数组：外层是不同的切分方案，内层是方案内的实例组合
MigTemplates [][]MigTemplate `json:"migTemplates,omitempty"``
```

— [api/device/v1alpha1/devicemodel_types.go:107-114](#)

例如 A100 80GB 可以有以下切分方案：

方案	切分配置	总实例数
方案 A	7 x 1g.10gb	7 个 10GB 实例
方案 B	3 x 2g.20gb + 1 x 1g.20gb	4 个实例
方案 C	2 x 3g.40gb	2 个 40GB 实例

第 2 层 : ComputeTemplate — 算力配比模板

ComputeTemplate 定义了如何对物理设备进行算力切分和超卖配比。

```
type ComputeTemplateSpec struct {
    // GPU 厂商
    Manufacturer constants.Manufacturer `json:"manufacturer"`

    // GPU 型号 (或 "all" 表示适用于该厂商所有型号)
    Model string `json:"model"`

    // 显存超分比例 (字符串, 如 "1.5" 表示 1.5 倍超卖)
    DeviceMemoryScaling string `json:"deviceMemoryScaling,omitempty"`

    // 算力超分比例 (字符串, 如 "2.0" 表示 2 倍超卖)
    DeviceCoreScaling string `json:"deviceCoreScaling,omitempty"`

    // 切分数量 (1-20)
    DeviceSplitCount int `json:"deviceSplitCount,omitempty"`

    // 切分模式: 硬切分 (MIG) 或软切分 (HAMi-Core)
    ComputeSplitMode ComputeSplitMode `json:"computeSplitMode,omitempty"`
}
```

— [api/device/v1alpha1/computetemplate_types.go:8-51](#)

超分比例 (Scaling Factor)

超分是边缘场景的常见需求。以推理场景为例，GPU 的显存利用率通常只有 40-60%，合理的超卖可以提高资源利用率：

参数	默认值	含义
deviceMemoryScaling	"1.0"	显存超分比例。"1.5" 表示 40GB 显存可分配 60GB
deviceCoreScaling	"1.0"	算力超分比例。"2.0" 表示 100% 算力可分配 200%
deviceSplitCount	10	切分数量。10 表示一张卡最多切分为 10 个虚拟 GPU

缩放因子同样使用字符串存储，与 ComputeCapability 的 Value 字段设计一致，避免浮点精度问题。

切分模式联动

ComputeTemplate 的 ComputeSplitMode 与 DeviceSplitCount 联动：

切分模式	ComputeSplitMode	切分特点
硬切分 (MIG)	"hard"	硬件级 GPU 分区，需要特定硬件支持 (A100/H100)，切分方案受 MigTemplates 约束
软切分 (HAMi-Core)	"soft"	软件级虚拟化，所有 GPU 厂商通用，支持 1%-99% 任意比例切分

源码实证：

```
`go
const (
    // 硬切分 (英伟达 MIG 模式)
```

```

    ComputeSplitModeHard = "hard"

    // 软切分 (HAMi-Core 模式)
    ComputeSplitModeSoft = "soft"
)
、

— api/device/v1alpha1/devicemodel_types.go:80-85

```

第 3 层 : ResourcePool — 资源池与调度策略

ResourcePool 将设备组织成逻辑资源池，并关联调度器：

```

type ResourcePoolSpec struct {
    // 调度器名称: hami、volcano、kubernetes
    Scheduler string `json:"scheduler,omitempty"`
}

type ResourcePoolStatus struct {
    Items []ItemDetail `json:"items,omitempty"`
    Allocatable *v1alpha1.DeviceResource `json:"allocatable,omitempty"` // 可分配资源
    Request *v1alpha1.DeviceResource `json:"request,omitempty"` // 已请求资源
    Limit *v1alpha1.DeviceResource `json:"limit,omitempty"` // 资源上限
}

— api/device/v1alpha1/resourcepool_types.go:29-41

```

三种调度器

调度器	标识	适用场景	中文说明
HAMi	hami	AI 推理、开发调试	支持虚拟 GPU 切分，按算力/显存细粒度分配
Volcano	volcano	AI 训练、微调	支持 Gang Scheduling，确保训练任务的 GPU 同时分配
Kubernetes	kubernetes	标准工作负载	使用 K8s 原生调度器，适用于不需要 GPU 虚拟化的场景

源码实证：

```

`go
const (
    SchedulerHAMi = "hami"
    SchedulerVolcano = "volcano"
    SchedulerKubernetes = "kubernetes"
)
、

— pkg/constants/device.go:134-136

```

三种资源池策略

ResourcePool 通过 Label 标记其使用策略：

策略	含义	适用场景
shared	共享资源池	多个团队/应用共享 GPU 资源
dedicated	独占资源池	特定应用独占 GPU，保证性能隔离
reserved	预留资源池	为未来扩展预留，当前不参与调度

源码实证：

```
`go
type ResourcePoolStrategy string

const (
    ResourcePoolStrategyShared ResourcePoolStrategy = "shared"
    ResourcePoolStrategyDedicated ResourcePoolStrategy = "dedicated"
    ResourcePoolStrategyReserved ResourcePoolStrategy = "reserved"
)
`
— pkg/constants/device.go:119-125
```

资源池的三级作用域

ResourcePool 的成员可以按三种作用域定义：

```
const (
    ResourcePoolScopeGlobal ResourcePoolScope = "global" // 全局：所有 GPU 设备
    ResourcePoolScopeNode ResourcePoolScope = "node" // 节点级：指定节点上的 GPU
    ResourcePoolScopeDevice ResourcePoolScope = "device" // 设备级：指定的 GPU 设备
)
— pkg/constants/device.go:128-131
```

通过 ResourcePoolItem CRD 将具体的节点或设备关联到资源池：

```
type ResourcePoolItemSpec struct {
    Scope constants.ResourcePoolScope `json:"scope,omitempty"`
    ResourcePoolName string `json:"resourcePoolName,omitempty"`
}
— api/device/v1alpha1/resourcepoolitem_types.go:28-36
```

第 4 层：NodeConfig — 节点级配置

NodeConfig 是每个计算节点的算力配置聚合体。它将 DeviceModel、ComputeTemplate 和 Device Plugin 在节点级别关联起来：

```
type NodeConfigSpec struct {
    // 每个厂商的独立配置
    ManufacturerSpecs []ManufacturerSpec `json:"manufacturerSpec,omitempty"`
}
```



```

type ManufacturerSpec struct {
    // Device Plugin 名称
    DevicePlugin string `json:"devicePlugin,omitempty"`

    // 默认 ComputeTemplate
    ComputeTemplate string `json:"computeTemplate,omitempty"`

    // 每个设备可以使用不同的 ComputeTemplate
    DeviceComputes []DeviceComputeTemplate `json:"deviceComputeTemplate,omitempty"`

    // 禁用的设备列表（用于故障隔离）
    DisabledDevices []string `json:"disabledDevices,omitempty"`
}

```

— [api/device/v1alpha1/nodeconfig_types.go:27-36](#)

NodeConfig 的设计体现了“每个厂商独立配置”的原则——同一个节点可能插着 NVIDIA 和昇腾两种卡，每种卡有不同的 Device Plugin 和 ComputeTemplate。

NodeConfig 的 Status 包含运行时信息：

```

type ManufactureStatus struct {
    DevicePgingMeta v1alpha1.DevicePluginMeta `json:"devicePluginMeta,omitempty"`
    DevicePgingRuntime DevicePgingRuntime `json:"devicePgingRuntime,omitempty"`
}

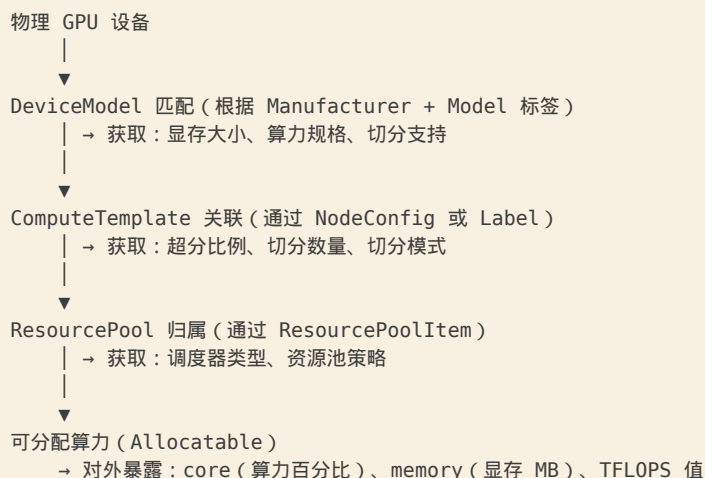
type DevicePgingRuntime struct {
    DevicePluginStatus string `json:"devicePluginStatus,omitempty"` // init/running/error
    DriverVersion string `json:"driverVersion,omitempty"`
    ToolkitVersion string `json:"toolkitVersion,omitempty"`
    ComputeTemplate ComputeTemplateSpec `json:"computeTemplate,omitempty"`
}

```

— [api/device/v1alpha1/nodeconfig_types.go:43-55](#)

四层抽象的协作关系

四层 CRD 的数据流形成了一条从物理硬件到可分配算力的丰富化管线：



4.3 GPU 虚拟化 — 双模切分

为什么需要 GPU 虚拟化

一张物理 GPU 的算力远超单个推理任务的需求。以 NVIDIA A100 为例，其 FP16 算力达到 312 TFLOPS，而一个典型的质检推理模型只需要 10-20 TFLOPS。如果每个推理任务独占一张卡，GPU 利用率不到 10%。

GPU 虚拟化的核心价值是：将一张物理卡切分为多个虚拟 GPU，让多个推理任务共享同一张卡，显著提高利用率。

硬切分（MIG 模式）

适用硬件：NVIDIA A100、H100 等支持 MIG（Multi-Instance GPU）的型号。

MIG 是 NVIDIA 提供的硬件级 GPU 分区技术。它将一张 GPU 在硬件层面切分为多个独立的实例，每个实例拥有独立的显存、缓存和计算引擎，实现硬件级隔离。

MIG 的切分方案是预定义的，通过 DeviceModel 的 `MigTemplates` 字段配置。切分后的每个实例作为一个独立的虚拟 DeviceModel 注册到系统中。

优势：硬件级隔离，一个实例的故障不影响其他实例。 局限：只有特定的 NVIDIA 高端卡支持，切分粒度受硬件限制。

软切分（HAMI-Core 模式）

适用硬件：所有 9 大 GPU 厂商的硬件，无硬件限制。

HAMi-Core 是软件层面的 GPU 虚拟化方案。它通过拦截 CUDA/ROCm 等 GPU 运行时的 API 调用，实现算力和显存的动态分配。

```
ComputeTemplate 配置：
DeviceSplitCount: 10 → 一张卡最多切分为 10 个虚拟 GPU
DeviceMemoryScaling: "1.5" → 显存超卖 1.5 倍
DeviceCoreScaling: "1.0" → 算力不超卖
ComputeSplitMode: "soft" → 使用 HAMi-Core 软切分
```

优势：所有厂商通用，支持 1%-99% 任意比例切分，灵活性极高。局限：软件隔离，极端负载下可能存在资源争抢。

效果对比

指标	切分前	切分后
GPU 数量	4 张 A100	2 张 A100（每张切分为 5 个 vGPU）
并行推理任务	4 个	10 个
平均 GPU 利用率	< 50%	~92%
GPU 采购成本	基准	节省 50%

这是第二章“同样的硬件，价值倍增”的直接体现——通过 GPU 虚拟化，同样数量的 GPU 卡可以服务 2.5 倍的推理任务。

4.4 全局调度策略

GlobalConfig — 集群级调度配置

GlobalConfig CRD 定义了集群维度的全局调度策略：

```
type GlobalConfigSpec struct {  
    // 资源感知模式：按分配率 / 按使用率  
    ResourceAwareness ResourceAwarenessMode `json:"resourceAwareness,omitempty"`  
  
    // 节点调度策略：紧凑 / 均衡  
    NodeScheduling SchedulingStrategy `json:"nodeScheduling,omitempty"`  
  
    // GPU 调度策略：紧凑 / 均衡  
    GPUScheduling SchedulingStrategy `json:"gpuScheduling,omitempty"`  
  
    // 是否启用资源池  
    ResourcePoolEnabled bool `json:"resourcePoolEnabled"`  
  
    // 故障码告警级别：critical/high/middle/low  
    AlertLevel []v1alpha1.Severity `json:"alertLevel,omitempty"`  
}
```

— [api/device/v1alpha1/globalconfig_types.go:47-78](#)

资源感知模式

模式	含义	适用场景
<code>allocation_rate</code> （默认）	按已分配资源的比例进行调度决策	稳定负载，关注资源分配的均衡性
<code>usage_rate</code>	按实际使用率进行调度决策	动态负载，关注实际利用率

源码实证：

```
`go
```

```
const (
    AllocationRate ResourceAwarenessMode = "allocation_rate"
    UsageRate ResourceAwarenessMode = "usage_rate"
)
```

— [api/device/v1alpha1/globalconfig_types.go:28-33](#)

调度策略组合

调度策略分为两个维度——节点级和 GPU 级：

维度	Binpack (紧凑)	Spread (均衡)
节点调度	尽量把 Pod 集中到少数节点	尽量把 Pod 分散到不同节点
GPU 调度	尽量把任务集中到少数 GPU	尽量把任务分散到不同 GPU

默认组合：节点级 Binpack + GPU 级 Spread。

```
// NodeScheduling 默认紧凑
// +kubebuilder:default:=binpack
NodeScheduling SchedulingStrategy `json:"nodeScheduling,omitempty"`

// GPUScheduling 默认均衡
// +kubebuilder:default:=spread
GPUScheduling SchedulingStrategy `json:"gpuScheduling,omitempty"`
```

— [api/device/v1alpha1/globalconfig_types.go:54-64](#)

为什么这个默认组合是边缘场景的最佳选择？

- **节点级 Binpack**：边缘场景下节点数量有限且分散。紧凑调度意味着尽量利用已有节点的资源，减少需要开机的边缘节点数量——在能耗敏感的边缘站点（如偏远基站），这直接降低了电力成本。
- **GPU 级 Spread**：单个 GPU 故障是边缘场景的常见问题（高温、灰尘、震动）。分散调度意味着即使一张 GPU 故障，只有部分推理任务受影响——而不是所有任务都集中在同一张卡上全军覆没。

这个默认组合体现了对边缘场景的深度理解：节点资源紧凑使用（减少边缘开机成本），GPU 内分散调度（提高单节点容错性）。

4.5 设备状态与健康管理的

设备状态

每个 GPU 设备有三种状态：

状态	含义
Available（启用）	设备正常，参与调度
Unschedulable（禁用）	设备被人工禁用，不参与调度
Unknown（未知）	设备状态不可知

健康检查

设备健康状态通过 Device Plugin 上报：

健康状态	含义
Healthy（已就绪）	设备正常运行
UnHealthy（异常）	设备存在故障（温度过高、驱动异常等）

源码实证：

```
`go
const (
    DeviceHealthy = "Healthy"
    DeviceUnHealthy = "UnHealthy"
)
```

— `pkg/constants/device.go:168-170`

结合 GlobalConfig 的告警级别配置（`AlertLevel`），设备异常时可以触发 critical/high/middle/low 不同级别的告警，对接第九章的可观测体系。

设备禁用机制

NodeConfig 的 `DisabledDevices` 字段支持在不物理移除硬件的情况下禁用特定 GPU：

```
// 禁用的设备列表（用于故障隔离）
DisabledDevices []string `json:"disabledDevices,omitempty"`
```

这在边缘场景下特别有用——当某张 GPU 出现间歇性故障时，运维人员可以远程禁用该设备（通过修改 NodeConfig），等待现场维修，而不影响同一节点上其他 GPU 的正常调度。

4.6 国产化适配策略

为什么国产 GPU 适配是刚需

随着国产化替代政策的推进，越来越多的边缘项目要求使用国产 GPU/NPU。但国产 GPU 的生态成熟度远不如 NVIDIA，具体表现为：

- Device Plugin 实现各不相同，资源请求格式不统一
- 部分厂商缺少虚拟化支持
- 调度器集成程度参差不齐

平台的适配策略

平台通过四层 CRD 架构实现了对国产 GPU 的透明适配：

1. **DeviceModel** 层：每种国产 GPU 型号注册为独立的 DeviceModel，定义其算力规格和切分能力
2. **ComputeTemplate** 层：通过 `theriseunion.io/model-scope: "all"` 标签，可以为一个厂商的所有型号设置统一的超卖模板
3. **NodeConfig** 层：每个节点按厂商独立配置 Device Plugin，互不影响
4. **ResourcePool** 层：国产 GPU 和 NVIDIA GPU 可以在同一个资源池中统一调度

对应用开发者而言，底层是 NVIDIA 还是昇腾是透明的。应用只需声明算力需求（如 `core: 50, memory: 20480`），调度器自动选择满足需求的 GPU，无论其厂商和型号。

4.7 本章小结

算力就绪回答了"应用的 GPU 需求如何被透明满足"的问题：

能力	实现方式	效果
9 大厂商统一	Manufacturer 常量 + 统一 Label 体系	应用无需感知底层是哪家的 GPU
硬件规格抽象	DeviceModel CRD (物理/虚拟统一建模)	多精度算力、MIG 模板、显存步长全面描述
算力切分配比	ComputeTemplate CRD (超分 + 切分)	一张卡切分为 1-20 个虚拟 GPU
资源池管理	ResourcePool + ResourcePoolItem	shared/dedicated/reserved 三种策略
节点级配置	NodeConfig CRD (按厂商独立配置)	同一节点支持多厂商 GPU 共存
双模虚拟化	MIG 硬切分 + HAMI-Core 软切分	GPU 利用率从 <50% 提升到 ~92%
全局调度	GlobalConfig (节点 Binpack + GPU Spread)	边缘场景的最佳成本/容错平衡
国产化支持	四层 CRD 透明适配	应用定义不包含任何厂商细节

这套算力管理体系的核心价值在于：将"应用需要 GPU"这个需求，从硬件选型问题转化为声明式配置问题。应用开发者不需要知道边缘有什么 GPU，不需要关心 Device Plugin 的配置方式，只需要声明算力需求——其他一切由平台自动处理。

这是第二章提出的"应用与硬件解耦"的关键实现。换 GPU 不改应用，加 GPU 不改调度。

下一章：第五章 — 组织与安全，多团队安全共享应用平台

第五章：组织与安全 — 多团队安全共享应用平台

应用面对的问题：多个团队共用一个平台开发和部署应用，怎么隔离？谁能部署到哪里？谁能看到什么？

引言：应用越多，权限越关键

前两章解决了应用的供应链（第六章）和部署（第七章）问题。但在实际企业环境中，一个边缘平台不是只有一个团队在使用——AI 算法团队在开发质检模型，运维团队在管理集群，工厂 A 的人员不应该看到工厂 B 的数据。

当平台上的应用数量从个位数增长到数十个、使用团队从一个增长到多个时，权限管理就从"有比没有好"变成了"必须精确到位"。一个权限设计不当的平台会带来两类问题：

- 安全风险：开发者可以访问不属于自己的生产数据；实习生可以删除关键部署。
- 管理混乱：无法回答"谁在什么时间做了什么操作"；出了事故无法定位责任人。

本章介绍平台的多租户隔离架构、六层权限模型、认证审计机制和数据安全策略。

5.1 多租户隔离架构

问题：多个团队如何在一个平台上互不干扰

在边缘计算场景中，“多租户”不是一个抽象的企业管理概念，而是非常具体的物理隔离需求：

- 工厂 A 的运维人员不应该看到工厂 B 的节点和应用
- 算法团队只需要开发和提交应用，不需要管理集群基础设施
- 平台管理员需要审核所有应用、管理所有集群，但不直接操作业务应用

三层隔离模型

平台通过 Workspace（工作空间）、Namespace（项目）和 NodeGroup（节点组）三个 CRD 实现逻辑隔离：

```
Platform (平台全局)
├── Cluster A (北京集群)
│   ├── NodeGroup: beijing-factory-a
│   └── NodeGroup: beijing-factory-b
├── Cluster B (上海集群)
│   └── NodeGroup: shanghai-factory
├── Workspace: ai-team (算法团队工作空间)
│   ├── Namespace: ai-team-dev (开发项目)
│   └── Namespace: ai-team-prod (生产项目)
└── Workspace: ops-team (运维团队工作空间)
    └── Namespace: ops-monitoring (监控项目)
```

Workspace CRD — 团队的逻辑边界

Workspace 是团队级别的隔离边界。每个 Workspace 拥有独立的成员列表、资源配额和项目空间。

```
type WorkspaceSpec struct {
    Manager string `json:"manager,omitempty"` // 工作空间管理员
    Template *WorkspaceTemplate `json:"template,omitempty"` // 模板配置
}

type WorkspaceTemplate struct {
    Namespaces []string `json:"namespaces,omitempty"` // 关联的命名空间列表
    ResourceQuotas map[string]string `json:"resourceQuotas,omitempty"` // 资源配额
}

type WorkspaceStatus struct {
    Phase string `json:"phase,omitempty"` // 工作空间状态
    NamespaceCount int32 `json:"namespaceCount,omitempty"` // 命名空间数量
    MemberCount int32 `json:"memberCount,omitempty"` // 成员数量
}
```

— [api/scope/v1alpha1/workspace_types.go](#)

Workspace Controller 负责两项核心工作：

1. 自动初始化默认角色：每个新创建的 Workspace 自动获得一组标准角色（workspace-admin、workspace-viewer、workspace-regular、workspace-self-provisioner），运维人员无需手动配置。

源码实证：

```
`go

// 动态查询 scope="workspace" 的 RoleTemplate，自动创建默认角色
if err := roletemplate.InitializeDefaultRoles(ctx, r.Client, workspace,
constants.ScopeTypeWorkspace); err != nil {
return ctrl.Result{RequeueAfter: time.Minute}, err
}
、
```

— [internal/controller/tenant/workspace_controller.go](#)

2. 统计信息维护：定期更新 Workspace 下的命名空间数量和成员数量，供管理界面展示。成员计数通过扫描 IAMRoleBinding 中的 Subject 去重实现，确保同一用户在不同角色下只计数一次。

NodeGroup CRD — 物理站点的权限边界

NodeGroup 不仅是应用部署的拓扑单元（第七章），也是权限控制的重要边界。

```
type NodeGroupSpec struct {
    Selector *NodeSelector `json:"selector,omitempty"` // 节点选择器
    Template *NodeGroupTemplate `json:"template,omitempty"` // 节点组模板
}
```

```
type NodeSelector struct {  
    MatchLabels map[string]string `json:"matchLabels,omitempty"`  
    MatchExpressions []metav1.LabelSelectorRequirement `json:"matchExpressions,omitempty"`  
}
```

— [api/scope/v1alpha1/nodegroup_types.go](#)

NodeGroup 级别的权限控制意味着：“北京工厂的运维人员可以管理北京工厂的节点，但看不到上海工厂的节点”不是运维约定，而是系统级强制。

5.2 六层权限模型

设计理念：Scope-Aware 授权链

Kubernetes 原生的 RBAC (Role-Based Access Control) 是单集群、两层模型——Role (Namespace 级) 和 ClusterRole (集群级)。这在边缘多租户场景下严重不足：

- 没有 Workspace 的概念——无法限制"这个用户只能在算法团队的工作空间内操作"
- 没有 NodeGroup 的概念——无法限制"这个运维人员只能管理北京工厂的节点"
- 没有跨集群的概念——无法限制"这个管理员只能管理北京集群"

平台在 K8s 原生 RBAC 的基础上，扩展了六层 Scope (作用域) 授权模型：

```
Platform ( 平台全局 ) ← 最高权限，平台管理员
↓
CrossCluster ( 跨集群 ) ← 跨集群路由，访问其他集群资源时触发
↓
Cluster ( 集群级 ) ← 集群管理员
↓
Workspace ( 工作空间级 ) ← 工作空间管理员
↓
NodeGroup ( 节点组级 ) ← 站点运维人员
↓
Namespace ( 项目级 ) ← 项目开发者
```

源码实证：`pkg/constants/scope.go` 定义了全部六种 Scope 类型：`platform`、`cross-cluster`、`cluster`、`workspace`、`nodegroup`、`namespace`。其中 `cross-cluster` 在请求目标集群与当前集群不同时由 `extractScopeFromRequestInfo` 提前识别并处理，其余五层由 `checkScopeChain` 按就近原则逐级检查。

授权检查流程

当一个用户发起 API 请求时，Authorizer 按照从最具体到最宽泛的顺序检查权限，找到匹配的权限即停止（短路优化）：

源码实证：

```
`go
func (a *Authorizer) checkScopeChain(ctx context.Context,
    attrs authorizer.Attributes, chain ScopeChain) AuthResult {
>
    // Level 1: Namespace（最具体）
    if chain.Namespace != "" {
        if result := a.checkAtScope(ctx, attrs,
            constants.ScopeTypeNamespace, chain.Namespace); result.Allowed {
            result.Level = constants.ScopeTypeNamespace
            return result
        }
    }
>
    // Level 2: Workspace
    if chain.Workspace != "" {
        if result := a.checkAtScope(ctx, attrs,
            constants.ScopeTypeWorkspace, chain.Workspace); result.Allowed {
            result.Level = constants.ScopeTypeWorkspace
            return result
        }
    }
>
    // Level 3: NodeGroup
    if chain.NodeGroup != "" {
        if result := a.checkAtScope(ctx, attrs,
            constants.ScopeTypeNodegroup, chain.NodeGroup); result.Allowed {
            result.Level = constants.ScopeTypeNodegroup
            return result
        }
    }
}
```



```
    }  
    }  
>  
    // Level 4: Cluster  
    if chain.Cluster != "" {  
        if result := a.checkAtScope(ctx, attrs,  
            constants.ScopeTypeCluster, chain.Cluster); result.Allowed {  
            result.Level = constants.ScopeTypeCluster  
            return result  
        }  
    }  
>  
    // Level 5: Platform (最宽泛)  
    if chain.Platform != "" {  
        if result := a.checkAtScope(ctx, attrs,  
            constants.ScopeTypePlatform, chain.Platform); result.Allowed {  
            result.Level = constants.ScopeTypePlatform  
            return result  
        }  
    }  
>  
    // 兜底：匿名角色（系统级默认权限）  
    if result := a.checkAnonymousRole(ctx, attrs); result.Allowed {  
        result.Level = constants.AuthLevelAnonymous  
        return result  
    }  
>  
    return &AuthResult{  
        Allowed: false,  
        Decision: authorizer.DecisionDeny,  
        Reason: fmt.Sprintf("no RBAC permission for %s:%s:%s",
```

```

    user, verb, resource),
  }
}
、
— internal/authorizer/authorizer.go:358-433

```

两个核心原则

1. 就近原则（**Proximity Principle**）：从最具体的 Scope 开始检查。一个在 Namespace 级别有权限的用户，不需要向上搜索 Workspace 或 Cluster 级别的权限。
2. 权限继承（**Permission Accumulation**）：高层级的权限自动覆盖低层级。一个拥有 Platform Admin 权限的用户，自动拥有所有 Cluster、Workspace、NodeGroup 和 Namespace 的权限——无需在每个层级重复绑定。

Scope 提取机制

Authorizer 需要从每个 API 请求中提取出 Scope 上下文（“这个请求属于哪个 Workspace？哪个 NodeGroup？”）。平台通过三种机制实现 Scope 提取：

机制一：ScopePattern — URL 模式匹配

ScopePattern CRD 定义了 URL 路径与 Scope 的映射关系：

```

type ScopePatternSpec struct {
    Pattern string `json:"pattern"` // URL 模式，如 "/api/edge/{nodegroup}/*"
    ScopeType string `json:"scopeType"` // 提取的 Scope 类型
}

```

— api/iam/v1alpha1/scopepattern_types.go

例如，请求 `GET /api/edge/beijing-factory-a/nodes/node-123` 会被模式 `/api/edge/{nodegroup}/*` 匹配，提取出 `nodegroup = beijing-factory-a`，然后在 NodeGroup 层级检查权限。

机制二：Namespace Label 传播

Namespace 资源上的 Label（如 `theriseunion.io/workspace: ai-team`）被用来推导其所属的 Workspace。对于 Namespace 级别的请求，Authorizer 自动从 Namespace 的 Label 中提取 Workspace 和 Cluster 信息。

机制三：请求信息解析

RequestInfoResolver 从 HTTP 请求中解析出 Cluster、Resource、Verb 等信息，为 Scope 提取提供基础数据。

IAM CRD 体系

六层权限模型通过 7 个 IAM CRD 实现：

CRD	职责	关键设计
User	用户身份	bcrypt 密码、Active/Pending/Disabled 状态
IAMRole	角色定义	兼容 K8s PolicyRule + UI 权限 + RoleTemplate 聚合
IAMRoleBinding	角色绑定	Scope Label 标记作用域
RoleTemplate	权限模板	可复用的权限集合，按 Scope 类型组织
Category	分类	将 RoleTemplate 组织为类别，支持 i18n
ScopePattern	Scope 提取	URL 模式 Scope 映射
LoginRecord	登录审计	记录每次认证的成功/失败

IAMRole 的双层设计

IAMRole 的设计兼顾了 K8s 原生生态兼容和企业管理需求：

```
type IAMRoleSpec struct {
    // 标准 K8s RBAC PolicyRule，完全兼容 K8s 生态
    Rules []rbacv1.PolicyRule `json:"rules,omitempty"`

    // 前端 UI 权限标识符
    UIPermissions []string `json:"uiPermissions,omitempty"`

    // RoleTemplate 聚合，组合式权限构建
    AggregationRoleTemplates *AggregationRoleTemplates
    `json:"aggregationRoleTemplates,omitempty"`
}
```

— `api/iam/v1alpha1/iamrole_types.go`

K8s 兼容层：`Rules` 字段使用标准的 `rbacv1.PolicyRule`，IAMRole Controller 将其自动转换为 K8s ClusterRole。这意味着平台的权限模型完全建立在 K8s 原生 RBAC 之上——不是替代，而是扩展。

源码实证：IAMRole Controller 将每个 IAMRole 转换为带有 Scope Label 的 ClusterRole：

```
`go
return &rbacv1.ClusterRole{
    ObjectMeta: metav1.ObjectMeta{
        Name: clusterRoleName,
        Labels: map[string]string{
            "iam.theriseunion.io/managed": "true",
            "iam.theriseunion.io/scope": naming.GetScope(iamRole),
            "iam.theriseunion.io/scope-value": naming.GetScopeValue(iamRole),
        },
    },
},
```

```
Rules: rules,
```

```
}
```

```
、
```

```
— internal/controller/iam/iamrole_controller.go
```

UI 权限层：`UIPermissions` 字段允许前端根据用户角色动态显示或隐藏功能入口——后端 API 权限和前端界面权限通过同一个角色定义统一管理。

模板聚合层：通过 `AggregationRoleTemplates`，管理员可以将多个 `RoleTemplate` 组合成一个 `IAMRole`，而不需要手动拼装 `PolicyRule`。例如，“工作空间管理员”角色可以由“应用管理”、“成员管理”、“资源查看”三个模板聚合而成。

跨集群权限同步

平台级和集群级的角色需要在所有成员集群中生效。`IAMRole Controller` 和 `IAMRoleBinding Controller` 在检测到 Platform 级别的角色变更时，自动将其同步到所有成员集群。

源码实证：

```
`go
```

```
// 平台级角色自动同步到成员集群
```

```
scope := naming.GetScope(iamRole)
```

```
if scope == v1alpha1.ScopePlatform {
```

```
if err := r.syncRoleToMemberClusters(ctx, iamRole); err != nil {
```

```
klog.ErrorS(err, "Failed to sync role to member clusters",
```

```
"role", iamRole.Name)
```

```
}
```

```
}
```

```
、
```

```
— internal/controller/iam/iamrole_controller.go
```

授权缓存

六层 Scope 检查需要多次查询 `ClusterRole/ClusterRoleBinding`，性能开销不可忽视。平台通过 `SimpleCache` 实现授权缓存：

```
type SimpleCache struct {
    data map[string]*cacheEntry // 授权结果缓存
    namespaceLabels map[string]*namespaceCacheEntry // Namespace Label 缓存
    mu sync.RWMutex
    ttl time.Duration // 默认 5 分钟
```

```
}
```

— `internal/authorizer/cache.go`

缓存 Key 设计：`user:verb:resource:scopeType:scopeValue:resourceName`。注意 Key 中包含了 `resourceName`——这确保了即使缓存了用户对"pods"资源的权限，也不会错误地允许访问特定名称的 Pod（如果用户只有对部分 Pod 的权限）。

CRD 变化触发失效：CacheInvalidator 是一个独立的 Controller，监听 IAMRoleBinding、IAMRole 和 ClusterRoleBinding 的变化事件。当角色或绑定发生变更时，精确失效相关用户的缓存条目：

源码实证：

```
`go
func (c *CacheInvalidator) SetupWithManager(mgr manager.Manager) error {
    return ctrl.NewControllerManagedBy(mgr).
        For(&iamv1alpha1.IAMRoleBinding{}).
        Watches(&iamv1alpha1.IAMRole{}),
        handler.EnqueueRequestsFromMapFunc(c.mapIAMRoleToUsers)).
        Watches(&rba1.ClusterRoleBinding{}),
        handler.EnqueueRequestsFromMapFunc(c.mapClusterRoleBindingToUsers)).
        Complete(c)
}
```

— `internal/authorizer/cache_invalidator.go`

双重失效机制：TTL 过期是兜底策略（5 分钟内权限变更一定生效），CRD 变化触发是即时策略（权限变更秒级生效）。两者结合确保了性能与一致性的平衡。

5.3 认证与审计

OAuth2/OIDC Server

平台内置了完整的 OAuth2/OIDC 认证服务器，支持四种授权模式：

授权模式	用途	适用场景
Password Grant	用户名/密码直接认证	管理控制台登录
Authorization Code	OAuth2 标准授权码流程	第三方应用集成
Refresh Token	Token 续期	长时间会话保持
Client Credentials	服务账号认证	服务间调用

说明：生产环境默认启用 HMAC 模式（`pkg/oapis/oauth/hmac_handler.go`），该模式仅实现 Password Grant 和 Client Credentials 两种授权模式，其余模式返回 `unsupported_grant_type`。Authorization Code 和 Refresh Token 需切换至 RSA 模式方可使用。

— `pkg/oapis/oauth/handler.go`

JWT Token 结构

认证成功后，平台签发 JWT（JSON Web Token），包含用户的基本信息和组成员关系：

```
type Claims struct {
    Subject string `json:"sub"` // 用户名
    Name string `json:"name,omitempty"` // 显示名称
    PreferredUsername string `json:"preferred_username,omitempty"`
    Email string `json:"email,omitempty"`
    Groups []string `json:"groups,omitempty"` // 用户组
    Issuer string `json:"iss,omitempty"` // 签发者
    Audience string `json:"aud,omitempty"` // 受众
}
```

```
ExpiresAt time.Time `json:"exp,omitempty"` // 过期时间
}
```

— `pkg/oapis/oauth/types.go`

Token 使用 **RS256** (RSA + SHA-256) 签名算法，确保不可篡改。Authorizer 通过验证 Token 签名获取用户身份，然后执行六层 Scope 授权检查。

LoginRecord — 登录审计记录

每次认证尝试（无论成功还是失败）都会创建一条 LoginRecord CRD：

```
type LoginRecordSpec struct {
    Type string `json:"type"` // 认证方式: Password/OAuth/Token
    Provider string `json:"provider"` // 认证提供者: LDAP/GitHub 等
    SourceIP string `json:"sourceIP"` // 来源 IP
    Success bool `json:"success"` // 是否成功
    Reason string `json:"reason"` // 结果原因
    UserAgent string `json:"userAgent"` // 客户端标识
}
```

— `api/iam/v1alpha1/loginrecord_types.go`

LoginRecord 通过 Label `iam.theriseunion.io/username` 关联到用户，支持按用户名快速查询其登录历史。

源码实证：认证失败时同样记录，用于安全审计和入侵检测：

```
`go
// 密码验证失败
if !utils.VerifyPassword(password, user.Spec.EncryptedPassword) {
    ua.createLoginRecord(ctx, username, "Password", "LDAP", req,
        false, "invalid password")
    return nil, fmt.Errorf("invalid password")
}
>
// 认证成功
ua.createLoginRecord(ctx, username, "Password", "LDAP", req,
    true, "Login successful")
、
— pkg/oapis/oauth/user_auth.go
```

审计价值：LoginRecord 不仅记录“谁在什么时间登录了”，还记录“谁尝试登录但失败了”。后者对安全分析至关重要——短时间内大量失败的登录尝试可能意味着暴力破解攻击。

5.4 数据安全

密码安全

平台对用户密码的处理遵循三项原则：

原则一：bcrypt 加密存储

所有密码使用 bcrypt 算法加密存储，成本因子（Cost）为 10（约 100ms 计算时间）：

```
func HashPassword(password string) (string, error) {
    bytes, err := bcrypt.GenerateFromPassword([]byte(password), bcrypt.DefaultCost)
    if err != nil {
        return "", err
    }
    return string(bytes), nil
}
```

— pkg/utils/password.go

bcrypt 的核心优势在于自适应安全——随着硬件算力提升，可以增加 Cost 参数来保持破解难度。当前 Cost=10 意味着即使攻击者获取了数据库中的密码哈希，每次暴力破解尝试也需要约 100ms，使大规模暴力破解在时间上不可行。

原则二：写保护（Write Protection）

密码哈希字段 EncryptedPassword 是只写的——API 响应中永远不返回密码哈希：

源码实证：

```
`go

func (h Handler) GetUser(request restful.Request,
response *restful.Response) {
    // ...
```



```
// 返回响应前清除密码字段
responseUser := user.DeepCopy()
responseUser.Spec.EncryptedPassword = ""
response.WriteEntity(responseUser)
}
```

— pkg/oapis/iam/v1alpha1/handler_user.go

这确保了即使有人获取了 API 访问权限（如通过泄露的 Token），也无法从 API 响应中获取密码哈希。

原则三：密码复杂度校验

创建用户和重置密码时，平台强制执行密码复杂度校验：

```
complexity := utils.DefaultPasswordComplexity()
if err := utils.ValidatePassword(userReq.Spec.Password, complexity); err != nil {
    response.WriteError(http.StatusBadRequest,
        fmt.Errorf("password does not meet complexity requirements: %v", err))
    return
}
```

— pkg/oapis/iam/v1alpha1/handler_user.go

User 状态管理

用户 CRD 通过 `Status.State` 字段管理账号的生命周期：

状态	含义	是否可登录
Active	正常活跃	是
Pending	等待激活	否
Disabled	已禁用	否

认证时，系统检查用户状态——只有 Active 状态的用户才能通过认证：

```
if user.Status.State != "" && user.Status.State != iamv1alpha1.UserActive {
    ua.createLoginRecord(ctx, username, "Password", "LDAP", req,
        false, fmt.Sprintf("user account is not active: %s", user.Status.State))
    return nil, fmt.Errorf("user account is not active: %s", user.Status.State)
}
```

— pkg/oapis/oauth/user_auth.go

敏感信息保护

平台在多个层面防止敏感信息泄露：

层面	保护措施
API 响应	密码哈希字段在返回前清除
日志输出	密码和 Token 不记录到日志
CRD 存储	密码使用 bcrypt 单向加密，即使 etcd 被泄露也无法还原
网络传输	JWT Token 使用 RS256 签名，防篡改

5.5 与竞品的差异

能力	本平台	云厂商方案	Rancher
权限层级	5 层（PlatformClusterWorkspaceNodeGroupNamespace）	2 层（云 IAM + Namespace）	3 层（GlobalClusterProject）
Workspace 隔离	CRD 原生支持	无	无
NodeGroup 权限	节点组级别细粒度控制	无	无
认证方式	内置 OAuth2/OIDC（不依赖云服务）	依赖各自云 IAM	内置
审计追踪	LoginRecord CRD 原生记录	依赖云审计服务	有（审计日志）
授权缓存	TTL + CRD 变化触发失效	云端缓存	无缓存

关键差异：云厂商的权限体系依赖各自的云 IAM 系统——华为 IAM、阿里 RAM、腾讯 CAM。其粒度止于 Namespace 级别，无法实现“节点组 A 的运维人员不能操作节点组 B”的场景。本平台的 5 层 Scope 模型是专为边缘多团队场景设计的。

Workspace 和 NodeGroup 两个层级是竞品中不存在的——在大型企业的边缘场景中，这两个层级恰好对应“业务团队”和“物理站点”两个最重要的隔离边界。

5.6 本章小结

组织与安全回答了"多团队怎么安全共享一个平台"的问题：

环节	机制	效果
团队隔离	Workspace CRD + 自动角色初始化	每个团队拥有独立的工作空间，默认角色自动就位
站点隔离	NodeGroup 级权限控制	工厂 A 的运维看不到工厂 B 的节点
六层授权	CrossClusterNamespaceWorkspaceNodeGroupClusterPlatform	精确到每个层级的权限控制
认证安全	OAuth2/OIDC + JWT RS256 + bcrypt 密码	内置认证，不依赖外部服务
审计追踪	LoginRecord CRD	每次登录（成功/失败）可查可追溯
性能优化	SimpleCache（TTL 5min + CRD 变化触发失效）	授权延迟毫秒级，权限变更秒级生效
K8s 兼容	IAMRole ClusterRole 自动转换	完全建立在 K8s 原生 RBAC 之上

这套权限体系的核心价值在于：让"谁能在哪里做什么"成为系统级保证，而非运维约定。当多个团队共享一个边缘平台时，权限的精确性直接决定了平台的可信度。六层 Scope 模型确保了从全局管理员到项目开发者，每个角色只能看到和操作属于自己的资源。

这是第二章提出的"应用并行能力"的制度性保障——多个团队的多个应用可以安全地在同一硬件上并行运行，互不干扰。

下一章：第八章 — 应用运行，断网不断服务

第六章：应用供应链 —— 从开发到上架

应用面对的问题：应用怎么打包？支持哪些部署形态？怎么让其他团队用到我的应用？
怎么审核质量？

引言：应用的"最后一公里"不是部署，是上架

当我们谈论"应用到达边缘"时，最容易被忽视的环节不是部署，而是部署之前的准备：应用怎么定义？怎么打包？怎么让其他团队发现并使用？怎么确保质量？

传统边缘项目中，应用的交付方式通常是"打包一个容器镜像，写一份部署文档，交给运维手动操作"。这种方式在管理 3-5 个应用时尚可接受，但当应用数量增长到数十个、参与团队增长到数个时，就会暴露出根本性的缺陷：

- 无标准化打包：每个应用的部署方式不同，有的是单个容器，有的是 Helm Chart，有的是 AI 模型，运维需要为每种类型编写不同的部署脚本。
- 无质量关卡：任何应用都可以直接部署到生产边缘，缺少审核机制，质量失控的应用可能影响整个站点的稳定性。
- 无共享机制：A 团队开发的优秀应用，B 团队无法发现也无法复用，应用的价值被限制在单个团队内。

本章介绍平台的应用供应链体系——从应用定义、多形态支持、审核上架到版本管理的完整链路。

6.1 三层应用资源模型

设计哲学：类 PVC 的声明式模型

平台的应用管理采用了类似 Kubernetes PersistentVolumeClaim (PVC) 的设计模式。就像用户通过 PVC 声明“我需要 10GB 存储”而无需关心底层是 NFS 还是 Ceph，应用开发者通过 Application CRD 声明“这是一个什么应用”而无需关心底层的部署引擎。

源码实证：Application CRD 的设计注释明确标注了这一设计哲学：

```
`go
// ApplicationSpec defines the desired state of Application
// Application 类似 PVC，用户申请的应用定义
、

— api/app/v1alpha1/application_types.go:24
```

三层资源层级

应用资源通过三层 CRD 组织：

```
Application (应用定义)
├── ApplicationVersion v1.0.0 (版本 1，包含具体配置)
├── ApplicationVersion v1.1.0 (版本 2，配置变更)
└── ApplicationVersion v2.0.0 (版本 3，大版本更新)
    ├── ApplicationDeployment (部署到北京站点)
    └── ApplicationDeployment (部署到上海站点)
```

每一层的职责分离清晰：

CRD 层级	职责	作用域	关键字段
Application	应用的身份定义	Cluster 级	Provisioner 类型 (Label)、类别、所属 Workspace
ApplicationVersion	应用的具体配置	Cluster 级	版本号、WorkloadConfig / HelmConfig / ModelConfig、审核状态
ApplicationDeployment	应用的部署实例	Namespace 级	引用的应用版本、部署拓扑列表、运行状态

为什么 **Application** 的 **Spec** 几乎为空？

一个值得注意的设计决策是：**ApplicationSpec** 结构体几乎没有字段，所有元数据通过 Labels 和 Annotations 管理。

```
type ApplicationSpec struct {
    // Spec 当前为空，所有配置通过 Labels 和 Annotations 管理
}
```

这不是偶然的简化，而是有意为之的架构选择：

1. **Provisioner** 类型不可变：通过 Label `app.theriseunion.io/provisioner` 指定，Webhook 验证确保创建后不可修改。这保证了应用的部署引擎在整个生命周期内一致。
2. 元数据灵活扩展：显示名称、描述、分类等通过 Annotations 存储，无需修改 CRD Schema 即可扩展新的元数据。
3. 统计信息外置：版本计数、部署计数等统计数据不存储在 CRD Status 中，而是通过可观测系统 (Prometheus) 查询，避免 Controller 复杂度增加和 etcd 写入压力。

源码实证：

```
`go
// 注意：统计信息（如 versionCount, latestVersion）应通过可观测系统（Prometheus）查询
// 不应存储在 CRD Status 中，避免增加 Controller 复杂度和 etcd 压力
、
— api/app/v1alpha1/application_types.go:48-49
```


6.2 可插拔 Provisioner 架构

问题：不同应用需要不同的部署引擎

边缘场景下的应用形态远比云端复杂：

- 标准微服务：一个 Nginx 反向代理 + 一个业务 API 容器，需要 Kubernetes Deployment。
- 复杂多组件应用：一个完整的监控栈（Prometheus + Grafana + AlertManager），需要 Helm Chart 统一管理。
- AI 推理模型：一个 PyTorch 质检模型，需要特定的推理运行时（TorchServe）和 GPU 资源配置。

如果为每种应用类型编写独立的管理逻辑，代码会迅速膨胀且难以维护。

解决方案：类 CSI 的 Provisioner 插件模式

平台借鉴了 Kubernetes CSI（Container Storage Interface）的设计思想：通过 Provisioner 标识路由到不同的部署引擎，所有 Provisioner 实现统一的操作接口。

三种 Provisioner 类型

1. Workload Provisioner（标准微服务）

适用场景：由一个或多个容器组成的标准 Kubernetes 工作负载。

```
type WorkloadConfig struct {  
    // Deployments 定义 Deployment 列表  
    Deployments []WorkloadDeployment `json:"deployments,omitempty"`  
}  
  
type WorkloadDeployment struct {
```

```

    Name string `json:"name"` // 工作负载名称
    Spec DeploymentSpec `json:"spec"` // 简化的 Deployment 规格
}

```

Workload Provisioner 直接将 `WorkloadConfig` 中定义的 Deployment 转换为 Kubernetes 原生 Deployment 资源。一个 `ApplicationVersion` 可以包含多个 `WorkloadDeployment`，支持多容器组合的应用定义。

源码实证：Controller 在 Reconcile 过程中遍历 `WorkloadConfig.Deployments` 列表，为每个 Deployment 配置创建对应的 K8s Deployment 资源。

— `internal/controller/application/applicationdeployment_controller.go:666`

2. Helm Provisioner（复杂多组件应用）

适用场景：使用 Helm Chart 打包的复杂应用，包含多个 K8s 资源（Deployment、Service、ConfigMap、PVC 等）。

```

type HelmConfig struct {
    ChartName string `json:"chartName"` // Helm Chart 名称
    ChartVersion string `json:"chartVersion"` // Helm Chart 版本
    Repository string `json:"repository"` // Helm 仓库地址
    Values string `json:"values,omitempty"` // Helm Values (YAML 格式)
}

```

Helm Provisioner 利用 Helm 的成熟生态，支持：

- 标准 Helm 仓库的 Chart 引用
- 自定义 Values 覆盖
- Helm 的版本管理与回滚能力

3. Model Provisioner（AI 模型）

适用场景：AI 推理模型的部署，需要特定的模型框架、推理运行时和 GPU 资源。

```

type ModelConfig struct {
    ModelName string `json:"modelName"` // 模型名称
    ModelVersion string `json:"modelVersion"` // 模型版本
    Framework string `json:"framework"` // 框架 (PyTorch/TensorFlow/ONNX)
    Runtime string `json:"runtime"` // 运行时 (TorchServe/Triton 等)
    StorageURI string `json:"storageURI"` // 模型存储地址
    Replicas *int32 `json:"replicas"` // 副本数
    GPU int32 `json:"gpu"` // GPU 数量
    CPURequest string `json:"cpuRequest"` // CPU 请求
    MemoryRequest string `json:"memoryRequest"` // 内存请求
}

```

Model Provisioner 将 AI 模型的部署抽象为声明式配置：开发者只需声明“用 PyTorch 框架、TorchServe 运行时、需要 1 个 GPU”，平台自动处理运行时环境配置、模型文件加载和服务暴露。

Provisioner 的路由机制

Provisioner 类型通过 Application CRD 的 Label 指定，实现了配置与路由的解耦：

```
metadata:
  labels:
    app.theriseunion.io/provisioner: "workload" # 或 "helm" 或 "model"
```

Controller 在 Reconcile 时根据此 Label 选择对应的处理逻辑。这意味着：

- 添加新的 **Provisioner** 类型不需要修改已有代码——只需注册新的处理函数
- 应用定义与 **Provisioner** 实现解耦——Application 不知道自己将被哪种引擎部署
- 同一套管理 **API** 覆盖所有应用类型——前端不需要为不同类型的应用编写不同的管理界面

与竞品的差异

能力	本平台	华为 IEF	阿里 ACK@Edge
应用类型抽象	三种 Provisioner 统一接口	容器应用 + 函数应用	容器应用
AI 模型支持	原生 ModelConfig (框架+运行时+GPU)	需自行编排	需自行编排
扩展性	新增 Provisioner 只需实现接口	固定类型	固定类型
Helm 生态	原生 HelmConfig 支持	有限支持	有限支持

6.3 应用商店与审核流程

设计理念：借鉴 App Store 模式

平台的应用商店借鉴了移动端 App Store 的核心模式：工作空间开发 → 平台审核 → 全域可见。

这解决了企业内部应用共享的三个关键问题：

1. 发现性：B 团队如何发现 A 团队开发的可复用应用？
2. 质量保障：未经验证的应用是否会影响其他团队的稳定性？
3. 权限控制：谁有权上架应用？谁有权审核？

审核状态机

每个 `ApplicationVersion` 拥有独立的审核状态（ReviewPhase），状态流转如下：



源码实证：审核状态定义清晰，三种状态互斥：

```
`go
const (
    ReviewPhasePending ReviewPhase = "Pending" // 待审核
```

```

ReviewPhaseApproved ReviewPhase = "Approved" // 已通过
ReviewPhaseRejected ReviewPhase = "Rejected" // 已拒绝
)
、

```

— [api/app/v1alpha1/applicationversion_types.go:28-35](#)

审核流程的完整链路

第一步：Workspace 开发者提交审核

开发者在自己的工作空间完成应用开发和测试后，调用提交接口：

```
POST /workspaces/{workspace}/applications/{application}/versions/{version}/review
```

系统执行以下验证：

1. 验证应用存在且属于指定 Workspace (Label 校验)
2. 查找匹配的版本 (通过 [app.theriseunion.io/application](#) Label 过滤)
3. 检查版本未曾提交过 (ReviewPhase 为空)
4. 设置 `ReviewPhase = Pending`，记录提交者和提交时间

源码实证：提交审核时会进行严格的所属验证：

```

`go
if app.Labels["theriseunion.io/workspace"] != workspace {
// 应用不属于此 Workspace，返回 404
}
、

```

— [pkg/oapis/app/v1alpha1/handler_review.go:102](#)

第二步：平台管理员审核

管理员可以查看所有待审核的应用版本，逐个审核通过或拒绝：

```
POST /applicationversions/{name}/approve # 审核通过
POST /applicationversions/{name}/reject # 审核拒绝
```

审核操作具备幂等性——对已通过的版本重复调用 `approve` 不会报错，直接返回成功：

```

// 幂等性：如果已经是 Approved，直接返回成功
if appVersion.Status.ReviewPhase == appv1alpha1.ReviewPhaseApproved {
    resp.WriteHeaderAndEntity(http.StatusOK, appVersion)
    return
}

```

— [pkg/oapis/app/v1alpha1/handler_review.go:185-188](#)

第三步：全局商店展示

审核通过后，应用出现在全局商店的浏览列表中。商店的 API 设计确保只有审核通过的版本才可见：

```
GET /store/applications # 浏览所有有审核通过版本的应用
GET /store/applications/{name} # 查看应用详情
GET /store/applications/{name}/versions # 列出审核通过的版本
```

源码实证：商店浏览接口严格过滤审核状态：

```
`go
// 只返回 ReviewPhase=Approved 的应用版本
for _, version := range versionList.Items {
    if version.Status.ReviewPhase == appv1alpha1.ReviewPhaseApproved {
        approvedApps[version.Spec.ApplicationRef] = true
    }
}
`
```

— pkg/oapis/app/v1alpha1/handler_store.go:96-98

同一应用的多版本并行审核

一个关键设计：同一个 **Application** 可以有多个 **ApplicationVersion** 处于不同的审核状态。例如：

版本	审核状态	说明
v1.0.0	Approved	当前生产版本，全局商店可见
v1.1.0	Pending	新功能版本，等待审核
v2.0.0	（未提交）	开发中，仅在 Workspace 内可用

这支持了应用的持续迭代——开发者可以在不影响已上架版本的情况下，提交新版本等待审核。

商店搜索能力

全局商店支持按 Provisioner 类型过滤和关键词搜索：

```
GET /store/applications?provisioner=model&search=质检
```

搜索覆盖应用名称、显示名称和描述三个维度，不区分大小写。

6.4 版本管理与资源所有权

版本的生命周期

每个 `ApplicationVersion` 记录完整的审核信息：

```
type ApplicationVersionStatus struct {
    Phase string // 版本阶段
    ReviewPhase ReviewPhase // 审核状态
    ReviewMessage string // 审核信息（拒绝原因等）
    ReviewTime *metav1.Time // 审核时间
    Reviewer string // 审核人
    Conditions []metav1.Condition // 状态条件
}
```

这确保了完整的审计追踪：谁在什么时间以什么理由通过或拒绝了这个版本。

资源所有权链

应用资源的层级关系通过 Kubernetes OwnerReference 机制管理：

```
Application
├── ApplicationVersion (通过 spec.applicationRef 引用)
│   └── ApplicationDeployment (通过 spec.applicationVersion 引用)
│       ├── K8s Deployment (OwnerReference 指向 ApplicationDeployment)
│       └── ReplicaSet → Pod
```

这个所有权链的关键设计：

1. **Finalizer 保护**：`ApplicationDeployment` 注册了 `applicationdeployment.finalizer.app.therisunion.io`。删除时，Finalizer 确保先清理所有底层 K8s Deployment，再移除 Finalizer 允许删除。

源码实证：

```
`go
```

```
const TopologyFinalizer = "applicationdeployment.finalizer.app.theriseunion.io"
>
// 正在删除，清理资源
if hasStringTopology(appDeployment.ObjectMeta.Finalizers, TopologyFinalizer) {
if err := r.cleanupTopologies(ctx, appDeployment); err != nil {
return ctrl.Result{}, err
}
// 移除 Finalizer
appDeployment.ObjectMeta.Finalizers = removeStringTopology(...)
}
、
— internal/controller/application/applicationdeployment_controller.go:47,115-127
```

2. **OwnerReference** 级联：K8s Deployment 设置了 `Controller: true` 和 `BlockOwnerDeletion: true` 的 OwnerReference 指向 ApplicationDeployment。这意味着删除

ApplicationDeployment 会自动触发 K8s 的垃圾回收机制，级联删除所有底层 Deployment。

3. 双重保护：Finalizer 确保清理逻辑的执行，OwnerReference 确保即使 Finalizer 逻辑有遗漏，K8s 原生的垃圾回收仍会兜底清理。

6.5 本章小结

应用供应链回答了"应用怎么从开发走向生产"的问题：

环节	机制	效果
应用定义	类 PVC 的三层 CRD 模型	身份、版本、部署三层分离
多形态支持	可插拔 Provisioner (Workload/Helm/Model)	一套管理接口覆盖所有应用类型
质量关卡	ReviewPhase 三态审核	Workspace 开发 平台审核 全域可见
应用发现	全局应用商店 + 搜索过滤	跨团队应用共享和复用
生命周期保护	Finalizer + OwnerReference 双重保护	删除安全，不残留孤儿资源

这套供应链体系的核心价值在于：让应用的价值不被限制在单个团队、单个站点内。一个团队的优秀应用，经过审核后可以被全平台共享和复用——这是第二章提出的"应用价值持续释放"的制度性保障。

下一章：第七章 — 应用部署，到达每一个边缘

第七章：应用部署 —— 到达每一个边缘

应用面对的问题：同一个应用要部署到北京、上海、云端多个位置，怎么统一管理？版本怎么批量切换？部署状态怎么一眼看全？

引言：部署不是一次性操作，是持续管理

在第六章中，应用完成了打包、审核和上架。接下来的问题是：应用怎么从商店到达每一个边缘站点？

传统方式下，部署一个应用到 N 个边缘站点意味着重复 N 次操作：登录每个站点的管理界面、选择应用和版本、配置参数、确认部署。当站点数量从 5 个增长到 50 个、500 个时，这种方式的运维成本呈线性增长。

更棘手的问题是版本管理。当质检模型从 v1.0 升级到 v1.1 时，运维人员需要：

1. 记住哪些站点部署了这个模型
2. 逐个站点执行版本更新
3. 逐个验证更新结果
4. 如果出错，逐个站点回滚

本章介绍平台的 Topology-Based 部署模型——通过一个声明式的 `ApplicationDeployment` 资源，统一管理应用在多个边缘站点的部署、版本切换和状态追踪。

7.1 Topology-Based 云边混合部署

核心概念：一个部署，多个拓扑

`ApplicationDeployment` 是应用部署的核心 CRD。它的设计思想是：用一个资源声明应用在所有目标位置的部署期望，**Controller** 自动为每个位置创建和管理对应的工作负载。

```
apiVersion: app.theriseunion.io/v1alpha1
kind: ApplicationDeployment
metadata:
  name: quality-inspection-deploy
  namespace: manufacturing
spec:
  application: quality-inspection
  version: "1.1.0"
  topologies:
    - nodeGroup: "beijing-factory" # 北京工厂站点
    - nodeGroup: "shanghai-factory" # 上海工厂站点
    - nodeGroup: "guangzhou-factory" # 广州工厂站点
    - nodeGroup: "__cloud__" # 云端运行实例
```

这个简洁的声明背后，Controller 会自动执行以下操作：

1. 解析 `version: "1.1.0"` 找到对应的 `ApplicationVersion` 资源
2. 从 `ApplicationVersion` 中提取 `WorkloadConfig` 配置
3. 为每个 Topology 创建独立的 Kubernetes Deployment
4. 为边缘拓扑设置 `nodeSelector` 确保调度到正确的节点组
5. 为云端拓扑（`__cloud__`）使用默认调度策略
6. 持续追踪每个 Topology 的部署状态

云端与边缘的差异处理

Controller 根据 Topology 的 `nodeGroup` 值自动判断部署类型：

nodeGroup 值	部署类型	nodeSelector	调度策略
<code>__cloud__</code>	云端部署	不设置	K8s 默认调度器决定
其他值（如 <code>beijing-factory</code> ）	边缘部署	<code>nodeGroup: beijing-factory</code>	限定到指定节点组

源码实证：Controller 通过 `__cloud__` 标识区分云端和边缘部署：

```
`go
func (r *ApplicationDeploymentReconciler) isCloudDeployment(
ctx context.Context, appDeployment *appv1alpha1.ApplicationDeployment,
) (bool, error) {
for _, topology := range appDeployment.Spec.Topologies {
if topology.NodeGroup == "__cloud__" {
return true, nil
}
}
return false, nil
}
、
```

— `internal/controller/application/applicationdeployment_controller.go:292-309`

边缘部署时，Controller 为每个 Deployment 设置精确的 `nodeSelector`：

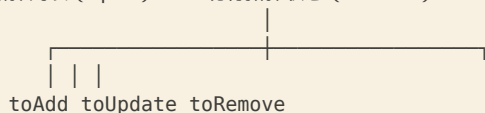
```
// 添加拓扑相关的节点选择器
if topology.NodeGroup != "" {
    deploy.Spec.Template.Spec.NodeSelector[constants.NodeGroupLabelKey] = topology.NodeGroup
}
if topology.NodeName != "" {
    deploy.Spec.Template.Spec.NodeSelector["kubernetes.io/hostname"] = topology.NodeName
}
}
```

— `internal/controller/application/applicationdeployment_controller.go:847-858`

Topology 差异计算

当 `ApplicationDeployment` 的 Topology 列表发生变化时（例如新增了一个站点或移除了一个站点），Controller 通过三路差异计算精确执行操作：

期望拓扑列表（Spec） vs 现有拓扑状态（Status）



```
      (新拓扑) (需更新) (需删除)
      | | |
      | | |
  创建 Deployment 更新配置 删除 Deployment
```

源码实证：

```
`go

func (r *ApplicationDeploymentReconciler) calculateTopologyDifferences(
    expectedTopologies []appv1alpha1.DeploymentTopology,
    existingTopologyMap map[string]appv1alpha1.TopologyStatus,
) ([]topologyWithIndex, []appv1alpha1.TopologyStatus, []appv1alpha1.TopologyStatus) {
    var toAdd []topologyWithIndex
    var toUpdate []appv1alpha1.TopologyStatus
    var toRemove []appv1alpha1.TopologyStatus

    // ... 三路差异计算
}

— internal/controller/application/applicationdeployment_controller.go:376-417
```

这意味着：

- 新增站点：在 Topology 列表中添加一项，Controller 自动创建该站点的 Deployment
- 移除站点：从 Topology 列表中删除一项，Controller 自动清理该站点的 Deployment
- 无需逐个站点操作：所有变更通过修改一个 `ApplicationDeployment` 资源一次性声明

7.2 部署状态聚合与追踪

每个 Topology 独立追踪

`ApplicationDeployment` 的 Status 中包含每个 Topology 的独立状态：

```
type TopologyStatus struct {
    TopologyIndex int32 // 拓扑索引
    Topology DeploymentTopology // 拓扑定义
    WorkloadName string // 创建的 K8s Deployment 名称
    Nodes []string // Pod 实际运行的节点列表
    Replicas int32 // 期望副本数
    ReadyReplicas int32 // 就绪副本数
    Phase DeploymentPhase // 当前状态
    Message string // 状态消息
}
```

每个 Topology 拥有独立的 Phase 状态：

Phase	含义
Pending	等待部署或部署中
Deploying	部署进行中
Running	所有副本就绪，正常运行
Failed	部署失败
Upgrading	版本升级中

全局状态聚合

Controller 从所有 Topology 的状态中聚合出全局视图：

```
func (r *ApplicationDeploymentReconciler) calculateOverallStatus(
    deployment *appv1alpha1.ApplicationDeployment,
) {
```

```

var readyCount int32
var totalReplicas int32
var readyReplicas int32

for _, topologyStatus := range deployment.Status.TopologyStatuses {
    if topologyStatus.Phase == appv1alpha1.DeploymentPhaseRunning {
        readyCount++
    }
    totalReplicas += topologyStatus.Replicas
    readyReplicas += topologyStatus.ReadyReplicas
}

deployment.Status.ReadyStatus = fmt.Sprintf("%d/%d", readyReplicas, totalReplicas)
// ...
}

```

— `internal/controller/application/applicationdeployment_controller.go:536-576`

运维人员通过 `kubectl get` 即可一眼看到全局部署状态：

```

$ kubectl get applicationdeployments -n manufacturing
NAME APPLICATION VERSION READY PHASE
quality-inspection-deploy quality-inspection 1.1.0 12/12 Running
energy-optimizer-deploy energy-optimizer 2.0.0 8/10 Pending

```

READY 列的 12/12 表示所有 12 个副本（分布在 4 个站点）全部就绪。8/10 表示还有 2 个副本未就绪——可能某个边缘站点正在拉取镜像或等待资源调度。

事件驱动的状态更新

Controller 不仅监听 `ApplicationDeployment` 的变化，还通过 Watch 机制监听底层 K8s Deployment 的状态变化：

```

return ctrl.NewControllerManagedBy(mgr).
    For(&appv1alpha1.ApplicationDeployment{}).
    Watches(
        &appsv1.Deployment{},
        &deploymentHandler{reconciler: r},
    ).
    Complete(r)

```

— `internal/controller/application/applicationdeployment_controller.go:69-80`

当某个边缘站点的 K8s Deployment 状态发生变化（Pod 重启、副本数变化等），事件会自动传播到对应的 `ApplicationDeployment`，触发状态更新。这确保了 `ApplicationDeployment` 的状态始终反映最新的真实情况。

7.3 版本管理与变更历史

版本切换

更新应用版本只需修改 `ApplicationDeployment` 的 `version` 字段：

```
spec:
  version: "1.1.0" # 从 "1.0.0" 更新到 "1.1.0"
```

Controller 检测到版本变更后，会自动为所有 Topology 执行滚动更新：

1. 获取新版本的 `ApplicationVersion` 配置
2. 对比新旧配置的差异
3. 更新每个 Topology 对应的 K8s Deployment
4. K8s 的滚动更新策略确保零停机

变更历史追踪

注：`DeploymentHistory` 字段类型定义已完成，写入逻辑为规划中功能，当前运行时此字段始终为空。以下为字段结构说明，不代表当前运行时行为。

每次版本变更都被记录在 `DeploymentHistory` 中：

```
type DeploymentHistory struct {
  Application string // 应用名称
  FromVersion string // 变更前版本
  ToVersion string // 变更后版本
  Reason HistoryReason // 变更原因 (Initial/Upgrade/Rollback)
  Timestamp metav1.Time // 变更时间
  Operator string // 操作人
  Message string // 变更说明
  WorkloadCount int32 // 变更时的工作负载数量
  TotalReplicas int32 // 变更时的总副本数
  ReadyReplicas int32 // 变更时的就绪副本数
```

```
    Topologies []DeploymentTopology // 变更时的拓扑快照  
}
```

变更原因（`HistoryReason`）区分三种场景：

- `Initial`：首次部署
- `Upgrade`：版本升级
- `Rollback`：版本回滚

每条历史记录不仅保存了版本信息，还保存了变更时刻的拓扑快照和副本状态，为事后审计和故障排查提供完整的上下文。

设计决策：历史记录最多保留 10 条（`History []DeploymentHistory`，注释标注 "最多保留 10 条"），避免 Status 无限膨胀。

7.4 Label 体系与资源关联

精确的 Label 关联

Controller 为每个创建的 K8s Deployment 设置了完整的 Label 体系：

Label	用途	示例值
app.theriseunion.io/application	应用名称	quality-inspection
app.theriseunion.io/version	应用版本	1.1.0
app.theriseunion.io/deployment	部署实例名称	quality-inspection-deploy
app.theriseunion.io/topology-hash	拓扑哈希	a3f8c2b1e9
app.theriseunion.io/topology-index	拓扑索引	0
app.theriseunion.io/workload	工作负载名称	api-server
theriseunion.io/nodegroup	节点组	beijing-factory

这套 Label 体系支持多维度的资源查询：

```
# 查看某个应用的所有 Deployment
kubectl get deploy -l app.theriseunion.io/application=quality-inspection

# 查看某个站点的所有 Deployment
kubectl get deploy -l theriseunion.io/nodegroup=beijing-factory

# 查看某个部署实例的所有 Deployment
kubectl get deploy -l app.theriseunion.io/deployment=quality-inspection-deploy
```

Annotation 补充信息

除 Label 外，Controller 使用 Annotation 存储非查询性的补充信息，如完整的 Topology Key：

```
annotations := map[string]string{
    constants.ApplicationTopologyLabel: topologyKey,
}
```

Label 使用 Hash 值（`topology-hash`）用于选择器匹配，Annotation 存储完整的拓扑标识用于调试和展示。

7.5 删除与清理

Finalizer 保护的删除流程

当 `ApplicationDeployment` 被删除时，Controller 通过 Finalizer 机制确保资源的正确清理：

```
删除 ApplicationDeployment
├── DeletionTimestamp 被设置
├── Controller 检测到 Finalizer 存在
│   ├── cleanupTopologies：删除所有关联的 K8s Deployment
│   │   ├── 遍历 getDeploymentsForApplication 的结果
│   │   └── 逐个删除（忽略 NotFound 错误）
│   └── 移除 Finalizer
└── K8s 垃圾回收完成删除
```

源码实证：清理逻辑对 `NotFound` 错误的容忍，确保了幂等性——即使某个 `Deployment` 已经被手动删除，清理过程不会因此中断：

```
`go
for _, deploy := range deployments {
    if err := r.Delete(ctx, &deploy); err != nil && !apierrors.IsNotFound(err) {
        return err
    }
}
`
```

— `internal/controller/application/applicationdeployment_controller.go:925-930`

7.6 端到端部署场景示例

场景：质检模型从试点到全国推广

第一阶段：试点部署

在一个工厂验证质检模型 v1.0.0：

```
spec:
  application: quality-inspection
  version: "1.0.0"
  topologies:
    - nodeGroup: "beijing-factory-a"
```

结果：1 个 Topology，1 个 K8s Deployment。ReadyStatus: "2/2"。

第二阶段：小范围推广

验证通过，扩展到 3 个工厂：

```
spec:
  topologies:
    - nodeGroup: "beijing-factory-a"
    - nodeGroup: "beijing-factory-b"
    - nodeGroup: "shanghai-factory"
```

Controller 计算差异：toAdd=2，toUpdate=0，toRemove=0。自动创建 2 个新的 K8s Deployment。

第三阶段：模型升级

v1.0.0 → v1.1.0，准确率从 85% 提升到 92%：

```
spec:
  version: "1.1.0" # 版本切换
  topologies: # 拓扑不变
    - nodeGroup: "beijing-factory-a"
    - nodeGroup: "beijing-factory-b"
    - nodeGroup: "shanghai-factory"
```

Controller 为 3 个 Topology 同时执行滚动更新。（注：变更历史记录为规划中功能，当前版本 `DeploymentHistory` 字段始终为空，`Reason: Upgrade, FromVersion: 1.0.0, ToVersion: 1.1.0` 为未来预期行为。）

第四阶段：全国推广

扩展到 50 个工厂站点：

```
spec:
  version: "1.1.0"
  topologies:
    - nodeGroup: "beijing-factory-a"
    - nodeGroup: "beijing-factory-b"
    # ... 47 个更多站点 ...
    - nodeGroup: "chengdu-factory"
    - nodeGroup: "__cloud__" # 云端也运行一个实例，用于演示和测试
```

Controller 差异计算后，自动创建 48 个新的 K8s Deployment。全局状态：`ReadyStatus: "100/100"`（50 个站点 x 2 副本）。

全程操作次数：4 次 YAML 修改。传统方式的操作次数：50 个站点 x 3 次操作（部署+升级+验证）= 150 次。

7.7 本章小结

Topology-Based 部署模型回答了"应用怎么到达每一个边缘"的问题：

能力	实现方式	效果
一处声明，多处部署	Topology 列表 + Controller 自动创建 K8s Deployment	50 个站点只需一个 YAML
云边统一	__cloud__ 标识 + nodeSelector 差异化	云端和边缘用同一套部署模型
状态全局可视	TopologyStatus 独立追踪 + 全局聚合	ReadyStatus: "100/100" 一眼看全局
版本批量切换	修改 version 字段 + Controller 滚动更新	所有站点同步升级
变更可追溯	DeploymentHistory 记录	谁在什么时间做了什么变更
删除安全	Finalizer + 清理逻辑	不残留孤儿 Deployment

这套部署模型的核心价值在于：将"应用推广到 N 个站点"的操作复杂度从 $O(N)$ 降低到 $O(1)$ 。无论是 5 个站点还是 500 个站点，运维人员的操作步骤都是相同的——修改一个 ApplicationDeployment 资源。

这是第二章提出的"应用推广速度"指标的直接回答：从分钟级别到分钟级别，不随站点数量增长。

下一章：第三章 — 基础设施就绪，应用运行的基石

第八章：应用运行 — 断网不断服务

应用面对的问题：边缘网络不稳定，断网了应用还能跑吗？应用需要操作传感器等 IoT 设备怎么办？

引言：边缘的常态是"不完美"

前面几章描述了应用从打包、审核、上架到部署的完整旅程。但在边缘场景下，应用部署成功只是起点——应用在运行期间面临的挑战远比云端复杂：

- 网络不可靠：边缘站点与云端的网络连接可能因为专线故障、移动网络波动、恶劣天气等原因中断。2023 年的行业调研显示，超过 60% 的边缘站点每月至少经历一次网络中断。
- 必须持续运行：工厂的质检模型不能因为网络断了就停止工作——每停止一分钟，未检测的瑕疵品就在流入市场。
- 需要操作物理设备：边缘应用不只是"容器里的服务"——很多应用需要采集传感器数据、控制执行器、与 PLC 通信。

本章介绍平台在应用运行阶段的两项核心能力：边缘离线自治（断网不断服务）和 IoT 设备交互（应用与物理世界的桥梁）。

8.1 边缘离线自治

问题：断网了应用还能跑吗

Kubernetes 的设计假设是"网络可靠"——API Server 是集群的控制中心，kubelet 需要持续与 API Server 通信来获取 Pod 的期望状态。如果网络断开，kubelet 无法确认 Pod 的最新配置，默认行为是逐渐驱逐 Pod。

这在边缘场景下是灾难性的。一次短暂的网络波动不应该导致应用停机。

三种离线方案适配不同场景

平台不是只提供一种离线方案，而是通过第三章介绍的可插拔运行时架构（ComponentInstaller）同时支持三种方案：

场景	特征	适合的方案	原理
嵌入式边缘	ARM 网关、树莓派、内存 < 1GB	KubeEdge	轻量级 EdgeCore 代理，本地保存 Pod 元数据
标准服务器边缘	工厂服务器、门店服务器	OpenYurt	YurtHub 本地缓存代理，透明拦截 API 请求
独立边缘站点	需要完整 K8s 控制面	VCluster（K3s/K0s）	本地运行完整的 K8s 控制面，完全独立自治

KubeEdge 方案：资源极度受限的嵌入式边缘

工作原理：KubeEdge 在边缘部署一个极轻量的 EdgeCore 代理（内存占用 < 70MB）。EdgeCore 缓存 Pod 的期望状态到本地 SQLite 数据库。当网络断开时，EdgeCore 使用本地缓

存维持 Pod 的运行状态。

适用场景：

- ARM 架构的边缘网关
- 资源极度受限的嵌入式设备
- 千级以上边缘节点的大规模部署

局限性：EdgeCore 只缓存 Pod 级别的元数据，断网期间无法处理 Service、Ingress 等高级网络策略的变更。

OpenYurt 方案：标准服务器级边缘

工作原理：OpenYurt 在每个边缘节点部署 YurtHub——一个本地 API 缓存代理。YurtHub 透明地拦截 kubelet 到 API Server 的所有请求，将响应缓存到本地。当网络断开时，YurtHub 直接从本地缓存返回数据，kubelet 感知不到网络中断。

自动离线自治启用：平台的 EdgeNode Controller 自动为边缘节点启用离线自治能力。当新的边缘节点加入集群时，Controller 检测其类型并自动添加自治标注：

源码实证：

```
`go

func (r *EdgeNodeReconciler) handleNodeAutonomy(ctx context.Context,
node *corev1.Node) (bool, error) {
// 1. 检查是否为边缘节点
nodeType, isEdgeNode := node.Labels[constants.NodeTypeLabelKey]
if !isEdgeNode || nodeType != constants.NodeTypeEdge {
return false, nil
}
>
// 2. 检查是否已添加自治标注
if _, exists := node.Annotations[constants.NodeAutonomyAnnotation]; exists {
return false, nil
}
>
// 3. 自动启用离线自治
node.Annotations[constants.NodeAutonomyAnnotation] =
constants.NodeAutonomyEnabled
```

```

r.Recorder.Eventf(node, corev1.EventTypeNormal, "AutonomyEnabled",
"Automatically enabled node autonomy for edge node")

return true, nil
}
、

— internal/controller/edgenode/edge_node_controller.go

```

相关常量定义：

```

// 兼容 OpenYurt 的边缘节点自治标注
NodeAutonomyAnnotation = "node.beta.openyurt.io/autonomy"
NodeAutonomyEnabled = "true"

// 边缘运行时类型标注
EdgeRuntimeAnnotation = "cluster.theriseunion.io/edge-runtime"
EdgeRuntimeOpenYurt = "openyurt"
EdgeRuntimeKubeEdge = "kubedge"

— pkg/constants/constants.go

```

关键设计：自治能力的启用是幂等的——Controller 只在标注不存在时添加，避免重复操作。同时，Controller 会为边缘节点自动添加 `node-role.kubernetes.io/edge:NoSchedule` 污点（Taint），防止非边缘工作负载被意外调度到边缘节点。

适用场景：

- 标准 x86/ARM64 服务器
- 工厂、门店、边缘数据中心
- 需要与云端保持弱连接的场景

优势：对应用完全透明——应用不需要做任何适配，YurtHub 在网络层自动处理了离线场景。

VCluster 方案：独立运行的边缘站点

工作原理：在边缘站点部署一个完整的 K8s 控制面（通过 VCluster 虚拟集群）。VCluster 运行在边缘的宿主集群内，但拥有完全独立的 API Server、etcd 和调度器。网络断开时，VCluster 作为独立集群继续运行，甚至可以处理新的 Pod 创建和调度。

适用场景：

- 偏远的独立站点（如海上平台、矿区）
- 网络长时间断开（数小时到数天）
- 需要在断网期间部署新应用

优势：最强的自治能力——断网期间几乎所有 K8s 操作都可以正常执行。

三种方案不是“选择困难”

需要强调的是：三种离线方案不是"选择困难"，而是为不同场景提供最合适的解法。

场景特征 → 推荐方案

资源极度受限（<1GB 内存）→ KubeEdge

标准服务器 + 弱网连接 → OpenYurt

完全独立 + 长时间断网 → VCluster

平台通过 ComponentInstaller 可插拔接口（第三章 3.3）同时支持三种方案，运维人员可以为不同的边缘站点选择最合适的方案。选择权在用户手中，而不是被平台绑定。

与分层镜像缓存的协同

离线自治保证了已运行的应用在断网时持续运行。但如果需要在断网期间启动新应用呢？

这就需要第三章介绍的分层镜像缓存（3.4）配合：边缘站点本地的 Repository 缓存了常用的应用镜像。即使云端网络断开，只要镜像在本地缓存中，新应用仍然可以快速启动。

断网场景下的应用保障

已运行的应用 → 离线自治保证持续运行

- └─ KubeEdge：本地 SQLite 缓存 Pod 元数据
- └─ OpenYurt：YurtHub 缓存 API 响应
- └─ VCluster：本地完整控制面

新应用启动 → 分层镜像缓存保证镜像可用

- └─ 边缘站点本地 Repository 有缓存 → 直接拉取启动
- └─ 本地无缓存 → 等待网络恢复

8.2 IoT 设备交互

问题：应用不只运行在容器里

在工业边缘场景中，应用通常需与物理世界的设备交互：

- 质检应用需从工业相机获取图像数据
- 环境监控应用需读取温湿度传感器
- 设备控制应用需向 PLC 发送指令
- 能耗优化应用需读取电表和水表数据

传统 IoT 平台使用私有 SDK 和私有协议来管理这些设备。但正如第二章 2.4 所述，这些能力已经被开源社区以云原生方式重新实现。

云原生化的 IoT 设备管理

平台将 IoT 设备管理完全 CRD 化——管理设备和管理 Pod 用同一套 Kubernetes API。

设备模型（DeviceProfile）：定义设备的属性和能力，类似于面向对象编程中的“类定义”。

```
DeviceProfile (设备模板)
├── 属性：温度、湿度、压力、电压...
├── 命令：启动、停止、重置...
└── 协议：Modbus、MQTT、OPC-UA...
```

设备实例（Device）：对应一个实际的物理设备，类似于“类的实例”。

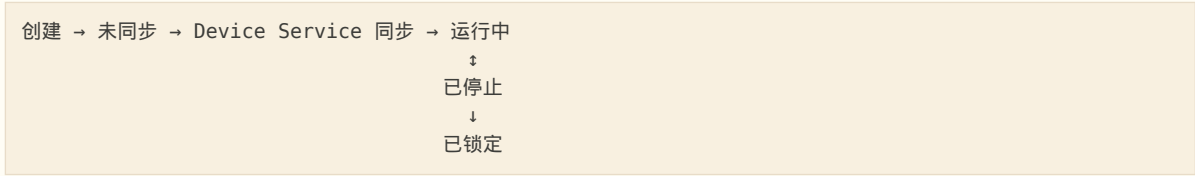
```
Device (设备实例)
├── profile: temperature-sensor-v1 ← 引用设备模板
├── service: modbus-device-service ← 关联设备服务
├── protocols: ← 通信配置
└── Modbus: {address: 1, port: 502}
```



```
└─ status:
    │   └─ operatingState: ENABLED
    └─ lastReported: 2024-01-15T10:30:00Z
```

设备状态机

每个设备有明确的生命周期状态：



状态	含义	可执行操作
未同步	设备已创建但 Device Service 尚未处理	等待同步
运行中（ENABLED）	设备正常运行，可以收发数据	读取/控制/停止
已停止（DISABLED）	设备已停止，不收发数据	启动/锁定
已锁定（LOCKED）	设备被锁定，禁止任何操作	解锁

与 EdgeX Foundry 集成

平台的 IoT 能力不是从零构建的，而是基于 EdgeX Foundry 这一成熟的开源边缘 IoT 框架。EdgeX Foundry 提供了：

- 多协议适配：Modbus、MQTT、OPC-UA、BACnet、REST 等
- 设备服务（Device Service）：负责与物理设备通信的边缘微服务
- 核心数据服务（Core Data）：设备数据的采集、存储和转发

平台的贡献在于：将 EdgeX Foundry 的设备管理能力从私有 API 提升为 Kubernetes CRD。



CRD 化的核心价值

IoT 设备管理 CRD 化带来了三个关键价值：

1. 统一 API：运维人员使用 `kubectl get devices` 查看设备列表，与 `kubectl get pods` 使用完全相同的命令体验。不需要学习另一套管理工具。
2. 统一权限：五层权限模型（第五章）自动覆盖设备资源。"工作空间 A 的用户只能操作工作空间 A 的设备"是权限系统的标准功能，不需要额外的权限配置。

3. 声明式管理：设备的期望状态通过 CRD 声明，Device Service 负责将期望状态同步到实际设备。设备异常时，Controller 可以自动尝试恢复——与 Kubernetes 管理 Pod 的逻辑完全一致。

与传统 IoT 平台的对比

能力	传统 IoT 平台	本平台
设备管理 API	厂商私有 REST API	Kubernetes 标准 CRD API
权限控制	独立的权限系统	复用五层 Scope 权限模型
设备发现	厂商私有协议	EdgeX Foundry 多协议适配
与应用集成	需要编写适配代码	同一个集群内，Pod 直接访问 Device Service
平台锁定	换平台需重写对接代码	基于标准 CRD，可替换实现

8.3 边缘节点管理

自动化的边缘节点生命周期

EdgeNode Controller 不仅处理离线自治的启用，还管理边缘节点的完整生命周期：

1. 节点类型识别

通过 Label `node.theriseunion.io/type: edge` 标识边缘节点。Controller 根据节点类型执行不同的管理策略。

2. 自动污点管理

边缘节点自动添加 `NoSchedule` 污点，确保只有显式声明容忍（Toleration）的工作负载才会调度到边缘。这防止了云端的管理 Pod 被意外调度到资源有限的边缘节点。

3. 节点统计

NodeGroup Controller 自动统计每个节点组的节点数量和就绪节点数量，存储在 Annotation 中：

```
annotations:
  theriseunion.io/node-count: "10" // 总节点数
  theriseunion.io/ready-node-count: "8" // 就绪节点数
```

这些统计信息供管理界面展示，帮助运维人员快速了解每个边缘站点的节点健康状况。

8.4 本章小结

应用运行回答了"应用在边缘如何持续稳定运行"的问题：

环节	机制	效果
离线自治	三种方案适配不同场景	嵌入式用 KubeEdge、服务器用 OpenYurt、独立站点用 VCluster
自动启用	EdgeNode Controller 自动添加自治标注	边缘节点加入即自治，无需手动配置
镜像保障	分层缓存（第三章 3.4）	弱网环境新应用仍可启动
IoT 交互	Device CRD + EdgeX Foundry	管理设备与管理 Pod 用同一套 API
统一权限	五层 Scope 覆盖设备资源	设备权限与应用权限统一管理
边缘管理	自动污点 + 节点统计	防止误调度，健康状况一目了然

这套应用运行保障体系的核心价值在于：让边缘的"不完美"对应用透明。网络断了，应用感知不到；需要读传感器数据，用标准 K8s API；节点需要特殊管理，Controller 自动处理。这是第二章提出的"断不了"承诺的技术实现——边缘离线自治确保应用持续运行，IoT CRD 化确保应用与物理设备的交互标准化。

下一章：第九章 — 应用可观测，看得见才管得住

第九章：应用可观测 — 看得见才管得住

应用面对的问题：怎么知道应用在每个边缘站点运行是否健康？出了问题怎么快速定位？GPU 利用率是不是太低？

引言：边缘的监控比云端更难

云端应用的监控有成熟的 SaaS 方案——Datadog、New Relic、Grafana Cloud。但边缘场景的监控面临三个独特挑战：

- 规模分散：100 个边缘站点分布在全国各地，每个站点有不同的应用组合和硬件配置。
- 网络受限：边缘站点到云端的带宽有限，不可能将所有原始指标实时传输到中心。
- 多租户隔离：AI 团队只应该看到自己应用的指标，运维团队需要看到基础设施指标，管理层需要看到全局概览。

传统的“每个站点部署一套 Prometheus + Grafana”方式可以解决单站点的监控问题，但无法提供跨站点的统一视图。而“所有指标汇聚到中心”的方式在边缘网络条件下又不现实。

本章介绍平台的三层监控架构——采集层预聚合、查询层多维度、展示层自适应——以及多租户指标隔离和算力设备监控能力。

9.1 三层监控架构

采集层：Prometheus + Recording Rules 预聚合

每个边缘集群运行独立的 Prometheus 实例，负责本地指标的采集和存储。指标来源包括：

采集器	指标维度	典型指标
node_exporter	节点资源	CPU/内存/磁盘/网络使用率
kube-state-metrics	K8s 对象状态	Pod 状态、Deployment 副本数、Service 端点
cadvisor	容器资源	容器级 CPU/内存/IO
GPU Exporter	算力设备	GPU 利用率、显存、温度、功耗

关键设计：Recording Rules 预聚合

平台在 Prometheus 层面配置了大量 Recording Rules（预计算规则），将高基数的原始指标预聚合为租户维度的汇总指标。例如：

```
原始指标: container_cpu_usage_seconds_total{namespace="ai-team-prod", pod="model-xyz"}
↓ Recording Rule 预聚合
汇总指标: workspace:container_cpu_usage_seconds:sum_rate{workspace="ai-team"}
```

源码实证：

```
`go

var promQLTemplatesEdgePlatform = map[string]string{

// Workspace 级别的 Recording Rule 引用

"workspace_cpu_usage":

workspace:container_cpu_usage_seconds:sum_rate,

"workspace_memory_usage":
```

```
workspace:container_memory_usage_bytes:sum,

"workspace_pod_count":

workspace:kube_pod_info:count,
>

// Namespace 级别的 Recording Rule 引用

"namespace_cpu_usage":

namespace:container_cpu_usage_seconds:sum_rate,

"namespace_memory_usage":

namespace:container_memory_usage_bytes:sum,
}
、

— monitoring-service/pkg/client/prometheus/promql_edge_platform.go
```

为什么 Recording Rules 是性能关键？

在管理 1000+ 集群、10000+ 节点的规模下，如果每次查询都需要实时聚合原始指标，查询延迟将从毫秒级退化到秒级甚至分钟级。Recording Rules 在数据写入时就完成聚合计算，查询时只需读取预计算结果——查询延迟从 $O(N)$ 降低到 $O(1)$ ， N 是原始时间序列数量。

查询层：monitoring-service + 并行查询

monitoring-service 是平台的监控查询中间层，提供 291+ 预定义指标的统一查询接口。

291+ 预定义指标覆盖 15 个监控维度：

维度	指标数量	目标读者
集群（Cluster）	49	平台管理员
节点（Node）	30	运维工程师
工作空间（Workspace）	26	工作空间管理员
命名空间（Namespace）	28	项目负责人
节点组（NodeGroup）	27	站点运维
工作负载（Workload）	6	应用开发者
Pod	14	应用开发者
容器（Container）	14	应用开发者
API Server	19	平台管理员
调度器（Scheduler）	11	平台管理员
Controller Manager	11	平台管理员
ETCD	14	平台管理员
Kubelet	24	运维工程师

维度	指标数量	目标读者
Kube Proxy	10	运维工程师
CoreDNS	8	运维工程师
合计	291	—

— [monitoring-service/MONITORING_METRICS_API_REFERENCE.md](#)

并行查询架构

monitoring-service 使用 goroutine 并行执行多个指标的查询，显著降低多指标查询的总延迟：

源码实证：

```
`go

func (p prometheus) QueryInstantCustomMetrics(
metrics []string, ts time.Time, o client.QueryOption,
) []client.Metric {
var wg sync.WaitGroup
res := make([]client.Metric, len(metrics))

>
for i, metric := range metrics {
wg.Add(1)
go func(i int, metric string) {
defer wg.Done()
expr := makeExpr(metric, *opts)
instantQueryFunc := func() (model.Value, v1.Warnings, error) {
return p.client.Query(context.Background(), expr, ts)
}
tempResult := p.executeQuery(instantQueryFunc,
parseQueryResp, params)
tempResult.MetricName = metric
res[i] = tempResult
}(i, metric)
}

>
wg.Wait()
```

```
return res
}
```

— `monitoring-service/pkg/client/prometheus/prometheus_client.go:148-184`

当前端仪表盘请求 20 个指标时，monitoring-service 启动 20 个并行查询，总延迟等于最慢的那个查询——而不是 20 个查询的总和。

自定义 PromQL 透传

除了 291 个预定义指标，monitoring-service 还支持自定义 PromQL 表达式的透传——高级用户可以直接编写 PromQL 查询，monitoring-service 将其转发到 Prometheus 并返回结果。

展示层：edge-console 自适应刷新

前端控制台（edge-console）提供 30+ 监控组件，采用自适应刷新策略：

场景	刷新间隔	原因
实时监控（最近 1 小时）	5 秒	需要及时发现异常
历史趋势（最近 24 小时）	5 分钟	数据变化较慢，节省查询开销
长期分析（最近 7 天）	15 分钟	关注趋势而非实时值

9.2 多租户指标隔离

三个扩展维度

在开源 Prometheus 的基础上，平台扩展了三个租户维度的 Recording Rules：

1. Workspace（工作空间）级别 — 26 个指标

工作空间管理员可以查看自己工作空间的资源总量和使用情况：

```
var workspaceMetrics = []string{
    // CPU 指标
    "workspace_cpu_usage",
    "workspace_cpu_utilisation",
    "workspace_cpu_requests_total",
    "workspace_cpu_limits_total",

    // 内存指标
    "workspace_memory_usage",
    "workspace_memory_rss",
    "workspace_memory_utilisation",
    "workspace_memory_requests_total",
    "workspace_memory_limits_total",

    // 工作负载统计
    "workspace_deployment_count",
    "workspace_statefulset_count",
    "workspace_daemonset_count",
    // ... 共 26 个指标
}
```

— [monitoring-service/pkg/api/monitoring/v1/named_metrics_edge.go](#)

2. Namespace（项目）级别 — 28 个指标

项目负责人可以查看自己项目的资源使用细节。

3. NodeGroup（节点组）级别 — 27 个指标

站点运维人员可以查看自己负责的边缘站点的节点和应用状况。

查询隔离机制

monitoring-service 通过 QueryOption 实现查询的租户隔离——每个查询携带 Scope 信息，只返回对应范围内的指标：

```
// 配置 Workspace 级别查询
func WithWorkspaceName(name string) QueryOption {
    return func(o *QueryOptions) {
        o.Level = LevelWorkspace
        o.WorkspaceName = name
    }
}

// 配置 NodeGroup 级别查询
func WithNodeGroupName(name string) QueryOption {
    return func(o *QueryOptions) {
        o.Level = LevelNodeGroup
        o.NodeGroupName = name
    }
}
```

— monitoring-service/pkg/client/query_options_edge.go

结合第五章的五层权限模型，一个 Workspace 用户调用监控 API 时，权限系统确保其只能查看自己 Workspace 的指标——即使 PromQL 表达式试图查询其他 Workspace 的数据，权限层也会拒绝。

9.3 算力与设备监控

GPU/NPU 设备指标

第四章介绍了平台的算力管理体系。算力监控是对算力管理的闭环补充——不仅要“管得了”，还要“看得见”。

设备监控覆盖以下维度：

指标类别	具体指标	价值
算力指标	core Request/Usage/Total、分配率/利用率	评估算力是否被充分利用
显存指标	memory Request/Usage/Total、分配率/利用率	判断显存是否成为瓶颈
环境指标	温度、功耗、风速、健康状态	预防硬件故障
效率指标	GPU 引擎活跃度	识别空闲或低效的 GPU
可视化	GPU 利用率排行 Top5、显卡类型分布	快速定位异常

设备健康检查

设备健康状态通过 Device Plugin 实时上报：

健康状态	含义	触发动作
Healthy	设备正常运行	参与正常调度
UnHealthy	设备异常（温度过高、驱动异常等）	触发告警 + 可能从调度池中移除

源码实证：

```
`go
const (
```

```
DeviceHealthy = "Healthy"

DeviceUnHealthy = "UnHealthy"

)

、
```

— pkg/constants/device.go:168-170

结合 GlobalConfig 的告警级别配置（第四章 4.4），设备异常时可以触发 critical/high/middle/low 不同级别的告警。

算力监控与算力管理的闭环

第四章：算力管理（分配侧）

DeviceModel → ComputeTemplate → ResourcePool → NodeConfig
"应用需要多少算力？分配给它。"

第九章：算力监控（使用侧）

GPU Exporter → Prometheus → monitoring-service → 仪表盘
"分配的算力用了多少？效率如何？"

闭环：

分配率高但利用率低 → ComputeTemplate 需要调整超卖比例
利用率高且温度异常 → 可能需要增加散热或减少负载
多张 GPU 利用率为零 → 资源被预留但未使用，需要重新分配

9.4 告警管理

四级告警体系

平台的告警按严重程度分为四个级别：

级别	含义	典型场景
Critical	需要立即处理	节点 NotReady、GPU 温度超阈值、磁盘满
High	需要尽快处理	CPU 利用率持续 > 90%、内存压力大
Middle	需要关注	Pod 重启次数异常、网络延迟增加
Low	信息提示	磁盘使用率 > 70%、证书即将过期

告警规则支持自定义——运维人员可以根据自身场景定义告警阈值和触发条件。

设备故障码告警

GPU/NPU 设备的特殊告警通过 GlobalConfig 配置的 `AlertLevel` 字段与监控系统对接：

```
type GlobalConfigSpec struct {
    // 设备故障码告警级别配置
    AlertLevel []v1alpha1.Severity `json:"alertLevel,omitempty"`
}
```

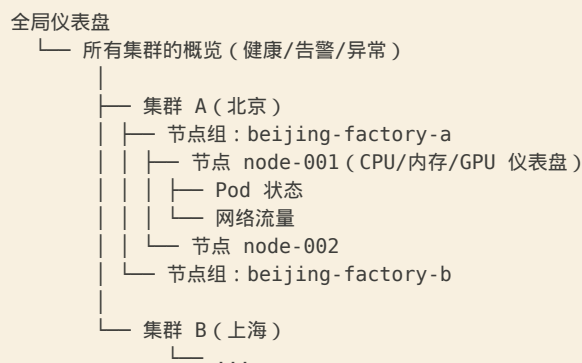
— `api/device/v1alpha1/globalconfig_types.go`

当 GPU 设备上报 ECC 错误、温度超限、功耗异常等故障码时，平台根据预设的告警级别触发对应的告警通知。

9.5 全局拓扑可视化

从全局到细节的钻取路径

平台提供从全局到单节点的逐层钻取视图：



每一层都提供对应粒度的监控指标：

- 全局层：集群健康状态概览、告警数量汇总
- 集群层：49 个集群级指标（CPU/内存总量、节点数、Pod 数）
- 节点组层：27 个节点组指标（站点级资源统计）
- 节点层：30 个节点指标（单节点详细资源使用）
- Pod/容器层：14+14 个指标（应用实例级监控）

节点级仪表盘

每个节点的仪表盘提供以下信息：

- 资源使用：CPU/内存/磁盘实时使用率和趋势

- **GPU 状态**：每张 GPU 的利用率、显存、温度
- **Pod 列表**：节点上运行的所有 Pod 及其资源消耗
- **网络流量**：入站/出站流量和错误包数

9.6 本章小结

应用可观测回答了"怎么知道应用在每个站点是否健康"的问题：

环节	机制	效果
指标采集	Prometheus + Recording Rules 预聚合	查询延迟毫秒级，不受原始数据量影响
统一查询	monitoring-service 291+ 预定义指标	15 个维度全覆盖
并行查询	goroutine 并行执行	20 个指标的查询时间 1 个指标
多租户隔离	Workspace/Namespace/NodeGroup Recording Rules	每个租户只看到自己的指标
算力监控	GPU 利用率、显存、温度、功耗	算力管理（分配侧）与监控（使用侧）闭环
告警管理	四级告警 + 设备故障码	从 Critical 到 Low 精确分级
拓扑可视化	全局 集群 节点组 节点 Pod	逐层钻取，快速定位问题

这套可观测体系的核心价值在于：让分散在全国各地的边缘应用"看得见"。运维人员不需要登录每个站点的监控系统，而是在一个统一的仪表盘上看到所有站点的应用状态。Recording Rules 预聚合策略确保了即使管理 1000+ 集群，查询性能也不会退化。

这是第二章提出的"应用运维成本"指标的直接回答：跨站点的应用监控和故障排查，从"逐站点查看"变成"统一视图一眼看全"。

下一章：第十章 — 平台架构设计

第十章：平台架构设计 —— 统一的技术底座

前面七章从应用的视角，依次讲述了基础设施就绪、算力就绪、组织安全、应用供应链、应用部署、应用运行和应用可观测。这一章给架构师看：平台是怎么把这些能力统一在一个一致的技术架构下的。

引言：一个平台，不是七个系统

前面的章节分别解决了应用生命周期中的不同问题：OTA 解决硬件接入，DeviceModel 解决算力抽象，IAMRole 解决权限控制，Provisioner 解决应用打包，Topology 解决应用分发，EdgeNode 解决离线运行，monitoring-service 解决可观测。

一个自然的问题是：这些能力是七个独立系统的简单拼凑，还是一个统一架构的有机组成？

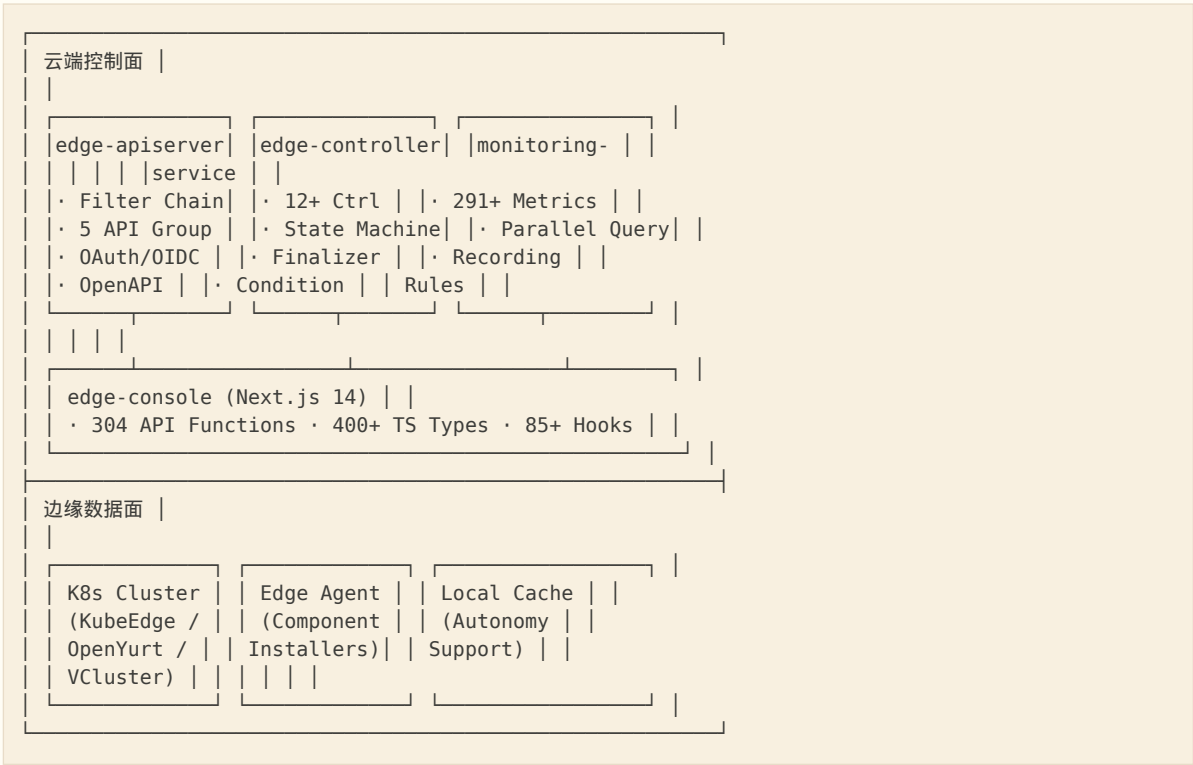
答案在源码中。打开 `edge-apiserver` 的入口文件，所有能力共享同一个 HTTP Filter Chain、同一套 CRD 资源模型、同一组 Controller Reconcile 模式。从一个 HTTP 请求进入平台，到它被路由到正确的集群、通过权限验证、触发 Controller 调谐、最终在边缘节点上产生效果——整个链路是一条完整的、可追踪的数据通路。

本章从四个维度展开平台的架构设计：请求处理链路（Filter Chain）、多集群请求路由、CRD 资源模型与 Controller 设计模式、以及 OpenAPI 驱动的前后端类型安全。

10.1 整体架构全景

四大组件协作

平台的运行时由四个核心组件构成，各自承担明确的职责：



组件	角色	关键能力
edge-apiserver	API 网关 + 认证授权 + 多集群路由	Filter Chain、OAuth2/OIDC、RequestInfo 解析、多集群代理
edge-controller	声明式调谐引擎	12+ Controller、Condition 状态机、Finalizer 资源保护

组件	角色	关键能力
monitoring-service	可观测查询层	291+ 预定义指标、Recording Rules 预聚合、并行查询
edge-console	前端交互层	OpenAPI 自动生成、类型安全 API 客户端、85+ 自定义 Hooks

四个组件通过两条纽带连接：**CRD 资源模型**（5 个 API 组、24+ CRD 类型）提供统一的数据契约，**Controller-Runtime** 提供统一的事件驱动调谐框架。任何一个前端操作——无论是创建集群、部署应用还是配置权限——最终都转化为 CRD 对象的 CRUD 操作，由对应的 Controller 完成实际的状态调谐。

10.2 七层 Filter Chain 架构

HTTP 请求的完整旅程

edge-apiserver 的所有 HTTP 请求都经过一条由七层 Filter 组成的处理链。这条链路借鉴了 Kubernetes API Server 的 Filter Chain 模式，扩展了多集群路由、第三方服务代理和多租户权限能力。

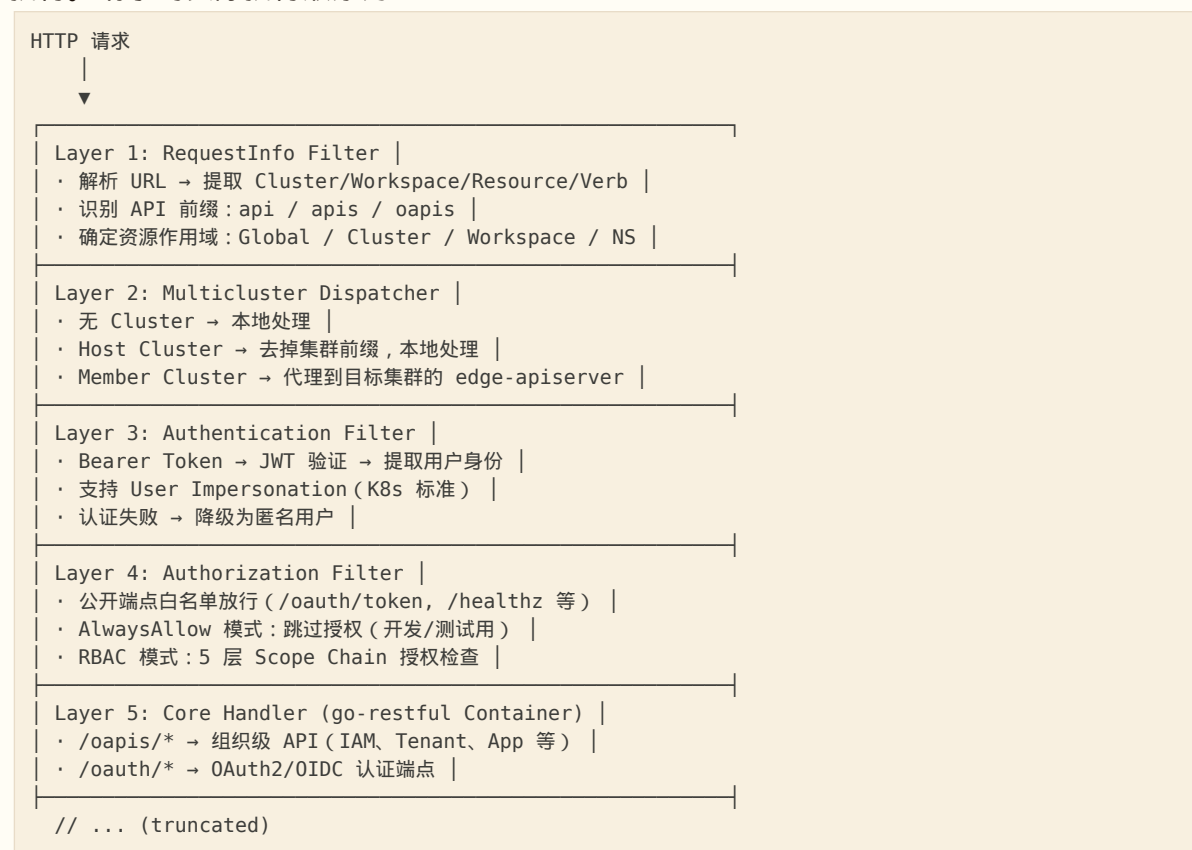
源码实证：Filter Chain 的构建过程在 `buildHandlerChain` 函数中完整展现：

```
`go
func (s *APIServer) buildHandlerChain(handler http.Handler) http.Handler {
    // Build filter chain (from inner to outer):
    // 1. Core handler (restful container for /oapis/* organization APIs)
    // 2. ReverseProxy filter (for third-party service proxy and custom proxy rules)
    // 3. KubeAPIServer proxy (for standard /api/ and /apis/ Kubernetes APIs)
    // 4. Authorization filter (our UniversalAuthorizer)
    // 5. Authentication filter (X-Remote-User etc.)
    // 6. Multicluster filter
    // 7. RequestInfo filter (request parsing and context injection)
    >
    finalHandler := handler
    finalHandler = filters.WithReverseProxy(finalHandler, s.RuntimeCache)
```

```
finalHandler = filters.WithKubeAPIServer(finalHandler, s.KubeConfig)
finalHandler = filters.WithAuthorization(finalHandler, s.Authorizer, s.AuthorizationMode)
finalHandler = filters.WithAuthentication(finalHandler, s.Authenticator)
finalHandler = filters.WithMulticluster(finalHandler, s.ClusterClient)
finalHandler = filters.WithRequestInfo(finalHandler, s.RequestInfoResolver)
return finalHandler
}
```

— pkg/apiserver/apiserver.go:456-496

由于 Go 的 HTTP middleware 采用"从外到内包裹"的构造方式，代码中最后添加的 Filter 最先执行。请求的实际执行顺序为：



每一层完全解耦

Filter Chain 的关键设计原则是：每一层只做一件事，通过 `http.Handler` 接口串联。每个 Filter 都是一个独立的 `struct`，实现 `ServeHTTP` 方法，持有 `next http.Handler` 引用。这意味着：

- 可独立测试：每个 Filter 可以用 `httptest.NewRecorder()` 独立验证，不依赖其他 Filter。
- 可独立禁用：认证、授权、多集群路由都可以通过传入 `nil` 独立跳过。

- 可独立替换：任何一层的实现都可以替换为另一种策略，不影响其他层。

源码实证：每个 Filter 都检查依赖是否为 nil，为 nil 时直接退化为透传：

```
`go
func WithMulticlient(next http.Handler, clusterClient clusterclientutil.Interface) http.Handler
{
    if clusterClient == nil {
        klog.V(4).Infof("Multiclient dispatcher is disabled")
        return next // 直接返回下一层，跳过本层
    }
    return &multiclientDispatcher{next: next, Interface: clusterClient}
}
`
```

— pkg/apiserver/filters/multiclient.go:42-53

这种设计让平台在不同部署场景下可以灵活裁剪：单集群部署可以禁用多集群路由，开发环境可以启用 AlwaysAllow 跳过授权，边缘独立集群可以精简到只保留核心 Handler。

10.3 RequestInfo：请求的语义化解析

从 URL 到结构化语义

Filter Chain 的第一层是 `RequestInfoResolver`，它的职责是将一个原始的 HTTP 请求解析为结构化的语义信息。这一层决定了后续所有 Filter 的决策基础。

源码实证：`RequestInfo` 的结构定义展现了平台扩展 K8s 标准请求模型的方式：

```
`go
type RequestInfo struct {
    Path string // URL 路径
    Verb string // K8s 语义动词：create/get/list/update/delete/watch/patch
    APIPrefix string // api / apis / oapis
    APIGroup string // 如 app.theriseunion.io
    APIVersion string // 如 v1alpha1
    Namespace string // 命名空间
    Resource string // 资源类型
    Name string // 资源名称
    IsResourceRequest bool // 是否为资源请求
    IsKubernetesRequest bool // 是否应由 K8s API Server 处理
}

// ===== 以下为平台扩展字段 =====
```



```
currentParts = currentParts[2:]
```

```
}
```

```
}
```

```
,
```

— [pkg/apiserver/request/requestinfo.go:207-216](#)

资源作用域的自动推断

解析完成后，`resolveResourceScope` 根据请求中携带的信息自动推断资源的作用域层级：

```
func (r *RequestInfoResolver) resolveResourceScope(request RequestInfo) string {
    if r.isGlobalScopeResource(request.APIGroup, request.Resource) {
        if request.Workspace != "" {
            return WorkspaceScope
        }
        return GlobalScope
    }
    if request.Namespace != "" { return NamespaceScope }
    if request.NodeGroup != "" { return NodeGroupScope }
    if request.Workspace != "" { return WorkspaceScope }
    return ClusterScope
}
```

— [pkg/apiserver/request/requestinfo.go:370-393](#)

这个五级作用域（Global → Cluster → Workspace → NodeGroup → Namespace）与第五章介绍的六层权限模型完全对齐，为授权 Filter 提供了精确的决策依据。

ScopePattern：动态 URL 模式匹配

除了内置的 URL 解析规则，平台还支持通过 `ScopePattern` CRD 动态注册新的 URL 模式。

`RequestInfoResolver` 通过 `controller-runtime` 的共享缓存监听 `ScopePattern` 资源的变化，实时更新匹配规则——无需重启 API Server 即可扩展新的作用域提取规则。

10.4 多集群请求路由

路由决策树

当 `RequestInfo` 中包含 `Cluster` 字段时，Multicluster Dispatcher 接管请求的路由决策。路由逻辑遵循一棵清晰的决策树：

```
info.Cluster == "" ?
├── 是 → 本地处理（单集群模式）
└── 否 → 获取 Cluster 对象
    ├── Cluster 不存在 → 400 Bad Request
    ├── IsHostCluster → 去掉集群前缀，本地处理
    ├── !IsReady → 503 Service Unavailable
    └── IsReady → 代理到目标集群
        ├── Direct + 无 APIEndpoint → K8s Service Proxy
        └── Direct + 有 APIEndpoint → 直连 edge-apiserver
```

源码实证：完整的路由决策在 `multiclusterDispatcher.ServeHTTP` 中实现：

```
`go
func (m multiclusterDispatcher) ServeHTTP(w http.ResponseWriter, req http.Request) {
    info, ok := request.RequestInfoFrom(req.Context())
    if !ok { ... }
}

// 无集群 → 本地处理
if info.Cluster == "" {
    m.next.ServeHTTP(w, req)
```

```
    return
  }
>
  // 获取集群对象
  cluster, err := m.GetCluster(info.Cluster)
  if err != nil { ... }
>
  // Host 集群 → 去掉前缀，本地处理
  if clusterutil.IsHostCluster(cluster) {
    req.URL.Path = strings.Replace(req.URL.Path,
    fmt.Sprintf("/clusters/%s", info.Cluster), "", 1)
    m.next.ServeHTTP(w, req)
    return
  }
>
  // 未就绪 → 503
  if !clusterutil.IsReadyCluster(cluster) {
    responsewriters.WriteRawJSON(http.StatusServiceUnavailable, ...)
    return
  }
>
  // 就绪 → 代理
  client, err := m.GetClusterClient(cluster.Name)
  ...
  }
  、

— pkg/apiserver/filters/multiclustergo:59-157
```

两种连接模式

平台支持两种到达成员集群的网络路径，适配不同的网络拓扑：

连接模式	适用场景	路由方式	认证传递
Direct (K8s Service Proxy)	中心与边缘在同一网络	通过 K8s Service 代理到目标集群的 edge-apiserver	Authorization 转为 X-Edge-Authorization
Direct (API Endpoint)	边缘有独立的公网地址	直连目标集群的 edge-apiserver URL	标准 HTTP 转发

在 K8s Service Proxy 模式下，请求通过 Kubernetes 的 Service 代理机制到达目标集群。代理路径的格式为：

```
/api/v1/namespaces/{ns}/services/{service}:{port}/proxy{path}
```

这种方式利用了 K8s 已有的网络基础设施，无需额外的网络组件。同时，为了避免与 K8s API Server 自身的 Bearer Token 认证冲突，平台在代理请求时将 **Authorization** 头转换为 **X-Edge-Authorization**——目标集群的 edge-apiserver 会识别这个自定义头并还原认证信息。

UpgradeAwareHandler：协议升级支持

多集群代理不仅需要处理普通的 HTTP 请求，还需要支持 WebSocket 连接（如 Pod 终端）和 SPDY 流式传输（如 `kubectl exec`）。平台使用 Kubernetes 的 **UpgradeAwareHandler** 实现协议透明升级：

```
upgrade := httpstream.IsUpgradeRequest(req)
httpProxy := proxy.NewUpgradeAwareHandler(location, transport, true, upgrade, &responder{})
httpProxy.UpgradeTransport = proxy.NewUpgradeRequestRoundTripper(transport, transport)
httpProxy.ServeHTTP(w, newReq)
```

这意味着运维人员可以在中心控制面通过浏览器终端直接操作边缘集群中的 Pod，WebSocket 连接会被透明地代理到正确的边缘节点——与操作本地集群的 Pod 体验完全一致。

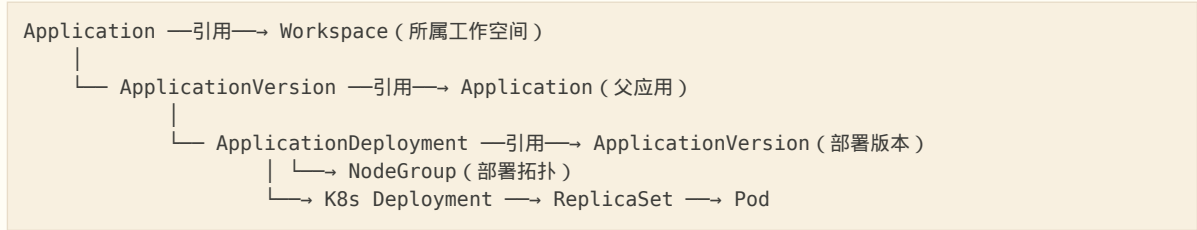
10.5 CRD 资源模型

五组 API，一个哲学

平台的所有资源都通过 CRD（Custom Resource Definition）定义，遵循 Kubernetes 的声明式 API 规范。5 个 API 组覆盖了平台的全部能力域：

API 组	域名	CRD 数量	核心资源	设计哲学
Scope	scope.theriseunion.io	3	Cluster, Workspace, NodeGroup	层级化资源边界，Finalizer 保护删除
IAM	iam.theriseunion.io	7+	User, IAMRole, IAMRoleBinding, RoleTemplate, Category, ScopePattern, LoginRecord	K8s RBAC 的多层级扩展
App	app.theriseunion.io	3	Application, ApplicationVersion, ApplicationDeployment	类 PVC 声明式 + 可插拔 Provisioner
Device	device.theriseunion.io	6	DeviceModel, ComputeTemplate, ResourcePool, ResourcePoolItem, NodeConfig, GlobalConfig	四层硬件抽象，配置与定义分离
Registry	registry.theriseunion.io	4	Registry, Repository, ClusterRegistry, ClusterRepository	分级镜像管理，Cluster 级与 Namespace 级

这些 CRD 不是各自独立的“配置表”，而是通过引用关系形成了完整的资源图谱。以应用部署为例：



每一层引用都有对应的保护机制：Finalizer 阻止有活跃子资源的父资源被删除，OwnerReference 保证父资源删除时级联清理子资源。

统一的 CRD 结构规范

所有 CRD 都遵循相同的结构约定：

```
type SomeResource struct {
    metav1.TypeMeta `json:",inline"`
    metav1.ObjectMeta `json:"metadata,omitempty"`
    Spec SomeResourceSpec `json:"spec,omitempty"`
    Status SomeResourceStatus `json:"status,omitempty"`
}
```

Spec（期望状态）由用户声明，Controller 不修改。**Status**（实际状态）由 Controller 维护，用户不修改。这种分离确保了声明式 API 的基本契约：用户描述"我要什么"，系统负责"怎么实现"。

元数据的管理遵循两条规则：

1. **Labels** 用于机器读取：Provisioner 类型、作用域标识、厂商分类等通过 Labels 管理，支持高效的列表查询和选择器过滤。
2. **Annotations** 用于人类阅读：显示名称（displayName）、描述（description）、多语言信息等通过 Annotations 存储，不影响 API 行为。

10.6 Controller 设计模式

八个核心架构模式

平台的 12+ Controller 共享一组经过生产验证的设计模式。这些模式不是理论上的"最佳实践"，而是从实际的并发冲突、资源泄漏、状态不一致等问题中沉淀出来的工程约束。

模式一：幂等 **Reconcile**

每个 Controller 的 **Reconcile** 函数都是幂等的——无论被调用一次还是一百次，最终效果相同。这是 Controller-Runtime 框架的基本要求，但实现起来需要纪律。

核心原则：**Reconcile** 不依赖"上一次的状态"，而是每次都从当前实际状态推导应该做什么。

```
func (r *WorkspaceReconciler) Reconcile(ctx context.Context, req ctrl.Request) (ctrl.Result, error) {
    workspace := &scopev1alpha1.Workspace{}
    if err := r.Get(ctx, req.NamespacedName, workspace); err != nil {
        if errors.IsNotFound(err) {
            return reconcile.Result{}, nil // 资源已删除，无事可做
        }
        return reconcile.Result{}, err
    }

    // 每次都检查 Finalizer 是否存在，而不是假设"上次已经加了"
    if !controllerutil.ContainsFinalizer(workspace, scopev1alpha1.WorkspaceFinalizer) {
        controllerutil.AddFinalizer(workspace, scopev1alpha1.WorkspaceFinalizer)
        if err := r.Update(ctx, workspace); err != nil {
            return reconcile.Result{}, err
        }
        return reconcile.Result{Requeue: true}, nil
    }

    // 每次都初始化默认角色，InitializeDefaultRoles 内部判断是否已存在
    if err := roletemplate.InitializeDefaultRoles(ctx, r.Client, workspace,
```

```

constants.ScopeTypeWorkspace); err != nil {
    return reconcile.Result{RequeueAfter: time.Minute}, err
}
...
}

```

模式二：Condition 状态机

Controller 通过 **Conditions** 数组跟踪资源的每个状态维度，而不是用单一的 **Phase** 字段表示整体状态。每个 Condition 独立跟踪，外部系统可以精确读取每个维度的状态。

源码实证：Cluster Controller 的 Condition 管理函数展示了这一模式：

```

`go

func (r *ClusterReconciler) setClusterCondition(
    cluster *scopev1alpha1.Cluster,
    conditionType scopev1alpha1.ClusterConditionType,
    status metav1.ConditionStatus,
    reason, message string) {
>
    var condition *scopev1alpha1.ClusterCondition
    for i := range cluster.Status.Conditions {
        if cluster.Status.Conditions[i].Type == conditionType {
            condition = &cluster.Status.Conditions[i]
            break
        }
    }
>
    if condition == nil {
        cluster.Status.Conditions = append(cluster.Status.Conditions, scopev1alpha1.ClusterCondition{
            Type: conditionType,
        })
        condition = &cluster.Status.Conditions[len(cluster.Status.Conditions)-1]
    }
>
    now := metav1.Now()

```

```

if condition.Status != string(status) {
    condition.LastTransitionTime = now
}

condition.Status = string(status)

condition.Reason = reason

condition.Message = message
}
、

```

— [internal/controller/cluster/cluster_utils.go:46-78](#)

以 Cluster 为例，其状态包含多个独立的 Condition（如 VCluster 安装状态、Kubeconfig 验证状态、系统核心组件状态等），每个 Condition 独立变化、独立恢复，不会因为某一个维度的异常导致整个资源被标记为“失败”。

模式三：Finalizer 保护

所有管理子资源的 CRD 都使用 Finalizer 保护删除流程。Finalizer 的作用是：在资源被标记为删除后，阻止 Kubernetes 立即从 etcd 中移除它，给 Controller 一个清理窗口。

删除流程始终遵循相同的模板：

```

创建阶段：添加 Finalizer → 正常处理
|
删除阶段：DeletionTimestamp 非零
|
├── 执行清理逻辑（undoReconcile / handleDeletion）
│   ├── 删除子资源（ApplicationVersion → K8s Deployment）
│   ├── 卸载组件（Cluster → Helm Release）
│   └── 清理关联资源（Workspace → 默认角色）
└── 移除 Finalizer → K8s 从 etcd 中删除资源

```

源码实证：Cluster Controller 的删除流程：

```

`go

if !instance.ObjectMeta.DeletionTimestamp.IsZero() {
    if sliceutil.HasString(instance.ObjectMeta.Finalizers, finalizer) {
        if _, err := r.undoReconcile(ctx, instance); err != nil {
            logger.Error(err, "undoReconcile failed")
        }
        // 清理完成后移除 Finalizer
    }
}

```

```

if err := r.UpdateCluster(rootCtx, req.NamespacedName, func(_instance
*scopev1alpha1.Cluster) error {
    _instance.ObjectMeta.Finalizers = sliceutil.RemoveString(
        _instance.ObjectMeta.Finalizers,
        func(item string) bool { return item == finalizer },
    )
    return nil
}); err != nil {
    return ctrl.Result{}, err
}
}

return ctrl.Result{}, nil
}
、

```

— [internal/controller/cluster/cluster_controller.go:110-129](#)

模式四：Patch 优先（避免并发冲突）

所有对现有资源的修改都优先使用 **Patch** 而非 **Update**。**Update** 需要匹配 **resourceVersion**，在 Controller 并发修改同一资源时容易产生 409 Conflict。**Patch** 是增量修改，不需要 **resourceVersion**，天然适合并发场景。

源码实证：DeviceModel Controller 使用 **MergeFrom** 生成 Patch：

```

`go
original := deviceModel.DeepCopy()
labelsChanged := r.updateLabels(deviceModel)
if labelsChanged {
    if err := r.Patch(ctx, deviceModel, client.MergeFrom(original)); err != nil {
        logger.Error(err, "failed to patch DeviceModel labels")
    }
    return ctrl.Result{}, err
}
}
、

```

— `internal/controller/device/devicemodel_controller.go:75-91`

模式五：Label 路由

资源的分类和路由不通过 Spec 字段硬编码，而是通过 Labels 实现声明式的服务发现。

典型场景：Application 的 Provisioner 类型通过 Label `app.theriseunion.io/provisioner` 指定，ApplicationVersion Controller 在 Reconcile 时从关联的 Application 读取这个 Label，决定使用哪个 Provisioner 处理部署。这种设计让新增 Provisioner 类型只需要注册一个新的 Label 值，不需要修改任何 CRD Schema。

模式六：并发控制

在多集群场景下，同一个 Controller 可能同时处理数十个集群的 Reconcile 事件。Cluster Controller 使用自定义的 `InstanceLock` 实现每资源独立锁定：

```
// 每个 Cluster 独立锁，不互相阻塞
if !r.InstanceLock.TryLock(req.Name) {
    return ctrl.Result{RequeueAfter: 5 * time.Second}, nil
}
defer r.InstanceLock.Unlock(req.Name)
```

配合 `MaxConcurrentReconciles: 3` 的设置，平台在保证并发效率的同时，避免了同一集群被多个 Goroutine 同时调谐导致的状态竞争。

模式七：事件记录

所有显著的状态变化都通过 `EventRecorder` 记录为 K8s Event，提供完整的审计跟踪：

```
r.Recorder.Event(iamRole, "Normal", "Created",
    fmt.Sprintf("Created ClusterRole: %s", clusterRole.Name))
r.Recorder.Eventf(deviceModel, corev1.EventTypeWarning, "DeviceManagerNotInitialized",
    "DefaultDeviceManager is not initialized, will retry after %v", waitTime)
```

Event 记录遵循两条规则：正常操作使用 `EventTypeNormal`，需要关注的异常使用 `EventTypeWarning`。这让运维人员可以通过 `kubectl get events` 快速定位问题。

模式八：定期重调谐

状态同步不仅依赖事件触发，还通过定期重调谐（Periodic Requeue）保证最终一致性：

```
return reconcile.Result{RequeueAfter: time.Minute * 5}, nil
```

即使没有外部事件触发，Controller 也会定期检查资源状态，自动修复任何偏移。这种“自愈”能力对边缘场景尤为重要——边缘节点的网络恢复后，Controller 能够在下一个调谐周期自动完成状态同步，无需人工干预。

10.7 API 注册与 Installer 模式

可插拔的 API 组注册

平台的 API 端点不是在一个巨大的路由表中硬编码，而是通过 `APIInstaller` 接口实现可插拔注册：

源码实证：

```
`go
type APIInstaller interface {
    Install(c *restful.Container) error // 安装 API 到容器
    GetGroupVersion() schema.GroupVersion // 获取 API 的 GroupVersion
    GetAPIPath() string // 获取 API 路径
}
`

— pkg/apiserver/runtime/runtime.go:92-102
```

每个 API 组实现 `APIInstaller` 接口，在 `Install` 方法中注册自己的路由。API Server 启动时统一调用所有 Installer：

```
func (s *APIServer) installAPIGroups() error {
    handlers := []loapis.Handler{
        configv1alpha1.NewHandler(s.RuntimeClient),
        iamapiv1alpha1.NewHandlerWithAuthorizer(s.RuntimeClient, s.Authorizer,
s.RequestInfoResolver),
        tenantv1.NewHandler(s.RuntimeClient),
        appapiv1alpha1.NewHandlerWithAuthorizer(s.RuntimeClient, s.Authorizer,
s.RequestInfoResolver),
        devicev1alpha1.NewHandler(s.RuntimeClient, s.ServerConfig, s.DeviceManager,
```

```
s.FaultManager),
    registryv1alpha1.NewHandler(s.RuntimeClient, s.ServerConfig),
    ...
}

for _, handler := range handlers {
    if err := handler.Install(s.container); err != nil {
        return fmt.Errorf("failed to install handler: %w", err)
    }
}
...
}
```

这种设计的好处是：新增一个 **API** 组只需要实现三个方法，注册到 **handlers** 列表中，不需要修改 **API Server** 的核心逻辑。每个 API 组的路由定义、参数验证、响应格式都封装在自己的 Handler 中，职责边界清晰。

WebService 工厂

每个 API 组使用统一的 **NewWebService** 工厂函数创建 **restful.WebService**，自动生成符合规范的 API 路径：

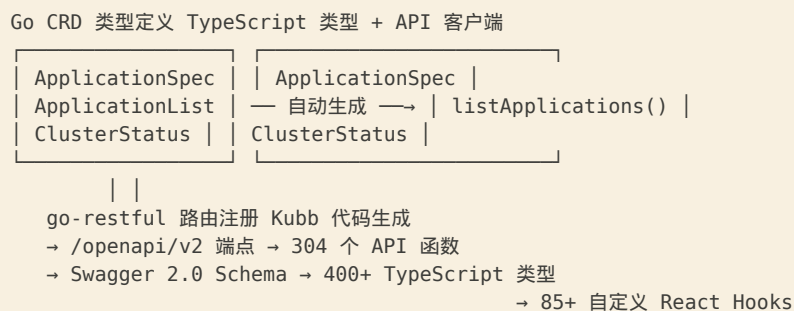
```
func NewWebService(gv schema.GroupVersion) *restful.WebService {
    webservice := restful.WebService{}
    path := ApiRootPath + "/" + gv.String() // /oapis/{group}/{version}
    webservice.Path(path).
        Produces(restful.MIME_JSON).
        Consumes(restful.MIME_JSON)
    return &webservice
}
```

这确保了所有 API 组的路径格式一致，与 **RequestInfoResolver** 的解析规则对齐。

10.8 OpenAPI 驱动的全链路类型安全

从 CRD 到前端组件的类型传导

传统前后端协作中，API 接口的定义通常靠文档同步，类型一致性靠人工保证。平台通过 OpenAPI 建立了从后端 CRD 到前端组件的自动类型传导链路：



具体的工作流程：

1. 后端：CRD 的 Go 类型通过 `go-restful-openapi` 自动生成 OpenAPI Schema，暴露在 `/openapi/v2` 端点。路由注册时使用 `Reads()` 和 `Returns()` 声明请求和响应类型，Schema 自动包含所有字段定义。
2. 代码生成：前端通过 `pnpm codegen` 命令，使用 Kubbb 代码生成器读取 `openapi.json`，自动生成类型安全的 API 客户端函数和 TypeScript 类型定义。
3. 前端：开发者使用生成的 API 函数和类型，IDE 提供完整的类型提示和编译时检查。

源码实证：路由注册时同步声明请求/响应类型，OpenAPI Schema 自动生成：

```
`go  
  
ws.Route(ws.GET("/clusters").
```

```
To(h.ListClusters).
Doc("List all clusters with pagination and sorting").
Returns(http.StatusOK, "OK", scopev1alpha1.ClusterList{}).
Param(ws.QueryParameter("limit", "Maximum number of items").DataType("integer")).
Param(ws.QueryParameter("page", "Page number").DataType("integer")))
>
ws.Route(ws.POST("/clusters").
To(h.CreateCluster).
Reads(scopev1alpha1.Cluster{}).
Returns(http.StatusCreated, "Created", scopev1alpha1.Cluster{}))
、
— pkg/oapis/tenant/v1/register.go:25-55
```

零手写 API 调用代码

这条链路的最终效果是：前端没有一行手写的 API 调用代码。所有 API 请求都通过生成的类型安全函数发起，后端接口变更后执行 `pnpm codegen` 一键同步。如果后端修改了某个字段的类型或删除了某个端点，前端在编译时就会报错——不是在运行时，更不是在用户点击后。

前端技术栈与生成代码的集成：

技术组件	版本	角色
Next.js	14	SSR/SSG + 路由 + 中间件
React	18	UI 组件渲染
TypeScript	5.9	静态类型检查
Kubb	—	OpenAPI TS 代码生成
TanStack Query	—	数据获取 + 缓存 + 状态管理
Monaco Editor	—	YAML/JSON 在线编辑（应用配置）
Xterm.js	—	Pod 终端（WebSocket）
@xyflow/react	—	集群拓扑可视化

10.9 本章小结

本章从架构师的视角，展示了平台的技术底座如何将前面七章的能力统一在一个一致的架构下：

- **Filter Chain** 是平台的神经中枢——每个请求经过语义解析、多集群路由、认证和授权四道关卡，确保到达正确的处理端点时已经携带了完整的上下文信息。
- **CRD 资源模型** 是平台的骨架——5 个 API 组、24+ CRD 类型构成了统一的数据契约，所有组件通过 CRD 对话，不通过私有 RPC。
- **Controller 设计模式** 是平台的肌肉——幂等 Reconcile、Condition 状态机、Finalizer 保护、Patch 优先等八个模式，确保声明式 API 的"自动驾驶"能力。
- **OpenAPI 类型链路** 是平台的感知神经——从后端 CRD 到前端组件的类型传导链路，消除了前后端协作中最常见的"接口不一致"问题。

回到第二章的核心论点：平台的使命是让应用在每个边缘节点上更快地产生业务价值。本章展示的架构设计解释了"为什么这个平台能做到"——不是因为功能列表更长，而是因为所有能力都建立在同一个声明式、事件驱动、类型安全的技术底座上。新增一种应用类型只需要实现一个 Provisioner；新增一个边缘站点只需要创建一个 Cluster CRD；新增一个 API 只需要实现一个 Installer。统一的架构让扩展的边际成本趋近于零。

第十一章：竞品对比与优势分析

本章的对比视角不是功能清单，而是应用价值。竞品讲的故事是"支持什么硬件、管理多少节点"；我们讲的故事是"应用如何在边缘持续释放价值"。

引言：对比的标尺

边缘计算平台市场上不缺产品——华为 IEF、阿里 ACK@Edge、腾讯 TKE Edge、Rancher/SUSE 都是活跃的玩家。但如果用传统的"功能打勾"方式对比（支持 x86？支持 ARM？支持离线？），结论往往是"大家都差不多"。

这是因为功能清单只回答了"有没有"，没有回答"好不好用"。

本章的对比标尺是第二章建立的核心命题：应用在边缘的每个生命周期阶段，平台提供了什么解法？我们不比"支持多少种 GPU"，而比"应用的算力需求如何被透明满足"；不比"管理多少节点"，而比"一千台边缘服务器纳管需要几步操作"；不比"有没有权限"，而比"五个团队共享一个平台时能否做到互不干扰"。

11.1 竞品全景

主要竞品定位

竞品	厂商	核心定位	技术基座
IEF (Intelligent EdgeFabric)	华为云	云边协同的边缘计算服务	KubeEdge + 华为云生态
ACK@Edge	阿里云	云端托管的边缘 K8s 集群	OpenYurt + 阿里云容器服务
TKE Edge	腾讯云	边缘容器服务	SuperEdge + 腾讯云 TKE
Rancher	SUSE	多集群 Kubernetes 管理	K3s + Rancher Server

关键差异：产品哲学

在进入技术对比之前，需要先明确四家竞品与本平台之间最根本的差异——产品哲学。

维度	云厂商边缘服务 (IEF/ACK@Edge/TKE Edge)	开源方案 (Rancher)	本平台
核心抽象	集群 / 节点 / 工作负载	集群 / 项目 / 工作负载	应用 / 版本 / 部署
组织方式	按基础设施资源分类	按集群分组	按应用生命周期阶段组织
平台边界	云服务的延伸，依赖云厂商生态	开源社区驱动，集成需自行完成	独立部署，全链路自治
收费模式	按节点/集群/流量计费	开源免费 + 企业版付费	私有化部署，按授权计费
GPU 视角	"支持哪些硬件"	不涉及 GPU 管理	"应用的算力需求如何被透明满足"
应用管理	标准 K8s 工作负载管理	标准 K8s + Helm 商店	三层 CRD + 应用商店 + 审核 + 拓扑部署

11.2 八维度竞品矩阵

以下从应用生命周期的八个关键维度展开对比。每个维度不仅标注"有没有"，更标注"解决到什么程度"。

维度一：多租户与权限

应用面对的问题：多个团队共用一个平台，怎么隔离？谁能在哪里部署什么？

能力	本平台	华为 IEF	阿里 ACK@Edge	腾讯 TKE Edge	Rancher
权限层级	5 层（PlatformClusterWorkspaceNodeGroupNameamespace）	2 层（IAM + 企业项目）	2 层（RAM + Namespace）	2 层（CAM + Namespace）	3 层（GlobalClusterProject）
工作空间隔离	Workspace CRD 原生支持	无	无	无	Project 级别
节点组权限	NodeGroup 级别细粒度控制	无	无	无	无
授权缓存	TTL 300s + CRD 变化触发失效	云端 IAM 缓存	云端 RAM 缓存	云端 CAM 缓存	无（每次鉴权）
审计记录	LoginRecord CRD 原生记录	依赖云审计服务	依赖云审计服务	依赖云审计服务	有（审计日志）

关键差异：云厂商的权限体系依赖各自的云 IAM 系统（华为 IAM、阿里 RAM、腾讯 CAM），本质上是"云权限"的边缘延伸。其粒度止于 Namespace 级别，无法实现"节点组 A 的运维人员不能操作节点组 B"的场景。本平台的 5 层 Scope 模型是专为边缘多团队场景设计的。

Rancher 的 3 层权限（Global → Cluster → Project）是竞品中最接近的方案，但缺少 Workspace 和 NodeGroup 两个关键层级——在大型企业的边缘场景中，这两个层级恰好对应"业务团队"和"物理站点"两个最重要的隔离边界。

维度二：边缘节点管理

应用面对的问题：硬件怎么快速接入？不同网络条件下怎么连？

能力	本平台	华为 IEF	阿里 ACK@Edge	腾讯 TKE Edge	Rancher
连接模式	双模（Direct + Proxy 反向隧道）	Agent 注册（单向）	OpenYurt Tunnel（反向隧道）	SuperEdge Tunnel（反向隧道）	Agent 注册（单向）
VCluster	支持（K3s/K0s/K8s Distro）	不支持	不支持	不支持	不支持
Condition 状态链	8 种 Condition 精确追踪	有/无 二态	Ready/NotReady	Ready/NotReady	多种 Condition
可插拔运行时	ComponentInstaller 接口	KubeEdge 固定	OpenYurt 固定	SuperEdge 固定	K3s 固定
万级节点支持	单集群 10000+ 节点	1000 节点（公开文档）	取决于集群规格	取决于集群规格	取决于集群规格

关键差异：三家云厂商各自绑定了一种边缘运行时——华为绑定 KubeEdge，阿里绑定 OpenYurt，腾讯绑定 SuperEdge。选择了云厂商，就选择了运行时。本平台通过 ComponentInstaller 可插拔接口同时支持 KubeEdge、OpenYurt 和 VCluster，运行时选择由业务场景决定，而非云厂商决定。

VCluster 支持是本平台的独特能力。对于需要完整 Kubernetes 控制面但不希望部署独立物理集群的场景（如远程小型站点），VCluster 以极低的资源开销提供了完整的集群隔离。

维度三：批量运维管理

应用面对的问题：一千台边缘服务器的环境怎么统一准备？

能力	本平台	华为 IEF	阿里 ACK@Edge	腾讯 TKE Edge	Rancher
OTA 批量运维	原生 OTA 模块（exec/read/write/playbook）	无	无	无	无
任务编排	多步骤任务链	无	无	无	无
执行结果追踪	每设备独立 DeviceResult	无	无	无	无
批量环境准备	声明式任务覆盖	需外部工具（Ansible 等）	需外部工具	需外部工具	需外部工具

关键差异：所有竞品都没有内置的批量运维能力。硬件上架后的环境准备（装驱动、配网络、初始化集群）需要借助 Ansible、SaltStack 等外部工具。本平台的 OTA 模块将批量运维纳入平台原生能力，运维人员在同一个管理界面内完成“硬件接入 → 环境准备 → 集群组建”的全流程，无需切换工具。

这在实际部署中的差异是显著的。以“50 个工厂各部署 20 台边缘服务器”为例：

- 竞品方案：需要先用 Ansible 完成驱动安装和环境配置，再登录边缘管理平台注册节点和创建集群。两个独立的工具链、两套独立的权限体系、两套独立的执行日志。
- 本平台：OTA 任务完成环境准备，集群 Controller 自动完成纳管。一个管理界面、一套权限体系、统一的执行追踪。

维度四：设备接入与 IoT

应用面对的问题：应用运行时需要与传感器等 IoT 设备交互，怎么接入？

能力	本平台	华为 IEF	阿里 ACK@Edge	腾讯 TKE Edge	Rancher
IoT 设备管理	CRD 原生 (Device/DeviceProfile)	有 (与华为 IoT 平台集成)	有 (通过 Link IoT Edge)	有限	无
设备协议	EdgeX Foundry 多协议适配	华为私有协议 + MQTT	阿里 Link 协议 + MQTT	MQTT	无
设备状态机	CRD 声明式管理	云端管控	云端管控	云端管控	无
与 K8s API 统一	管理设备和管理 Pod 用同一套 API	独立的 IoT API	独立的 IoT API	独立 API	无

关键差异：华为和阿里都有成熟的 IoT 生态，但它们的设备管理是独立于 Kubernetes 的系统——使用独立的 API、独立的控制面、独立的权限体系。本平台将 IoT 设备管理完全 CRD 化，管理设备和管理 Pod 用同一套 Kubernetes API，同一套权限体系。

这意味着：一个熟悉 kubectl 的运维人员无需学习额外的 API 就能管理 IoT 设备；一个权限策略（如“工作空间 A 的用户只能操作工作空间 A 的设备”）同时覆盖 Pod 和设备。

维度五：云边协同

应用面对的问题：云端管控面和边缘数据面之间如何高效协作？

能力	本平台	华为 IEF	阿里 ACK@Edge	腾讯 TKE Edge	Rancher
集群联邦	1000+ 集群统一管理	多集群 (依赖华为云)	单集群为主	单集群为主	多集群管理
请求路由	六层 Filter Chain (第十章)	云端代理转发	OpenYurt Tunnel	SuperEdge Tunnel	Agent 直连
镜像分发	四级分层缓存	云端 SWR 分发	阿里云 ACR	腾讯云 TCR	无内置
跨集群应用管理	Topology-Based 部署	无 (单集群部署)	无	无	Fleet (GitOps)

关键差异：云厂商的边缘服务本质上是“单集群管理 + 云端依赖”——每个边缘集群是独立管理的，跨集群的应用管理需要额外的编排。本平台的 Topology-Based 部署模型（第七章）让一个 ApplicationDeployment 声明式地管理多个集群的部署，50 个站点的推广与 5 个站点的操作步骤完全相同。

Rancher 的 Fleet 提供了基于 GitOps 的多集群部署能力，是竞品中最接近的方案。但 Fleet 的抽象层级是"集群组 + Git 仓库"，而非"应用版本 + 拓扑声明"——前者适合 DevOps 团队，后者更适合业务运维团队。

镜像分发方面，云厂商依赖各自的容器镜像服务（SWR/ACR/TCR），分发链路是"云端仓库 → 边缘节点"的两级结构。本平台的四级分层缓存（Registry → ClusterRegistry → Repository → Node）在弱网环境下的可用性更高——即使云端网络断开，边缘站点本地的缓存仍然可用。

维度六：离线自治

应用面对的问题：边缘网络断开了，应用还能继续运行吗？

能力	本平台	华为 IEF	阿里 ACK@Edge	腾讯 TKE Edge	Rancher
离线自治	三种方案（KubeEdge/OpenYurt/VCluster）	KubeEdge（单方案）	OpenYurt（单方案）	SuperEdge（单方案）	K3s（完整控制面）
断网后应用持续运行	支持	支持	支持	支持	支持（独立集群）
断网后新应用部署	支持（本地缓存 + 镜像缓存）	有限	有限	有限	支持（独立控制面）
方案灵活性	按场景选择最佳方案	绑定 KubeEdge	绑定 OpenYurt	绑定 SuperEdge	绑定 K3s

关键差异：离线自治是所有边缘平台的基础能力，各家方案在"断网后应用持续运行"这个场景上差异不大。但差异出现在方案灵活性上。

云厂商各自绑定了一种离线方案——华为 KubeEdge 适合资源极度受限的嵌入式场景，但对标准服务器级边缘而言过于轻量；阿里 OpenYurt 适合标准边缘，但不支持嵌入式设备。选择了云厂商就等于选择了离线策略，场景不匹配时没有备选。

本平台通过 ComponentInstaller 可插拔接口同时支持三种离线方案，运维人员可以为不同的边缘站点选择最合适的方案——嵌入式网关用 KubeEdge，工厂服务器用 OpenYurt，独立站点用 VCluster。

维度七：应用管理

应用面对的问题：应用怎么打包？怎么审核？怎么跨团队共享？怎么批量部署？

能力	本平台	华为 IEF	阿里 ACK@Edge	腾讯 TKE Edge	Rancher
应用抽象	三层 CRD（ApplicationVersionDeployment）	标准 K8s 工作负载	标准 K8s 工作负载	标准 K8s 工作负载	标准 K8s + Helm
应用商店	原生支持 + 审核流程	无	无	无	Helm 商店（无审核）

能力	本平台	华为 IEF	阿里 ACK @Edge	腾讯 TKE Edge	Rancher
审核流程	ReviewPhase (PendingApproved/Rejected)	无	无	无	无
可插拔 Provisioner	3 种 (Workload/Helm/Model)	无	无	无	无
AI 模型部署	Model Provisioner 原生支持	通过 ModelArts 集成	通过 PAI 集成	通过 TIONE 集成	无
Topology-Based 部署	一个声明管理多站点	每站点独立部署	每站点独立部署	每站点独立部署	Fleet GitOps
版本管理	ApplicationVersion 独立 CRD	无 (依赖镜像 Tag)	无	无	Helm Chart 版本
Finalizer 保护	存在活跃部署时阻止删除版本	无	无	无	无

关键差异：这是本平台与所有竞品之间最本质的差距。

竞品的应用管理层级是 K8s 原生的 **Deployment/StatefulSet/DaemonSet**——这是"工作负载"抽象，不是"应用"抽象。一个工作负载不等于一个应用——一个应用可能包含多个微服务（多个 Deployment），可能有多个版本（需要独立追踪），可能需要部署到多个边缘站点（需要拓扑管理），可能需要在不同团队间共享（需要审核机制）。

本平台的 Application/ApplicationVersion/ApplicationDeployment 三层 CRD 体系，将"应用"提升为与"集群""节点"同地位的一等公民资源。这不是功能上的增量——这是抽象层级上的跃迁。

具体而言，以"将一个质检模型从试点站点推广到 50 个工厂"为例：

步骤	竞品方案	本平台
1. 打包应用	手动构建镜像 + 手写 YAML	选择 Provisioner 类型，填写配置
2. 版本管理	依赖镜像 Tag 约定	ApplicationVersion CRD 独立追踪
3. 审核上架	无机制，靠口头约定或外部流程	ReviewPhase (PendingApproved)，平台管理员审核
4. 部署到 50 个站点	50 次独立部署操作，或编写自动化脚本	1 个 ApplicationDeployment，50 个 Topology 声明
5. 版本升级	50 次独立更新操作	修改 ApplicationDeployment 引用的版本，自动滚动更新
6. 状态监控	分别查看 50 个站点	统一视图，每个 Topology 独立追踪

操作步骤差异：竞品方案的操作量与站点数成正比（ $O(N)$ ），本平台的操作量与站点数无关（ $O(1)$ ）。

维度八：异构计算

应用面对的问题：我的 AI 模型需要 GPU，但边缘有各种不同厂商的卡，怎么办？

能力	本平台	华为 IEF	阿里 ACK @Edge	腾讯 TKE Edge	Rancher
GPU 厂商支持	9 大厂商	昇腾 + NVIDIA	NVIDIA + 部分	NVIDIA + 部分	无内置
GPU 虚拟化	双模（MIG 硬切分 + HAMi-Core 软切分）	部分（昇腾 vNPU）	部分（cGPU）	无	无
设备抽象	四层 CRD（DeviceModelComputeTemplateResourcePoolNodeConfig）	无统一抽象	无	无	无
多精度定义	FP32/FP16/BF16/FP8	无	无	无	无
超卖配比	ComputeTemplate 精确控制	无	无	无	无
调度策略	双维（节点 Binpack + GPU Spread）	单维度	单维度	无	无
资源池	shared/dedicated/reserved 三种策略	无	无	无	无
国产 GPU 深度适配	四层 CRD 透明适配	昇腾深度适配	有限	有限	无

关键差异：异构算力管理是本平台最突出的技术优势，没有之一。

云厂商的 GPU 支持集中在自家生态——华为深度适配昇腾，阿里和腾讯主要支持 NVIDIA。

9 大 GPU 厂商的统一建模和调度，在已知的公开产品中，只有本平台实现了完整覆盖。

更关键的差异在于抽象深度。竞品的 GPU 管理停留在“识别并调度”层面——发现节点上有 GPU，分配给请求 GPU 的 Pod。本平台的四层 CRD 架构实现了从“硬件规格定义”（DeviceModel）到“算力配比”（ComputeTemplate）到“资源池管理”（ResourcePool）到“节点级配置”（NodeConfig）的完整管线。

这个管线的价值在实际场景中体现为：

- 同一个节点上同时插着 NVIDIA A100 和昇腾 310P——NodeConfig 为每个厂商独立配置 Device Plugin 和 ComputeTemplate，互不干扰
- 一张 A100 同时服务 5 个推理任务——ComputeTemplate 定义软切分策略，HAMi-Core 自动执行虚拟化
- 应用定义不包含任何 GPU 厂商信息——只声明 `core: 30, memory: 8192`，调度器自动匹配

竞品方案中，应用开发者需要知道目标节点上有什么 GPU，需要在 YAML 中写入特定的 Device Plugin 资源请求（如 `nvidia.com/gpu: 1`）。更换 GPU 厂商意味着修改应用配置。本平台则实现了第二章提出的核心承诺：换 GPU 不改应用。

11.3 深度对比：逐家分析

vs 华为 IEF

华为 IEF 的优势：

- 与华为云生态深度集成（ModelArts AI 训练 + IEF 推理部署是成熟的流水线）
- KubeEdge 是 CNCF 毕业项目，社区生态成熟
- 昇腾硬件深度适配，芯片到平台的垂直整合

华为 IEF 的局限：

- 云厂商锁定：IEF 是华为云服务，依赖华为云账号、IAM、SWR（镜像仓库）等基础设施。私有化部署受限。
- 应用管理能力薄弱：没有应用商店、没有审核流程、没有 Topology-Based 部署。应用管理停留在 K8s 原生工作负载层面。
- 运行时绑定：绑定 KubeEdge，标准服务器级边缘场景缺乏更合适的选择。
- 多租户能力有限：依赖华为云 IAM 的 2 层权限，缺少 Workspace/NodeGroup 级别的细粒度隔离。

选择建议：如果企业已全面采用华为云 + 昇腾硬件，IEF 是生态内的自然选择。如果需要多云或私有化部署，或者边缘存在多厂商 GPU，IEF 的适配能力不足。

vs 阿里 ACK@Edge

阿里 ACK@Edge 的优势：

- OpenYurt 是 CNCF Sandbox 项目，社区活跃度高

- 阿里云容器服务 ACK 是国内最成熟的托管 K8s 服务之一
- 与阿里云 AI/IoT 生态集成（PAI、Link IoT Edge）

阿里 ACK@Edge 的局限：

- 单集群为主：ACK@Edge 的核心模型是“一个托管集群 + 多个边缘节点池”。跨集群的应用管理需要额外编排。
- 无应用商店和审核：应用管理依赖标准 K8s 工作负载 + Helm。
- GPU 虚拟化有限：cGPU 方案主要面向 NVIDIA，对国产 GPU 的虚拟化支持有限。
- 云端依赖：控制面运行在阿里云端，对云网络的依赖性高。

选择建议：如果企业的边缘节点都在同一个逻辑集群内（如 CDN 边缘节点），ACK@Edge 的单集群模型足够。如果需要管理分布在不同地理位置的多个独立集群，且每个集群有不同的应用组合，ACK@Edge 的能力不足。

vs 腾讯 TKE Edge

腾讯 TKE Edge 的优势：

- SuperEdge 社区背景，边缘自治能力成熟
- 腾讯云 TKE 容器服务生态
- 边缘隧道通信优化

腾讯 TKE Edge 的局限：

- GPU 管理最弱：在四个竞品中，TKE Edge 的异构计算管理能力最有限，无内置 GPU 虚拟化方案。
- 应用管理标准化：完全依赖 K8s 原生工作负载管理，无应用层面的扩展。
- 集群管理单一：以单集群管理为主，多集群场景能力有限。
- IoT 能力有限：相较华为和阿里，腾讯在 IoT 设备管理方面的生态积累较少。

选择建议：如果企业的边缘场景以标准容器工作负载为主，不涉及 GPU 和 IoT，TKE Edge 是一个轻量的选择。如果涉及 AI 推理和异构计算，TKE Edge 的能力缺口明显。

vs Rancher

Rancher 的优势：

- 真正的多集群管理能力（不局限于单个云厂商）
- K3s 是最成熟的轻量级 K8s 发行版
- Fleet 提供了 GitOps 风格的多集群部署
- 开源 + 企业版，部署灵活性高
- 社区生态丰富，Helm 商店集成

Rancher 的局限：

- 无 GPU 管理：Rancher 完全不涉及 GPU 虚拟化、设备抽象和算力调度。对于 AI 推理场景，需要自行集成 HAMi、MIG 等方案。
- 无 OTA 批量运维：硬件环境准备需要外部工具。
- 无 IoT 设备管理：不涉及物联网设备的接入和管理。
- 应用商店无审核：Helm 商店提供了应用分发能力，但没有审核流程——任何人都可以发布，缺少企业级的质量管控。
- 权限模型较浅：3 层权限（Global → Cluster → Project）缺少 Workspace 和 NodeGroup 的细粒度控制。

选择建议：Rancher 是最接近本平台定位的竞品——都是独立部署的多集群管理平台。如果边缘场景以标准容器工作负载为主，不涉及 GPU 和 IoT，Rancher 是一个成熟的选择。但一旦涉及异构算力管理、应用商店审核、批量运维，Rancher 需要大量的外部集成和定制开发。

11.4 总结：产品哲学的差异

以上八个维度的对比，归根结底反映的是产品哲学的差异：

竞品的思维模型：

基础设施 → 你来管理 → 应用自己想办法
"这是集群，这是节点，这是 GPU" → "你自己部署应用吧"

本平台的思维模型：

应用 → 你只需声明需求 → 平台自动满足
"你的应用需要什么？" → "算力、部署、监控，平台全部处理"

用一个表格总结每个维度的竞争态势：

维度	本平台优势程度	说明
多租户与权限	显著优势	5 层 vs 2-3 层，Workspace/NodeGroup 是独有层级
边缘节点管理	中等优势	可插拔运行时 + VCluster 是差异点
批量运维	独有能力	OTA 是竞品完全不具备的
设备接入（IoT）	中等优势	CRD 原生化 vs 独立 IoT 系统
云边协同	显著优势	四级镜像缓存 + Topology-Based 部署
离线自治	中等优势	三方案灵活选择 vs 单方案绑定
应用管理	决定性优势	三层 CRD + 应用商店 + 审核 + Provisioner
异构计算	决定性优势	9 厂商 + 四层 CRD + 双模虚拟化

两个"决定性优势"维度——应用管理和异构计算——恰好对应了本白皮书的核心论点：

- 应用管理是"价值重心从硬件转向应用"的直接体现
- 异构计算是"应用无需感知硬件差异"的技术保障

这不是巧合。当产品哲学从"面向硬件"转向"面向应用"时，在这两个维度上投入最深，是逻辑上的必然。

11.5 适用场景选择建议

场景	推荐方案	理由
已全面使用华为云 + 昇腾，不涉及多厂商 GPU	华为 IEF	生态内垂直整合最优
单集群边缘 + 阿里云生态 + 不需要应用商店	阿里 ACK@Edge	托管集群，运维成本低
纯容器工作负载 + 无 GPU + 无 IoT	腾讯 TKE Edge 或 Rancher	足够，无需额外复杂度
多厂商 GPU + AI 推理 + 多团队共享	本平台	异构算力 + 应用商店 + 5 层权限
多集群 + 批量运维 + 应用全生命周期	本平台	OTA + Topology-Based 部署 + 版本管理
私有化部署 + 不依赖云厂商	本平台或 Rancher	独立部署，不受云厂商锁定
国产化替代要求（全栈国产）	本平台	9 大国产 GPU 原生适配

11.6 本章小结

本章从应用生命周期的八个维度对比了本平台与四家主要竞品的能力差异。

竞品的故事是：我们支持什么硬件、管理多少节点、提供什么运行时。

我们的故事是：你的应用在边缘的每个阶段都有解法——打包、审核、部署、运行、监控——无论底层硬件怎么变。

两个决定性优势维度（应用管理和异构计算）不是偶然的技术突破，而是产品哲学“面向应用”的必然结果。当平台的设计起点是“应用如何在边缘持续释放价值”，而不是“如何管理更多硬件”时，技术投入的方向自然会聚焦在让应用更容易打包、更快速部署、更透明地使用算力、更安全地跨团队共享。

下一章：第十二章 — 典型行业场景

第十二章：典型行业场景 —— 应用价值的行业映射

每个场景以应用故事展开：这个行业的核心应用是什么 → 应用面对什么挑战 → 平台如何解决 → 量化收益。

引言：从技术能力到行业价值

前面十一章建立了完整的技术论证——从应用价值立论（第二章），到基础设施、算力、组织安全、供应链、部署、运行、可观测七个生命周期阶段，再到架构纵深和竞品对比。

但企业决策者最终关心的不是“平台有什么技术”，而是“这些技术在我的行业中能解决什么问题”。

本章选取六个代表性行业场景，每个场景都围绕一个核心应用展开。这些场景不是凭空构想的“典型案例”，而是从平台的功能设计反推出的真实需求映射——每一个场景都能精确对应到前面章节介绍的具体技术能力。

12.1 智能制造：质检 AI 从一条产线到全国工厂

行业痛点

一家汽车零部件制造企业在总部工厂开发了一套基于深度学习的质检模型（YOLOv8），在产线上部署后效果显著——缺陷检出率提升到 99.2%，人工质检成本降低 60%。

但企业在全国有 47 家分工厂，每家工厂有 3-8 条产线。将这套质检系统推广到全国时，企业遇到了三个核心问题：

问题	传统方式	影响
部署效率	每个工厂派驻工程师现场安装调试，每个工厂需要 3-5 天	全部推广需要 12-18 个月
版本同步	模型更新后逐工厂人工替换，版本碎片化严重	30% 的工厂运行着 3 个月前的旧模型
硬件差异	早期工厂用 NVIDIA T4，新工厂采购昇腾 310P，部署配置完全不同	运维团队需要维护两套部署方案

平台解法

第一步：算力抽象，消除硬件差异（第四章）

通过 DeviceModel CRD 统一建模 NVIDIA T4 和昇腾 310P，质检应用的部署配置不包含任何硬件相关参数：

```
应用声明：需要 GPU 算力 8 TFLOPS
├── 老工厂 (NVIDIA T4) → DeviceModel 自动匹配 → nvidia.com/gpu
└── 新工厂 (昇腾 310P) → DeviceModel 自动匹配 → ascend-310p
```

同一份应用定义，在两种硬件上自动适配，开发者无需感知底层 GPU 差异。

第二步：应用商店，标准化供应链（第六章）

质检模型通过 Provisioner 模式打包为标准 Application：

- 1. 研发团队 在总部 Workspace 开发质检应用，通过 Model Provisioner 打包（支持 PyTorch 模型格式）
- 2. 提交审核：ApplicationVersion 的 ReviewPhase 设置为 "Pending"，提交平台管理员审核
- 3. 审核通过：审核通过后出现在全域应用商店，所有工厂的运维人员可见

第三步：Topology-Based 部署，一键推广（第七章）

一个 ApplicationDeployment 管理 47 家工厂的部署：

```
ApplicationDeployment: quality-inspection-v3.2
├─ Topology: factory-beijing (NodeGroup: beijing-plant) → 3 replicas
├─ Topology: factory-shanghai (NodeGroup: shanghai-plant) → 5 replicas
├─ Topology: factory-guangzhou (NodeGroup: guangzhou-plant) → 4 replicas
└─ ... (47 个 Topology, 每个对应一家工厂的 NodeGroup)
```

版本更新时，修改 ApplicationDeployment 引用的 ApplicationVersion 即可自动滚动更新所有工厂，无需逐一操作。

第四步：离线自治，断网不停产（第八章）

工厂网络偶尔因设备维护或网络故障断连。通过 OpenYurt 的离线自治能力，质检应用在断网期间继续运行：

- 边缘节点的 API 缓存保证 Pod 不被驱逐
- 网络恢复后状态自动同步到中心
- 质检结果本地缓存，联网后批量上传

量化收益

指标	传统方式	使用平台	提升
全国推广周期	12-18 个月	2-4 周	85%
版本一致率	70%（版本碎片化）	100%（统一管理）	全量统一
运维人力	每工厂 1 人现场	中心团队 2 人远程	95%
GPU 适配成本	两套部署方案	一份应用定义	减半

12.2 智慧城市：多部门应用的安全共存

行业痛点

一个地级市的智慧城市项目涉及多个政府部门：公安局（视频分析）、城管局（市容监控）、交通局（信号优化）、环保局（排放监测）。各部门的应用运行在同一批边缘服务器上，但存在严格的安全隔离要求：

- 数据隔离：公安的视频数据不能被城管访问
- 资源隔离：环保的批量计算任务不能抢占交通的实时分析资源
- 权限隔离：每个部门只能管理自己的应用，看到自己的监控数据
- 审计合规：所有操作必须可追溯，满足等保要求

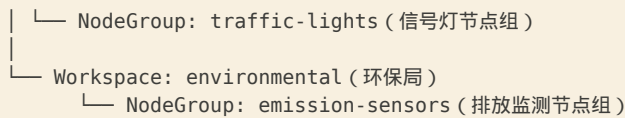
传统做法是为每个部门独立搭建一套边缘平台——成本翻倍，资源利用率不到 30%。

平台解法

多租户组织映射（第五章）

平台的五层权限模型精确映射政府组织架构：

```
Platform (市级平台管理员)
├── Workspace: public-security (公安局)
│   ├── NodeGroup: district-A-cameras (A区视频节点组)
│   └── NodeGroup: district-B-cameras (B区视频节点组)
├── Workspace: urban-management (城管局)
│   └── NodeGroup: city-monitoring (市容监控节点组)
└── Workspace: transportation (交通局)
```



每个 Workspace 是一个完整的隔离域：独立的应用列表、独立的部署空间、独立的监控视图。Workspace 之间通过 K8s Namespace 隔离实现资源边界。

细粒度权限控制

- 公安局运维人员：只能操作 `public-security` Workspace 下的应用，无法看到其他部门
- 平台管理员：可以查看所有 Workspace 的资源使用情况，但不能进入 Workspace 操作应用
- 审计人员：只读权限，通过 LoginRecord CRD 审查所有部门的操作日志

多租户监控隔离（第九章）

Recording Rules 按 Workspace 维度预聚合指标，每个部门的监控仪表盘只展示自己的数据：

- 公安局看到的是视频分析应用的 GPU 利用率和推理延迟
- 交通局看到的是信号优化服务的 CPU 使用率和响应时间
- 平台管理员看到的是全局资源分布和告警汇总

量化收益

指标	独立部署（4套）	统一平台	节省
硬件成本	4× 服务器集群	1× 共享集群	60-70%
运维团队	每部门 2 人	中心 3 人 + 部门各 1 人	45%
资源利用率	25-30%	65-75%	2.5x
合规审计	4 套独立审计	统一 LoginRecord	单一审计入口

12.3 电信 5G MEC：万级站点的应用统一管理

行业痛点

一家运营商在全国部署了 5000+ 个 5G MEC (Multi-access Edge Computing) 站点。每个站点运行着 CDN 缓存、视频转码、AR 渲染等边缘应用。核心挑战：

- 规模：5000+ 站点的应用需要统一管理，不可能逐站点手动操作
- 异构：站点硬件配置不同（高配站点 4 张 GPU，低配站点纯 CPU）
- 时效：新应用上线需要在 24 小时内推送到所有站点
- 可观测：需要实时掌握每个站点的应用运行状态

平台解法

大规模集群管理（第三章 + 第十章）

通过多集群架构，每个 MEC 站点注册为一个 Cluster CRD。中心控制面通过 Multicluster Dispatcher 路由请求到任意站点：

```
graph TD; CCP[中心控制面] --- C1[Cluster: mec-bj-001 (北京朝阳站点)]; CCP --- C2[Cluster: mec-bj-002 (北京海淀站点)]; CCP --- C3[Cluster: mec-sh-001 (上海浦东站点)]; CCP --- C4["... (5000+ Cluster CRD)"]
```

中心控制面

- Cluster: mec-bj-001 (北京朝阳站点)
- Cluster: mec-bj-002 (北京海淀站点)
- Cluster: mec-sh-001 (上海浦东站点)
- ... (5000+ Cluster CRD)

每个 Cluster 通过 Condition 状态机跟踪组件安装状态（VCluster / 系统核心 / 服务同步），Controller 自动完成新站点的初始化。

批量应用分发（第七章）

CDN 缓存应用通过 ApplicationDeployment 的 Topology 列表实现批量分发。新应用上线时，运维人员只需创建一个 ApplicationDeployment，将 5000+ 个 NodeGroup 列为 Topology——Controller 自动为每个站点创建对应的 K8s Deployment。

全局可观测（第九章）

monitoring-service 的并行查询架构支撑大规模指标查询。291+ 预定义指标覆盖每个站点的 CPU、内存、网络、GPU 状态。四级告警系统（critical/high/middle/low）将异常站点第一时间推送给运维团队。

全局拓扑可视化提供从"全国地图 → 省级 → 站点 → 节点 → Pod"的逐层下钻能力，运维人员可以在一个界面中从全局视角定位到具体问题 Pod。

量化收益

指标	传统方式	使用平台	提升
新应用上线周期	2-4 周（逐站点推送）	24 小时内（批量部署）	90%
运维人均管理站点	50-100 个	500-1000 个	10x
故障定位时间	2-4 小时（逐站点排查）	15-30 分钟（全局拓扑下钻）	85%
版本一致性	60-70%（部分站点更新失败）	95%+（状态机跟踪失败自动重试）	近乎 100%

12.4 金融网点：离线场景下的交易连续性

行业痛点

一家全国性商业银行有 8000+ 家网点，每家网点运行着柜面交易系统、反欺诈模型、客户画像服务等应用。金融行业的核心要求：

- 交易连续性：网络中断不能影响柜面业务，客户等在柜台前不允许系统不可用
- 数据安全：交易数据需要本地加密存储，网络恢复后安全回传
- 合规审计：每一笔操作都需要记录，满足监管要求
- 独立运行：每个网点必须具备独立运行能力，不依赖总行中心系统实时在线

平台解法

离线自治保障（第八章）

每个网点集群通过 OpenYurt 或 VCluster 实现边缘自治。EdgeNode Controller 自动为边缘节点启用自治模式：

```
// 自动添加离线自治注解  
node.Annotations["node.beta.openyurt.io/autonomy"] = "true"
```

网络中断时：

- Pod 不被驱逐：本地 API 缓存保证 kubelet 持续维护 Pod 生命周期
- 交易数据本地持久化：PVC 挂载到本地存储，交易记录不丢失
- 网络恢复自动同步：Controller 在下一个调谐周期自动同步状态到中心

多层安全防护（第五章）

- 网点维度隔离：每个网点是一个独立的 NodeGroup，网点之间互不可见
- 角色分离：总行管理员管理应用商店和审核，分行运维员管理部署，网点柜员只有使用权
- 操作审计：LoginRecord CRD 记录所有登录和操作行为，满足银监会审计要求

可控的版本推送（第六章 + 第七章）

金融系统的版本更新需要经过严格的审核流程。应用商店的审核机制（ReviewPhase: Pending → Approved）确保只有经过测试和审批的版本才能推送到网点。版本推送通过 ApplicationDeployment 实现灰度发布——先推送到 10 家试点网点，验证通过后再全量推送。

量化收益

指标	传统方式	使用平台	保障
网络中断恢复 RTO	30 分钟-2 小时（需要人工干预）	0 分钟（自治运行，无感知）	交易连续
版本推送周期	4-6 周（逐网点人工）	1-2 周（灰度 + 批量）	60%
安全审计覆盖	80%（部分操作缺失）	100%（CRD 原生记录）	满足监管
网点 IT 人力	每网点 0.5 人	远程管理，网点 0 人	成本归零

12.5 能源/电力：异构设备上的监控应用快速部署

行业痛点

一家电力公司在全国有 300+ 座变电站，每座变电站部署了多种监控设备：高清摄像头（视频巡检）、红外热成像仪（设备温度监测）、振动传感器（变压器健康监测）、环境传感器（温湿度、SF6 浓度）。

核心挑战：

- 设备异构：每座变电站的传感器品牌和协议不同（Modbus、OPC-UA、MQTT）
- 部署分散：变电站分布在偏远地区，现场维护成本极高
- 实时要求：设备温度异常需要秒级告警，延迟可能导致事故

平台解法

IoT 设备云原生化（第八章）

通过 CRD 化的 IoT 设备管理，将不同协议的传感器统一纳入 K8s 管理体系：

```
DeviceProfile (设备模板)
├── 温度传感器 Profile: Modbus 协议, 读取寄存器地址 40001
├── 振动传感器 Profile: OPC-UA 协议, 节点 ns=2;s=Vibration
└── 摄像头 Profile: RTSP 协议, 流地址配置

Device (设备实例)
├── substation-001-temp-sensor-01 → 运行中
├── substation-001-vibration-01 → 运行中
└── substation-001-camera-01 → 已锁定 (维护中)
```

设备管理和 Pod 管理使用同一套 K8s API——`kubectl get devices` 和 `kubectl get pods` 的操作体验一致。

监控应用快速部署（第三章 + 第七章）

OTA 系统批量将边缘服务器纳入平台，Topology-Based 部署将监控应用推送到所有变电站：

1. OTA 批量纳管：300+ 座变电站的边缘服务器通过系统升级和配置管理快速接入
2. 镜像预同步：AI 推理镜像通过分层镜像同步提前缓存到边缘，部署时无需在线拉取
3. 拓扑部署：一个 ApplicationDeployment 覆盖 300+ 个变电站的 NodeGroup

GPU/NPU 监控闭环（第四章 + 第九章）

变电站的 AI 推理应用运行在 GPU 上，平台的设备监控指标（GPU 温度、功耗、利用率）与业务告警形成闭环：

- GPU 温度异常 → 触发 critical 告警 → 自动降低推理频率
- GPU 利用率持续低于 10% → 触发 middle 告警 → 提示资源过度分配
- 设备健康状态异常 → 触发 high 告警 → 通知现场维护

量化收益

指标	传统方式	使用平台	提升
设备接入周期	每站 3-5 天（现场调试协议）	半天（CRD 声明 + Profile 模板）	80%
监控应用部署	每站 2-3 天	批量 1 天覆盖全部	95%
设备故障响应	4-8 小时（人工巡检发现）	秒级告警	实时
运维出差成本	每年 200+ 次现场	90% 远程解决	90%

12.6 AI 推理服务：算法镜像的应用商店化分发

行业痛点

一家 AI 算法公司为多个行业客户提供推理服务（人脸识别、车牌识别、行为分析等）。每个客户有自己的边缘站点，运行不同组合的算法模型。核心挑战：

- 算法版本管理：同一算法有多个版本（通用版 / 行业定制版 / 轻量版），客户需要选择适合自己硬件的版本
- 部署自助化：客户希望自主选择 and 部署算法，而不是每次都依赖算法公司派工程师
- 模型更新：算法迭代频繁（每月 1-2 个新版本），客户需要方便地升级
- 多硬件适配：客户硬件从 NVIDIA T4 到昇腾 310P 到寒武纪 MLU370 不等

平台解法

应用商店生态（第六章）

算法公司作为应用开发者，在自己的 Workspace 中开发和发布算法应用：

```
Application: face-recognition (人脸识别)
├─ ApplicationVersion v3.1-general → Model Provisioner (通用版, PyTorch)
├─ ApplicationVersion v3.1-lite → Model Provisioner (轻量版, ONNX)
├─ ApplicationVersion v3.0-ascend → Model Provisioner (昇腾定制版)
├─ ApplicationVersion v2.8-legacy → Model Provisioner (兼容旧版)
└─ ReviewPhase: Approved (已审核上架)
```

客户在应用商店中浏览可用算法，查看版本说明和硬件要求，选择适合自己的版本一键部署。

算力透明适配（第四章）

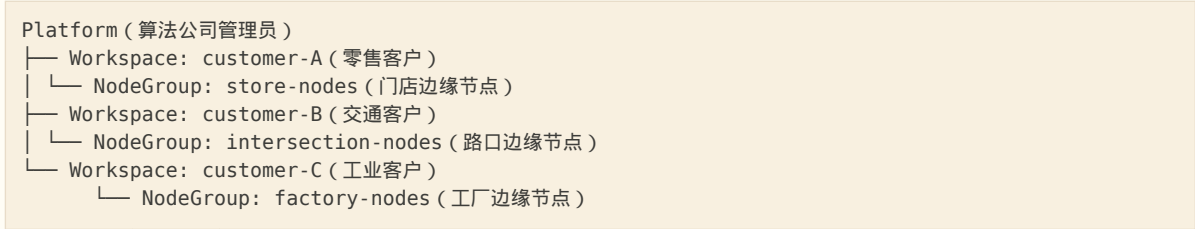
每个算法版本的 ModelConfig 声明算力需求，平台自动匹配客户边缘的 GPU 硬件：

- 通用版（v3.1-general）：需要 FP16 算力 ≥ 20 TFLOPS → NVIDIA T4 或更高
- 轻量版（v3.1-lite）：需要 FP16 算力 ≥ 8 TFLOPS → 可运行在低端 GPU 上
- 昇腾定制版（v3.0-ascend）：专为 Ascend 310P 优化 → 自动匹配昇腾节点

客户无需了解 GPU 规格，平台根据 DeviceModel 和 ComputeTemplate 自动完成匹配和调度。

多客户隔离（第五章）

每个客户一个 Workspace，算法公司作为平台管理员管理应用商店：



客户之间完全隔离：A 客户看不到 B 客户部署了哪些算法，也无法访问 B 客户的监控数据。

量化收益

指标	传统方式	使用平台	提升
客户部署周期	1-2 周（需要算法公司支持）	自助部署，30 分钟内	95%
算法版本管理	Excel 表 + 邮件沟通	应用商店 + 版本历史	系统化
多客户支持人力	每 10 客户 1 名部署工程师	1 名平台管理员管理 50+ 客户	5x
模型更新周期	2-4 周（逐客户推送）	审核通过后批量推送	75%

12.7 本章小结：六个行业，一个模式

回顾六个行业场景，尽管行业差异巨大，但平台解决问题的模式是一致的：

阶段	对应能力	章节
硬件接入	OTA 批量纳管 + 集群自动初始化	第三章
算力适配	DeviceModel + ComputeTemplate 硬件抽象	第四章
组织隔离	Workspace + NodeGroup + 5 层权限	第五章
应用上架	Provisioner + 应用商店 + 审核流程	第六章
应用分发	Topology-Based 部署 + 批量版本切换	第七章
离线运行	EdgeNode 自治 + 本地 API 缓存	第八章
健康监控	291+ 指标 + 多租户隔离 + 全局拓扑	第九章

这正是第二章核心论点的行业验证：平台围绕应用生命周期构建，每个阶段的能力在不同行业中复用，只是应用本身不同。制造业的核心应用是质检模型，金融业是交易系统，电信业是 CDN 缓存——但它们"从开发到上架、从上架到部署、从部署到运行"的生命周期路径是相同的。

一个面向应用的平台，自然适配所有需要应用的行业。